

Free-Energy Machine for Combinatorial Optimization

Zi-Song Shen,^{1,2,*} Feng Pan,^{1,*} Yao Wang,^{3,*} Yi-Ding Men,^{2,4} Wen-Biao Xu,^{2,4} Man-Hong Yung,³ and Pan Zhang^{1,4,†}

¹*CAS Key Laboratory for Theoretical Physics, Institute of Theoretical Physics,
Chinese Academy of Sciences, Beijing 100190, China*

²*School of Physical Sciences, University of Chinese Academy of Sciences, Beijing 100049, China*

³*2012 lab, Huawei Technologies Co., Ltd., Shenzhen 518129, China*

⁴*School of Fundamental Physics and Mathematical Sciences,
Hangzhou Institute for Advanced Study, UCAS, Hangzhou 310024, China*

(Dated: December 13, 2024)

Finding optimal solutions to combinatorial optimization problems is pivotal in both scientific and technological domains, within academic research and industrial applications. A considerable amount of effort has been invested in the development of accelerated methods that leverage sophisticated models and harness the power of advanced computational hardware. Despite the advancements, a critical challenge persists, the dual demand for both high efficiency and broad generality in solving problems. In this work, we propose a general method, Free-Energy Machine (FEM), based on the ideas of free-energy minimization in statistical physics, combined with automatic differentiation and gradient-based optimization in machine learning. The algorithm is flexible, solving various combinatorial optimization problems using a unified framework, and is efficient, naturally utilizing massive parallel computational devices such as graph processing units (GPUs) and field-programmable gate arrays (FPGAs). We benchmark our algorithm on various problems including the maximum cut problems, balanced minimum cut problems, and maximum k -satisfiability problems, scaled to millions of variables, across both synthetic, real-world, and competition problem instances. The findings indicate that our algorithm not only exhibits exceptional speed but also surpasses the performance of state-of-the-art algorithms tailored for individual problems. This highlights that the interdisciplinary fusion of statistical physics and machine learning opens the door to delivering cutting-edge methodologies that will have broad implications across various scientific and industrial landscapes.

INTRODUCTION

Combinatorial optimization problems (COPs) are prevalent across a broad spectrum of fields, from science to industry, encompassing disciplines such as statistical physics, operations research, and artificial intelligence, among many others [1]. However, most of these problems are non-deterministic polynomial-time-hard (NP-hard) [2], posing significant computational challenges. It is widely believed that exact algorithms are unlikely to provide efficient solutions unless NP=P. Consequently, a plethora of classical algorithms, including simulated annealing [3] and various local search algorithms [4–6], have been devised and widely adopted for approximately solving the problem in practical settings. It is worth noting that most of them realize series computations and were designed for CPUs. While some special problems with certain structures can be solved efficiently [7], most of the practical hard problems remain intractable by the standard tools.

In recent years, due to the remarkable emergence of massively parallel computational power given by Graphics Processing Units (GPUs) and Field-Programmable Gate Arrays (FPGAs), there has been a growing expectation for novel approaches. Many novel approaches have been developed. Notable examples include the Simulated Coherent Ising Machine (SimCIM) [8], Noise Mean Field Annealing (NMFA) [9] and the Simulated Bifurcation Machines (SBM) [10, 11], etc. They are originally inspired by simulating the mean-field dynamics of programmable specialized hardware devices called Ising machines [12–19], and have been shown to achieve even higher performance com-

pared to the hardware version [9, 11]. In addition to their high accuracy, a significant signature of the algorithms is their ability to simultaneously update variables which enables effective acceleration to address large-scale problems using GPUs and FPGAs [10, 11]. However, these algorithms have their applicability predominantly limited to quadratic unconstrained binary optimization (QUBO) problems [20], or Ising formulations [21]. This limitation becomes evident when addressing COPs inherently characterized by non-quadratic (e.g., higher-order p -spin glasses [22] for Boolean k -satisfiability (k -SAT) problems [23]) or non-binary features (e.g., the Potts glasses [24], coloring problems [25], and community detections [26]). To adapt these more complex problem structures to Ising formulations, additional conversion steps and significant overhead are required. They complicate the optimization problem and make them more challenging to solve when compared to the original formulations.

In this work, we propose to address the demands of combinatorial optimization in terms of generality, performance, and speed, drawing inspiration from statistical physics. Specifically, our approach is distinguished by its ability to solve general COPs without relying on Ising formulations, setting it apart from existing methods. From the statistical physics perspective, the cost function of a COP plays the role of energy (also the free energy at zero temperature) of the spin glass system. Solving COPs amounts to finding configurations minimizing the energy [21, 27]. However, directly searching for configurations minimizing the energy is difficult, because the landscape of energy is rugged, and searching may frequently be trapped by local minima of energy. In spin glass theory of statistical physics, this picture is de-

scribed by the theory of replica symmetry breaking, which uses the organization of fixed points of mean-field solutions to characterize the feature of the rugged landscapes [28].

Here, inspired by replica symmetry breaking, we propose a general method based on minimizing variational free energies at a temperature that gradually annealed from a high value to zero. The free energies are functions of replicas of variational mean-field distributions and are minimized using gradient-based optimizers in machine learning. We refer to our method as *Free-Energy Machine*, abbreviated as FEM. The approach incorporates two major features. First, the gradients of replicas of free energies are computed via automatic differentiation in machine learning, making it generic and immediately applied to various COPs. Second, the variational free energies are minimized by utilizing recognized optimization techniques such as Adam [29] developed from the deep learning community. Significantly, all replicas of mean-field probabilities are updated in parallel, thereby leveraging the computational power of GPUs for efficient execution and facilitating a substantial speed-up in solving large-scale problems. The pictorial illustration of our algorithm is shown in Fig. 1.

We have evaluated FEM using a wide spectrum of combinatorial optimization challenges, each with unique features. This includes tackling the maximum cut (MaxCut) problem, fundamentally represented by the two-state Ising spin glasses; addressing the q -way balanced minimum cut (bMin-Cut) problem, which aligns with the Potts glasses and encapsulates COPs involving more than two states; and solving the maximum k -satisfiability (Max k -SAT) problem, indicative of problems characterized by multi-body interactions. We measured FEM's efficacy by comparing it with the leading algorithms tailored to each specific problem. The comparative analysis reveals that the proposed approach not only competes well but in many instances outperforms these specialized, cutting-edge solvers across the board. This demonstrates FEM's exceptional adaptability and superior performance, both in terms of accuracy and efficiency, across a diverse set of COPs.

RESULTS

Free-Energy Machine

Consider a COP characterized by a cost function, i.e. the energy function in physics, $E(\sigma)$, that we aim to minimize. Here, $\sigma = (\sigma_1, \sigma_2, \dots, \sigma_N)$ represents a candidate solution or a configuration comprising N discrete variables. The energy function encapsulates the interactions among variables, capturing the essence of various COPs. This is depicted in Fig. 1(a), where it is further delineated into four distinct models, each representing different physical scenarios.

1. One of the simplest cases is the QUBO problem (or the Ising problem) [20], with $E(\sigma) = -\sum_{i<j} W_{ij}\sigma_i\sigma_j$, where $\sigma_i \in \{-1, +1\}$. The existing Ising solvers are tailor-made

to address problems confined to the Ising model category.

2. The COPs permitting variables to take multi-valued states, specifically $\sigma_i \in \{1, 2, \dots, q\}$, yet maintaining the two-body interactions, are categorized under the Potts model [24]. This model is defined as $E(\sigma) = -\sum_{i<j} W_{ij}\delta(\sigma_i, \sigma_j)$, where $\delta(\sigma_i, \sigma_j)$ is the Kronecker function, yielding the value 1 if $\sigma_i = \sigma_j$ and 0 otherwise.
3. Another category of COPs includes those with higher-order interactions in the cost function, yet retain binary spin states. An example is the p -spin (commonly with $p > 2$) Ising glass model [22], characterized by the energy function $E(\sigma) = -\sum_{i_1 < i_2 < \dots < i_p} W_{i_1, i_2, \dots, i_p} \sigma_{i_1} \sigma_{i_2} \dots \sigma_{i_p}$, which integrates interactions among p distinct spins. This class of COPs is also known as the polynomial unconstrained binary optimization (PUBO) problem [30]. We note that considerable efforts have been undertaken to extend existing Ising solvers to high-order architectures [30–33].
4. In more general scenarios, COPs can encompass both multi-valued states and many-body interactions, featuring a simultaneous coexistence of interactions across various orders. We term this class of COPs the general model, it poses more challenges for the design of extended Ising machines.

Our proposed approach aims to address all kinds of problems discussed above (also shown in Fig. 1(a)) using the same variational framework. Within this framework, we focus on analyzing the Boltzmann distribution at a specified temperature

$$P_B(\sigma, \beta) = \frac{1}{Z} e^{-\beta E(\sigma)}, \quad (1)$$

where $\beta = 1/T$ is the inverse temperature and $Z = \sum_{\sigma} e^{-\beta E(\sigma)}$ is the partition function. It is important to emphasize that we do not impose any constraints on the specific form of $E(\sigma)$. Consequently, we extend the traditional Ising model formulations by permitting the spin variable to adopt q distinct states and use $P_i(\sigma_i)$ to represent the marginal probability of the i -th spin taking value $\sigma_i = 1, 2, \dots, q$, as illustrated in Fig. 1(b). The ground state configuration σ^{GS} that minimizes the energy can be achieved at the zero-temperature limit with

$$\sigma^{\text{GS}} = \arg \min_{\sigma} E(\sigma) = \arg \max_{\sigma} \lim_{\beta \rightarrow \infty} P_B(\sigma, \beta). \quad (2)$$

As illustrated in Fig. 1(b) and (c), accessing the Boltzmann distribution at zero temperature would allow us to calculate the marginal probabilities $P_i(\sigma_i)$ and determine the configuration based on the probabilities. However, there are two issues to accessing the zero-temperature Boltzmann distribution.

The first issue is that directly accessing the Boltzmann distribution at zero temperature poses significant challenges due to the rugged energy landscape, often described by the concept of replica symmetry breaking in statistical physics [34,

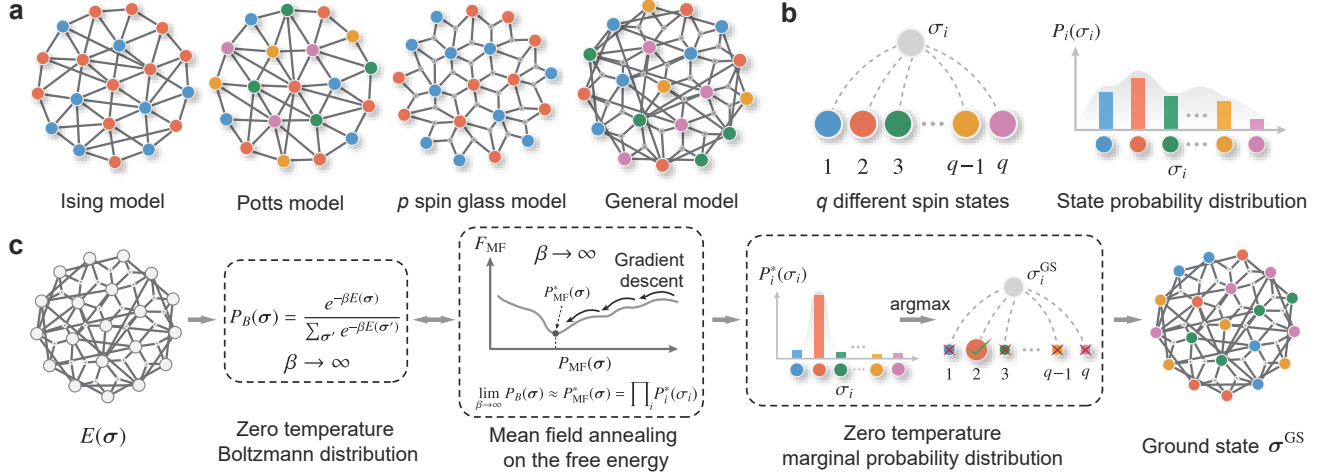


FIG. 1. **Illustration of the Free-Energy Machine in solving combinatorial optimization problems.** (a). Four distinct models offering representations of different types of combinatorial optimization problems. (b). In a general combinatorial optimization problem (COP), each spin variable is capable of adopting one of q distinct spin states, which are denoted as $1, 2, \dots, q$. The likelihood of assuming any given spin state is described by the marginal probability $P_i(\sigma_i)$. (c). To tackle a combinatorial optimization problem (COP) characterized by the cost function $E(\sigma)$, the primary computational challenge involves calculating the marginal probability from the zero-temperature Boltzmann distribution. This calculation can be effectively approximated through a gradient-based annealing approach applied to the variational mean-field free energy F_{MF} . By determining the optimal mean-field distribution $P_{\text{MF}}^*(\sigma)$ that minimizes F_{MF} , it becomes possible to ascertain the ground state of the COP. This is achieved by identifying the most probable spin state for each spin variable, utilizing the set of marginal probabilities $P_i^*(\sigma_i)$.

35]. To navigate this issue and facilitate a more manageable exploration of the landscape, we employ the strategy of annealing, which deals with the Boltzmann distribution at a finite temperature. This temperature is initially set high and is gradually reduced to zero.

The second issue is how to represent the Boltzmann distribution. Exactly computing the Boltzmann distribution belongs to the computational class of $\#P$, so we need to approximate it efficiently. Many approaches have been proposed, including Markov-Chain Monte-Carlo [3], mean-field and message-passing algorithms [28], and neural network methods [36, 37]. In this work, we use the variational mean-field distribution $P_{\text{MF}}(\sigma) = \prod_i P_i(\sigma_i)$ to approximate the Boltzmann distribution $P_B(\sigma, \beta)$. The parameters of P_{MF} can be determined by minimizing the Kullback-Leibler divergence $D_{\text{KL}}(P \| P_B) = \sum_{\sigma} P(\sigma) \ln(P(\sigma)/P_B(\sigma, \beta))$, and this is equivalent to minimizing the variational free energy

$$F_{\text{MF}} = \sum_{\sigma} P_{\text{MF}}(\sigma) E(\sigma) + \frac{1}{\beta} \sum_{\sigma} P_{\text{MF}}(\sigma) \ln P_{\text{MF}}(\sigma). \quad (3)$$

While the mean-field distribution may not boast the expressiveness of, for instance, neural network ansatzes, our findings indicate that it provides a precise representation of ground-state configurations at zero temperature via the annealing process. Furthermore, a significant advantage of the mean-field variational distribution is the capability for exact computation of the gradients of the mean-field free energy. This stands in stark contrast to variational distributions utilizing neural networks, where gradient computation necessitates stochastic sampling [36].

The pictorial illustration of implementing the FEM algorithm is depicted in Fig. 2. Given a COP defined on graph, we associate the N spin variables (each spin has q states) with the variational variables represented by the $N \times q$ marginal probabilities $\{P_i(\sigma_i)\}$ for the mean-field free energy. Then we parameterize the marginal probability $P_i(\sigma_i)$ using fields $\{h_i(\sigma_i)\}$ with a softmax function $P_i(\sigma_i) = \exp[h_i(\sigma_i)] / \sum_{\sigma'_i=1}^q \exp[h_i(\sigma'_i)]$ for a q -state variable, as illustrated in Fig. 2(a). This parameterization can release the normalization constraints on the variational variables $\{P_i(\sigma_i)\}$ (ensuring the probabilistic interpretation of $\sum_{\sigma_i} P_i(\sigma_i) = 1$ during the variational process). Moreover, our approach considers constraints on variables, such as the total number of spins with a particular value, or a global property that a configuration must satisfy.

At a high temperature, there could be just one mean-field distribution that minimizes the variational free energy. However, at a low temperature, there could be many mean-field distributions, each of which has a local free energy minimum, corresponding to a set of marginal probabilities. Inspired by the one-step replica symmetry breaking theory of spin glasses, we use a set of marginal distributions, which we term as m replicas of mean-field solutions with parameters $\{P_i^a(\sigma_i) | i = 1, 2, \dots, N; a = 1, 2, \dots, m\}$, each of which is updated to minimize the corresponding mean-field free energy and the minimization is reached through machine learning optimization techniques. This approach notably enhances both the number of parameters and the expressive capability of the mean-field ansatz.

The parameters of the replicas of the mean-field distribu-

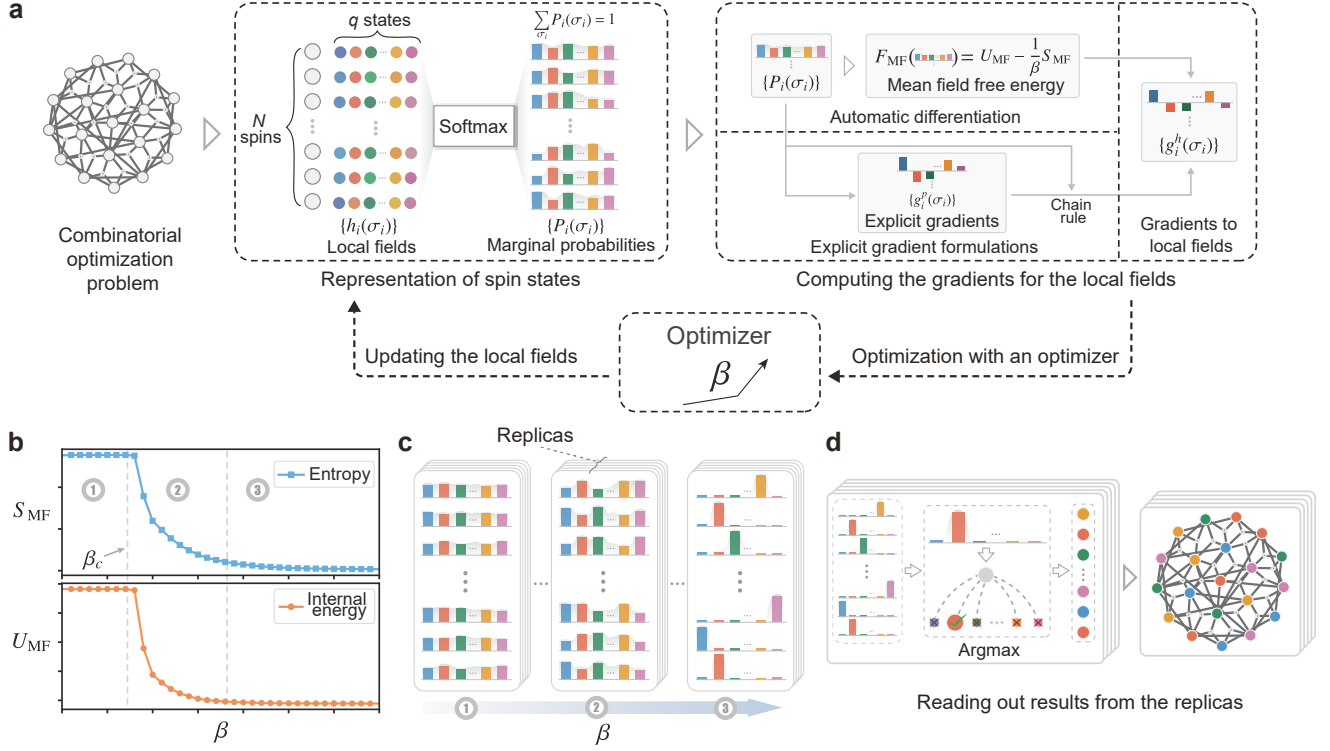


FIG. 2. **The framework of implementing Free-Energy Machine.** (a). Given a COP with N spin variables, the spin states are associated with $N \times q$ variables called the local fields $\{h_i(\sigma_i)\}$. The marginal probabilities for all spin states are calculated from the local fields through the softmax function, and the local fields serve as the genuine variational variables, consistently guaranteeing the probabilistic interpretation of $\{P_i(\sigma_i)\}$ ($\sum_{\sigma_i} P_i(\sigma_i) = 1$). The gradients of mean-field free energy F_{MF} with respect to the local fields, denoted as $\{g_i^h(\sigma_i)\}$, can be computed via the automatic differentiation or through the explicit gradient formulations. In the explicit gradient formulations, $\{g_i^h(\sigma_i)\}$ can be computed via the chain rule using the explicit gradients $\{g_i^p(\sigma_i)\}$ (please refer to Supplementary Materials for details). The well-developed continuous optimizers from the deep learning community can be employed for the optimization. With the annealing, the local fields are updated to optimize F_{MF} . (b). The schematic of U_{MF} and S_{MF} evolving with the inverse temperature β during the optimization process. β_c is the critical inverse temperature at which the distribution transition occurs. (c). The corresponding evolutions of the marginal probabilities with the annealing, depicting at the three stages marked in (b). Inspired by the one-step replica-symmetry breaking in statistical physics [28], we can update the marginal probabilities of many replicas parallelly. (d). Reading out the optimal solution from the replicas of marginal probabilities at the end of annealing.

tions are determined by minimizing the variational free energies, the gradients can be computed using automatic differentiation. This process is very similar to computing gradients of the loss function with respect to the parameters in deep neural networks. It amounts to expanding the computational process as a computational graph and applying the back-propagation algorithm. Thanks to the standard deep learning frameworks such as PyTorch [38] and TensorFlow [39], it can be implemented using just several lines of code as shown in the Methods section. Remarkably, for different combinatorial optimization problems, we only need to specify the form of the energy expectations $\sum_{\sigma} P_{MF}(\sigma) E(\sigma)$ as a function of marginal probabilities. Beyond leveraging automatic differentiation for gradient computation, we have the option to delineate the explicit gradient formulas for the problem. Utilizing explicit gradient formulations can halve the computational time. Another merit of adopting explicit gradients lies in the possibility of further enhancing our algorithm's stability through additional gradient manipulations, beyond

merely employing adaptive learning rates and momentum techniques, as seen in gradient-based optimization methods in machine learning [29].

With gradients computed, we adopt the advanced gradient-based optimization methods developed in the deep learning community for training neural networks, such as Adam [29], to update the parameters. They can efficiently maintain individual adaptive learning rates for each marginal probability from the first and second moments of the gradients, and require minimal memory overhead, so is well-suited for updating marginal probabilities in our algorithm. Fig. 2(b) shows a schematic of the typical evolutions of internal energy $U_{MF} = \sum_{\sigma} P_{MF}(\sigma) E(\sigma)$, and entropy $S_{MF} = -\sum_{\sigma} P_{MF}(\sigma) \ln P_{MF}(\sigma)$ as a function of β . The corresponding evolutions of $\{P_i(\sigma_i)\}$ of the replicas with the annealing are depicted in Fig. 2(c). All mean-field probabilities of the replicas are updated parallelly. Initially, the fields $\{h_i(\sigma_i)\}$ associated with spin i are randomly initialized around zero, making q -state marginal distributions $\{P_i(\sigma_i)\}$ around $1/q$.

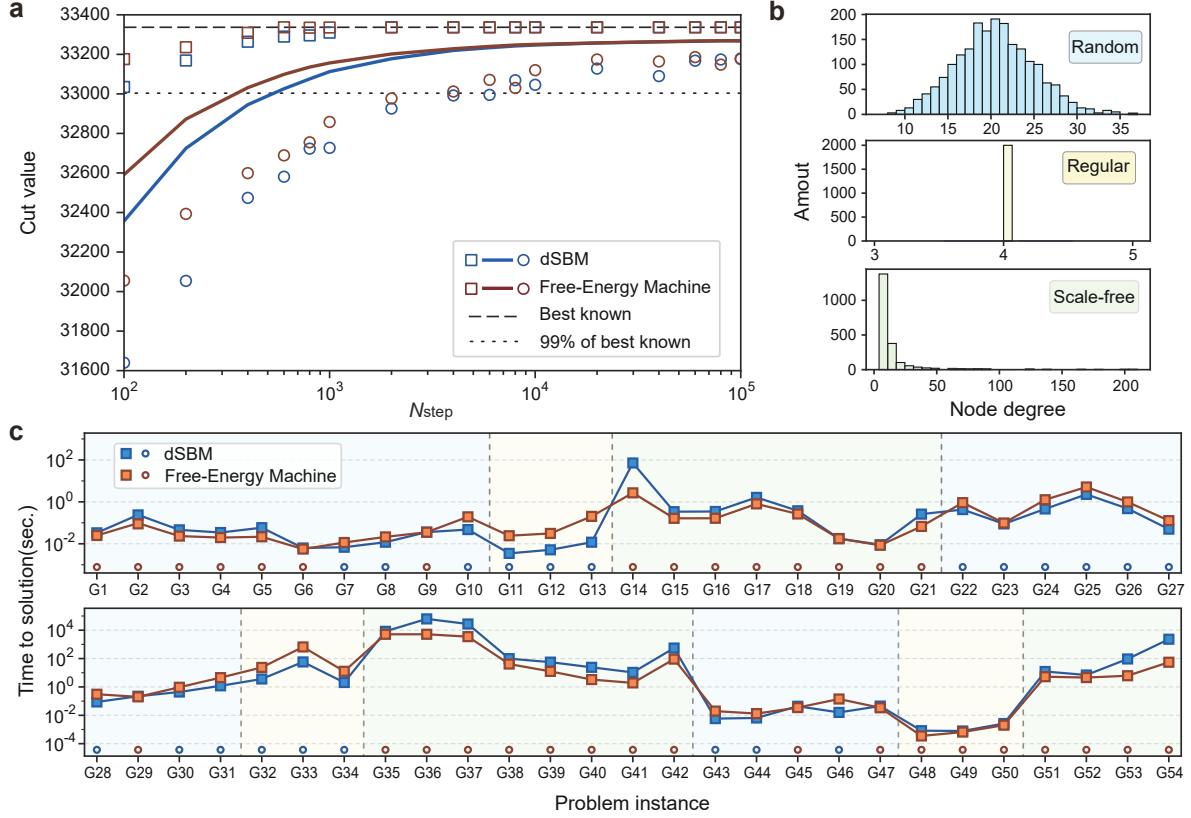


FIG. 3. **Benchmarking results for the maximum cut problem.** (a). The benchmarking results for solving the MaxCut problem on the complete graph K_{2000} with 2000 nodes. The graphical representations include squares, solid lines, and circles, which respectively illustrate the maximum, average, and minimum cut values as a function of the total number of annealing step N_{step} . For each N_{step} , we implement $R = 1000$ mean-field replicas using FEM, showcasing the distribution of cut values derived from these replicas. In parallel, the discrete Stochastic Bifurcation Machine (dSBM) was executed for 1000 trials with random initial conditions for each N_{step} to serve as a comparative benchmark. The best-known cut value for K_{2000} is 33337, with the 99% of the best-known cut values approximately reaching 33004. An inverse-proportional annealing scheduling with β ($T_{\text{max}} = 1.16$, $T_{\text{min}} = 6 \times 10^{-5}$) was applied. Please refer to main text and Supplementary Materials for more details. (b). The G-set instances commonly exhibit node degree distributions that categorize into three distinct types of graphs. (c). Time-to-Solution (TTS) benchmarking comparison of FEM and dSBM across G-set instances, which range from G1 to G54 and include graphs with 800 to 2000 nodes. The data for dSBM is referenced from the study in [11]. The G-set instances are visually categorized in the plot by regions marked with different colors, each color representing one of the three distinct graph types identified in these instances. Graph instances that exhibit shorter TTSs are notably highlighted with circles positioned at the lower section of the plot, indicating superior performance in those cases. For additional insights into the methodologies employed and the experimental setup, please refer to main text and Supplementary Materials.

In the first stage indicated in Fig. 2(b), since β is small (i.e. at a high temperature), the entropy term S_{MF} predominantly governs the energy landscape of F_{MF} . The uniform distributions of $\{P_i(\sigma_i)\}$ indicate that all possible spin configurations emerge with equal importance. Consequently, they maximize S_{MF} (i.e. minimize F_{MF} at a fixed β), and the value of S_{MF} remains around its maximal value in this stage. In the second stage, when β increases to some critical value β_c , U_{MF} becomes the predominant factor and the distribution transition occurs. As a consequence, internal energy plays a more important role in minimizing F_{MF} , leading $\{P_i(\sigma_i)\}$ to deviate from the uniform distributions and explore different mean-field solutions. In the third stage, when β is sufficiently large, U_{MF} gradually converges to a minimum value of F_{MF} , and $\{P_i(\sigma_i)\}$ gradually converges to approximate the zero-

temperature Boltzmann distribution, where the ground states are the most probable states to occur.

After the annealing process, as shown in Fig. 2(d), the temperature is decreased to a very low value, and we obtain a configuration for each replica according to the marginal probabilities, as

$$\tilde{\sigma}_i^a = \arg \max_{\sigma_i} P_i^a(\sigma_i). \quad (4)$$

Then we choose the configuration with the minimum energy from all replicas

$$\hat{\sigma}_i = \arg \min_a E(\tilde{\sigma}_i^a). \quad (5)$$

We emphasize that all the gradient computation and parameter updating on replicas can be processed parallelly

and is very similar to the computation in deep neural networks: overall computation only involves batched matrix multiplications and element-wise non-linear functions. Thus it fully utilizes massive parallel computational devices such as GPUs.

Applications to the Maximum-Cut problem

We begin by evaluating the performance of our algorithm on Quadratic Unconstrained Binary Optimization (QUBO) problems. To illustrate, we select the MaxCut problem as a representative example. This NP-complete problem is widely applied in various fields, including machine learning and data mining [40], the design of electronic circuits [41], and social network analysis [42]. Furthermore, it serves as a prevalent testbed for assessing the efficacy of new algorithms aimed at solving QUBO problems [10, 11, 43]. The optimization task in the MaxCut problem is to determine an optimal partition of nodes into $q = 2$ groups for an undirected weighted graph, in such a way that the sum of weights on the edges connecting two nodes belonging to different partitions (i.e. the cut size C) is maximized. Formally, we define the energy function as

$$E(\sigma) = - \sum_{(i,j) \in \mathcal{E}} W_{ij} [1 - \delta(\sigma_i, \sigma_j)], \quad (6)$$

where $\mathcal{E}$ is the edge set of the graph, W_{ij} is the weight of edge (i, j) , and $\delta(\sigma_i, \sigma_j)$ stands for the delta function, which takes value 1 if $\sigma_i = \sigma_j$ and 0 otherwise. Then the variational mean-field free energy for the MaxCut problem can be written out (see Methods section) and the gradients of F_{MF} to the variational parameters can be computed via automatic differentiation. In general, writing out the explicit formula for the gradients is not necessary, as the gradients computed using automatic differentiation are numerically equal to the explicit formulations. However, for the purpose of benchmarking, using the explicit formula will result in lower computation time. Moreover, in practice, we can further apply the normalization and clip of gradients to enhance the robustness of the optimization process, we refer to the Supplementary Materials for details.

We first benchmark FEM by solving a 2000-spin MaxCut problem named K_{2000} [44] with all-to-all connectivity, which has been intensively used in evaluating MaxCut solvers [10, 11, 45]. We compare the results obtained by FEM with discrete-SBM (dSBM) [11], which can be identified as the state-of-the-art solver for the MaxCut problem. Fig. 3(a) shows the cut values we obtained for the K_{2000} problem with different total annealing steps N_{step} used to increase β from $\beta_{\min}$ to $\beta_{\max}$. Similar to the role in FEM, the hyperparameter N_{step} introduced in dSBM controls the total number of annealing steps for the bifurcation parameter. To investigate the distribution of energy in all replicas, in the figure, we plot the minimum, average, and maximum cut value as a function of N_{step} , with $R = 1000$ mean-field replicas for

FEM. The results of FEM are compared with the dSBM algorithm, for which we also ran for 1000 trials from random initial conditions. From the results, we can see that the best value of FEM achieves the best-known results for this problem in less than 1000 annealing steps, and all the maximum, average, and minimum cut sizes are better than dSBM.

We also evaluate our algorithm using the standard benchmark for the MaxCut problem, the G-set [11, 45, 46]. The G-set benchmark contains various graphs including random, regular, and scale-free graphs based on the distribution of node degrees as shown in Fig. 3(b). Each problem contains a best-known solution which is regarded as the ground truth for evaluating algorithms. A commonly used statistic for quantitatively assessing both the accuracy and the computational speed in the MaxCut problem is the time-to-solution (TTS) [11, 43], which measures the average time to find a good solution by running the algorithm for many times (trials) from different initial states. TTS (or TTS_{99}) is formulated as $T_{\text{com}} \cdot \log(1 - 0.99) / \log(1 - P_S)$, where T_{com} represents the computation time per trial, and P_S denotes the success probability of finding the optimal cut value in all tested trials. When $P_S \geq 0.99$, the TTS is defined simply as T_{com} . The value of P_S is typically estimated from experimental results comprising numerous trials.

In Fig. 3(c) we present the TTS results obtained by FEM for 54 problem instances in G-set (G1 to G54, with the number of nodes ranging from 800 to 2000) and compare to the reported data for dSBM using GPUs. We can see that FEM surpasses dSBM in TTS for 33 out of the 54 instances, and notably for all G-set instances of scale-free graph, FEM achieves better performance than dSBM. The primary reason is that we employed the advanced normalization techniques for the optimization, please refer to the Supplementary Materials for more details. Furthermore, we notice that FEM outperforms the state-of-the-art neural network-based method in combinatorial optimization. For instance, the physics-inspired graph neural network (PI-GNN) approach [48] has been shown to outperform other neural-network-based methods on the G-set dataset. From the data in [48] we see that results of PI-GNN on the G-Set instances (with estimated runtimes in the order of tens of seconds) still give significant discrepancies to the best-known results of the G-set instances e.g. for G14, G15, G22 etc, while our method FEM achieves the best-known results for these instances only using several milliseconds.

Applications to the q -way balanced minimum cut problem

Next, we choose the q -way balanced MinCut (bMinCut) problem [49] as the second benchmarking type to evaluate the performance of FEM on directly addressing multi-valued problems featuring the Potts model. The q -way bMinCut problem asks to group nodes in a graph into q groups with a minimum cut size and balanced group sizes. The requirement to balance group sizes imposes a global constraint on the configurations, rendering the problem more complex

Instance	q	Best known	METIS	KaFFPaE	FEM	Instance	q	Best known	METIS	KaFFPaE	FEM
add20	2	596(0)	722(0)	597(0)	596(0)	data	2	189(0)	211(0)	189(0)	189(0)
	4	1151(0)	1257(0)	1158(0)	1152(0)		4	382(0)	429(0)	382(0)	382(0)
	8	1678(0)	1819(0.007)	1693(0)	1690(0)		8	668(0)	737(0)	668(0)	669(0)
	16	2040(0)	2442(0)	2054(0)	2057(0)		16	1127(0)	1237(0)	1138(0)	1129(0)
	32	2356(0)	2669(0.04)	2393(0)	2383(0)		32	1799(0)	2023(0)	1825(0)	1815(0)
3elt	2	90(0)	90(0)	90(0)	90(0)	bcsstk33	2	10171(0)	10205(0)	10171(0)	10171(0)
	4	201(0)	208(0)	201(0)	201(0)		4	21717(0)	22259(0)	21718(0)	21718(0)
	8	345(0)	380(0)	345(0)	345(0)		8	34437(0)	36732(0.001)	34437(0)	34440(0)
	16	573(0)	636(0.004)	573(0)	573(0)		16	54680(0)	58510(0)	54777(0)	54697(0)
	32	960(0)	1066(0)	966(0)	963(0)		32	77410(0)	83090(0.004)	77782(0)	77504(0)

TABLE I. **Benchmarking results for the q -way balanced minimum cut problem, using real-world graph instances from Chris Walshaw’s archive [47].** The graph partitioning was tested with the number of groups q set to 2, 4, 8, 16, and 32 across all solvers. The table showcases the minimum cut values obtained by three different solvers. The numbers in parentheses represent the imbalance ϵ (with $\epsilon = 0$ being ideal) satisfying the condition $|\Pi_n| \leq (1 + \epsilon)[N/q]$ for $n = 1, 2, \dots, q$, where N denotes the number of nodes and $|\Pi_n|$ indicates the size of each group. For each graph, the lowest cut values are emphasized in bold.

than unconstrained problems such as the MaxCut problem. Here we formulate the energy function with a soft constraint term

$$E(\sigma) = \sum_{(i,j) \in \mathcal{E}} W_{ij}[1 - \delta(\sigma_i, \sigma_j)] + \lambda \sum_i \sum_{j \neq i} \delta(\sigma_i, \sigma_j), \quad (7)$$

where λ is the parameter of soft constraints which controls the degree of imbalance. Based on the energy formulation, the expression of F_{MF} can be explicitly formulated (see Methods section), and its gradients can be calculated through automatic differentiation or derived analytically. In practice, we also used gradient normalization and gradient clip to enhance the robustness of the optimization, we refer to Supplementary Materials for detailed discussions. It’s worth noting that the bMinCut problem bears a significant resemblance to the community detection problem [26]. In the latter, the imbalance constraint is often substituted with the constraints derived from a random configuration model [26] or a generative model [50, 51], to avoid the trivial solution that puts all the nodes into a single group. Thus FEM can be easily adapted to the community detection problem.

To evaluate the performance of FEM in solving the q -way bMinCut problem, we conduct numerical experiments using four large real-world graphs from Chris Walshaw’s archive [47]. These include *add20* with 2395 nodes and 7462 edges, *data* with 2851 nodes and 15093 edges, *3elt* which comprises 4,720 nodes and 13,722 edges, and *bcsstk33* with 8738 nodes and 291583 edges. The graphs have been widely used in benchmarking q -way bMinCut solvers, e.g. used by the D-wave for benchmarking their quantum annealing hardware [49]. However, their work only presents the results of $q = 2$ partitioning, owing to the constraints of the quantum hardware. Here, we focus on the perfectly balanced problem which asks for group sizes the same. We evaluate the performance of FEM by partitioning the graphs into $q = 2, 4, 8, 16, 32$ groups. For comparison, we utilized two state-of-the-art solvers tailored to the q -way bMin-

Cut problem: METIS [52] and KAHIP (alongside its variant KaFFPaE, specifically engineered for balanced partitioning) [53], the latter being the winner of the 10th DIMACS challenge. The benchmarking results are shown in Tab. I, where we can observe that the results obtained by FEM consistently and considerably outperform METIS in all the problems and for all the number of groups q . Moreover, in some instances, METIS failed to find a perfectly balanced solution while the results found by FEM in all cases are perfectly balanced. We observe that FEM performs comparably to KaFFPaE for small group sizes q , and significantly outperforms KaFFPaE with a large q value. We have also evaluated the performance of FEM on extensive random graphs comprising up to one million nodes. The outcomes are depicted in Fig. 4. As observed from the figure, FEM achieves significantly lower cut values than METIS across the same collection of graphs when partitioned into $q = 4, 8$, and 16 groups. Notably, this performance disparity is maintained as the number of nodes increases to one million. The comparisons demonstrate FEM’s exceptional scalability in solving the large-scale q -way bMinCut problems.

Since the bMinCut modeling finds many real-world applications in parallel computing and distributed systems, data clustering, and bioinformatics [54, 55], we then apply FEM to address a challenging real-world problem of chip verification [56]. To identify and correct design defects in the chip before manufacturing, operators or computing units need to be deployed on a hardware platform consisting of several processors (e.g. FPGAs) for logic and function verification. Due to the limited capacity of a single FPGA and the restricted communication bandwidth among FPGAs, a large number of operators need to be uniformly distributed across the available FPGAs, while minimizing the communication volume among operators on different FPGAs. The schematic illustration is shown in Fig. 5(a). This scenario resembles load balancing in parallel computing and minimiz-

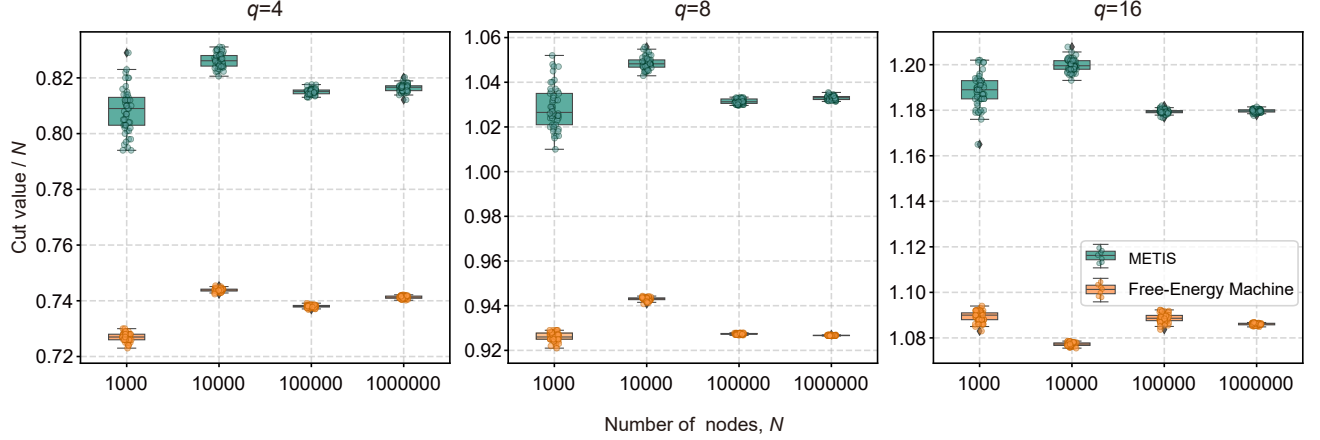


FIG. 4. **The scalability test of FEM in the q -way bMinCut problems on the Erdős-Rényi random graphs.** The generated random graphs contain a number of nodes ranging from 1,000 to 1,000,000, each with an average degree of 5. We benchmark the scalability of FEM with METIS over these generated random graphs. The number of partitions we evaluate are $q = 2, 4, 8$. Each box plot contains 50 points, representing the cut value per node, obtained from 50 runs of the algorithms. The number of replicas of FEM is set to $R = 50$ (in each run) throughout the experiments.

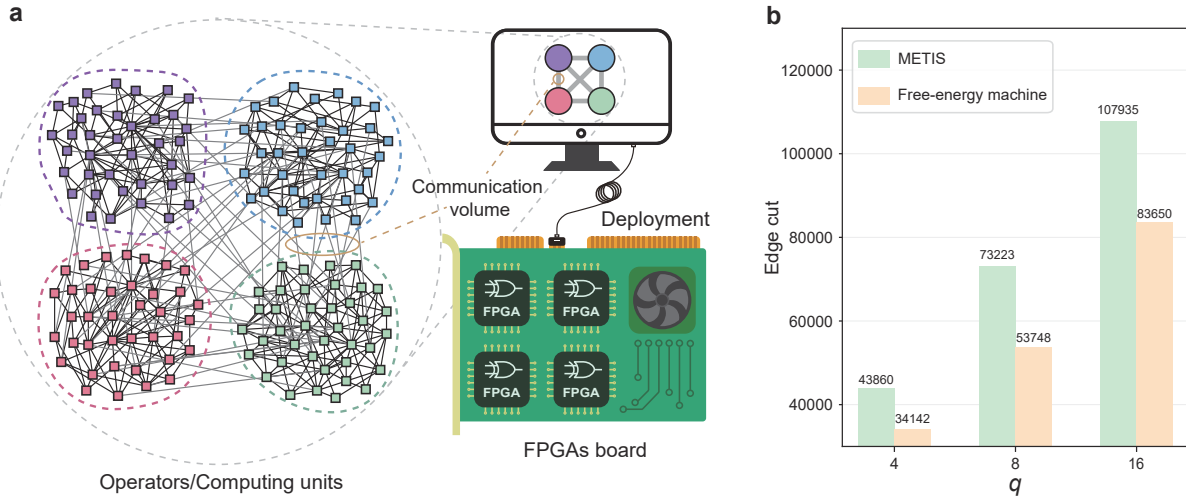


FIG. 5. **The application of FEM in large-scale FPGA-chip verification tasks.** (a). In the chip verification task, a vast number of operators or computing units with logical interconnections need to be uniformly deployed onto a hardware platform consisting of several FPGAs. The operators are partitioned into several groups corresponding to different FPGAs, and the communication volume between these groups of operators should be minimized. This can be modeled as a balanced minimum cut problem. (b). The results of FEM, along with a comparison to METIS, are presented for a large-scale real-world dataset consisting of 1,495,802 operators and 3,380,910 logical operator connections deployed onto ($q = 4, 8, 16$) FPGAs (see Supplementary Materials for details).

ing the edge cut while maintaining balanced partitions and can be modeled as a balanced minimum cut problem [57]. In this work, we address a large-scale real-world chip verification task consisting of 1495802 operators (viewed as nodes) and 3380910 logical operator connections (viewed as edges) onto $q = 4, 8, 16$ FPGAs. We apply FEM to solve this task. Since the dataset contains many locally connected structures among operators, we first conduct coarsening the entire graph before partitioning on it. Unlike the matching method used in the coarsening phase in METIS [52], we ap-

ply the Louvain algorithm [58] to identify community structures. Nodes within the same community are coarsened together. The results are shown in Fig. 5(b), along with comparative results provided by METIS (see Supplementary Materials for more details. We did not include the results of KaFFPaE, as its open-source implementation [59] runs very slowly and exceeds the acceptable time limits on large-scale graphs). From the figure, we can observe that the size of the edge cut given by FEM is 22.2%, 26.6%, and 22.5% smaller than METIS, for 4, 8, and 16 FPGAs, respectively, signifi-

cantly reduces the amount of communication among FPGAs and shortening the chip verification time.

Application to the Max k -SAT problem

Lastly, we evaluate FEM for addressing COPs with the higher-order spin interactions on the constraint Boolean satisfiability (SAT) problem. In this problem, M logical clauses, denoted as $C_1, C_2, \dots, C_M$, are applied to N boolean variables. Each clause is a disjunction of literals, namely $C_m = l_1 \vee l_2 \vee \dots \vee l_{k_m}$ (k_m is the number of literals in the clause C_m). A literal can be a Boolean variable σ_i or its negation $\neg\sigma_i$. The clauses are collectively expressed in the conjunctive normal form (CNF). For example, the CNF formula $C_1 \wedge C_2 \wedge C_3 = (\sigma_1 \vee \neg\sigma_2 \vee \neg\sigma_3) \wedge (\neg\sigma_1 \vee \sigma_4) \wedge (\sigma_2 \vee \neg\sigma_3 \vee \neg\sigma_4)$ is composed of 4 Boolean variables and 3 clauses. Note that, a clause is satisfied if at least one of its literals is true, and is unsatisfied if no literal is true. We can see that the SAT problem is a typical many-body interaction problem with higher-order spin interactions. The decision version of the SAT problem asks to determine whether there exists an assignment of Boolean variables to satisfy all clauses simultaneously (i.e. the CNF formula is true). The optimization version of the SAT problem is the maximum SAT (Max-SAT) problem, which asks to find an assignment of variables that maximizes the number of clauses that are satisfied. When each clause comprises exactly k literals (i.e. $k_m = k$), the problem is identified as the k -SAT problem, one of the earliest recognized NP-complete problems (when $k \geq 3$) [62, 63]. These problems are pivotal in the field of computational complexity theory. We benchmark FEM on the Max k -SAT problem, which is NP-hard for any $k \geq 2$. In our framework, the energy function for the Max k -SAT problem is formulated as the number of unsatisfied clauses, as

$$E(\sigma) = \sum_{m=1}^M \prod_{i \in \partial m} [1 - \delta(W_{mi}, \sigma_i)], \quad (8)$$

where $\sigma_i = \{0, 1\}$ is the Boolean variable, ∂m denotes the set of Boolean variables that appears in clause C_m , $W_{mi} = 0$ if the literal regarding variable i is negated in clause C_m and $W_{mi} = 1$ when not negated. Note that, in the case of Boolean SAT, $W_{mi} \in \{0, 1\}$ corresponds to the two states of spin variables. The energy function can be generalized to any Max-SAT problem, where clauses may vary in the number of literals, and to cases of non-Boolean SAT problem where W_{mi} has $q > 2$ states. The expression of F_{MF} can be also found in Methods section, and the form of its explicit gradients please refer to Supplementary Materials.

We access the performance of FEM using the dataset in MaxSAT 2016 competition [61]. The competition problems encompass four distinct categories: “s2”, “s3” by Abramson-Habet, and “HG3”, “HG4” with high-girth sets. The “s2” category consists of Max 2-SAT problems with $N \in [120, 200]$ and $M \in [1200, 2600]$. The “s3” category includes Max 3-SAT problems with $N \in [70, 110]$ and

$M \in [700, 1500]$. For the “HG3” category, the Max 3-SAT problems feature $N \in [250, 300]$ and $M \in [1000, 1200]$. Lastly, the “HG4” category contains Max 4-SAT problems with $N \in [100, 150]$ and $M \in [900, 1350]$. Fig. 6 shows the benchmarking results for all 454 competition instances in the four categories (see Supplementary Materials for the experimental details).

In Fig. 6(a), we illustrate the quality of solutions found by FEM, evaluated using the energy difference ΔE , between FEM the best-known results for all problem instances. To provide a comprehensive comparison, we also present the documented results for the competition problems achieved by a state-of-the-art solver, continuous-time dynamical heuristic (Max-CTDS), as reported in Ref. [60]. The results in Fig. 6(a) show that FEM found the optimal solution in 448 out of 454 problem instances. For the 6 instances that FEM did not achieve the optimal solution, it found a solution with an energy gap 1 to the best-known solution. Also, we can see that FEM outperforms Max-CTDS in all instances.

In Fig. 6(b), we list the computational time of FEM using GPU in solving each instance of SAT competition 2016 problems and compare them with the computation time of the specific-purpose incomplete MaxSAT solvers (using CPU) in the 2016 competition [61]. For each instance, we only chart the minimum computational time of the incomplete solver needed to reach the best-known results, as documented by all incomplete MaxSAT solvers [61]. Note that the fastest incomplete solver can differ across various instances. The data presented in the figure clearly demonstrates that FEM outperforms the quickest incomplete MaxSAT solvers from the competition, both significantly and consistently. On average, FEM achieves a computational time of 0.074 seconds across all instances, with a variation (standard deviation) of 0.077 seconds. The computational time ranges from as short as 0.018 seconds for the “s2v200c1400-2.cnf” to as long as 1.17 seconds for the “HG-3SAT-V250-C1000-14.cnf” instance. A key factor contributing to the rapid computation time of FEM is its ability to leverage the extensive parallel processing capabilities of GPU, which can accelerate computations by approximately tenfold compared to CPU processing. Nevertheless, it’s crucial to highlight that even when performing on a CPU, FEM significantly outpaces most of the competitors in the SAT competition. For example, on average, Max-CTDS demands an average time of 4.35 hours to approximate an optimal assignment across all instances, as reported in [60]. In stark contrast, FEM, when running on CPU, completes the same task in just a few seconds on average. Our benchmarking results demonstrate that FEM surpasses contemporary leading solvers in terms of accuracy and computational speed when addressing the problems presented in the MaxSAT 2016 competition.

DISCUSSION

We have presented a general and high-performance approach for solving COPs using a unified framework in-

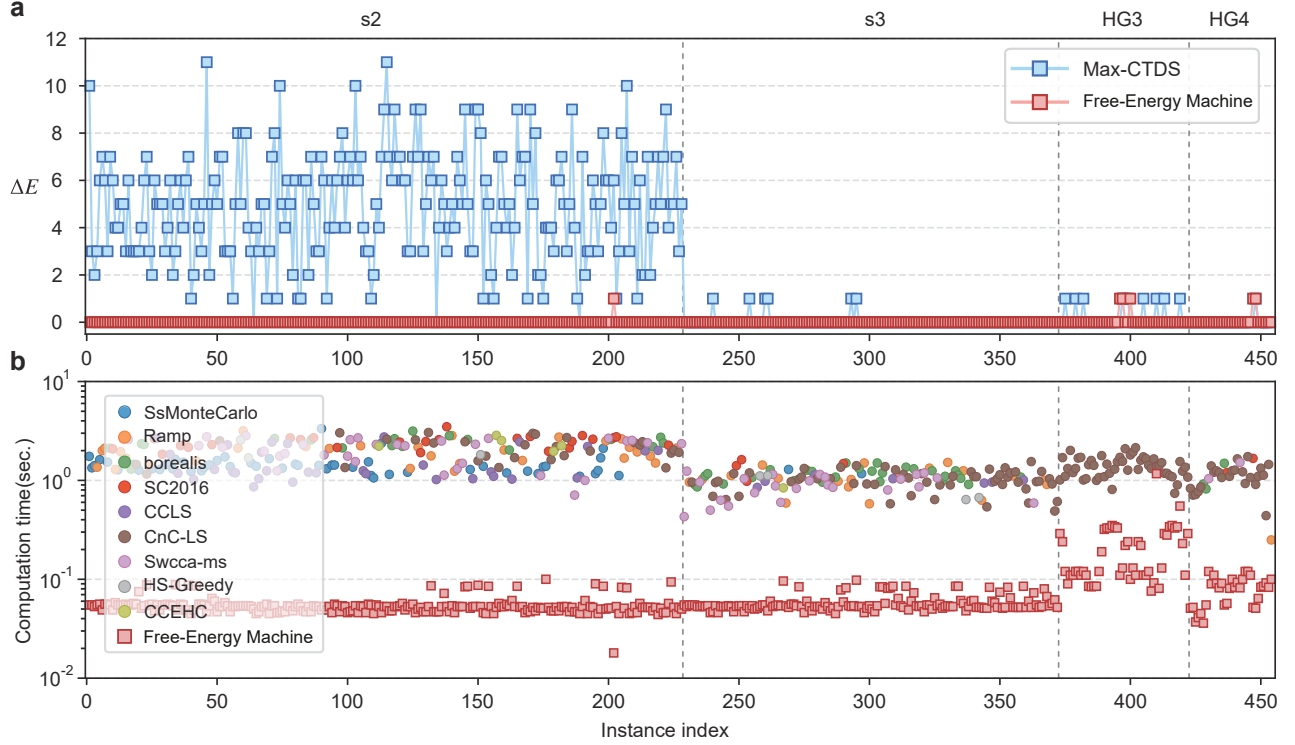


FIG. 6. **Benchmarking results on the MaxSAT 2016 competition problems.** (a). The results for the energy differences, $\Delta E = E_{\min} - E_{\text{bkr}}$, which represent the gap between the minimal energy values found by the solvers ($E_{\min}$) and the best-known results documented in the literature (E_{bkr}) with E indicating the number of unsatisfied clauses. These findings cover all 454 problem instances across four competition categories: “s2”, “s3”, “HG3”, and “HG4”. For further discussion on these results, please see the main text. The Max-CTDS algorithm data were obtained from Ref. [60]. (b). The computation time (measured by the running time of all replicas) of the Free-Energy Machine (FEM) to achieve these outcomes against the leading incomplete solvers from the 2016 competition across different instances. The performance data for these incomplete solvers were sourced from the 2016 MaxSAT competition documentation [61]. For detailed information on the experimental setup for FEM, refer to the main text and Supplementary Materials.

spired by statistical physics and machine learning. The proposed method, FEM, integrates three critical components to achieve its success. First, FEM employs the variational mean-field free energy framework from statistical physics. The framework facilitates the natural encoding of diverse COPs, including those with multi-valued states and higher-order interactions. This attribute renders FEM an exceptionally versatile approach. Second, inspired by replica symmetry breaking theory, FEM maintains a large number of replicas of mean-field free energies, exploring the mean-field spaces to efficiently find an optimal solution. Third, the mean-field free energies are computed and minimized using machine-learning techniques including automatic differentiation, gradient normalization, and optimization. This offers a general framework for different kinds of COPs, enables massive parallelization using modern GPUs, and fast computations.

We have executed comprehensive benchmark tests on a variety of optimization challenges, each exhibiting unique features. These include the MaxCut problem, characterized by a two-state and two-body interaction without constraints; the bMinCut, defined by a q -state and two-body interaction with global constraints; and the Max k -SAT problem,

which involves a two-state many-body interactions. The outcomes of our benchmarks clearly show that FEM markedly surpasses contemporary algorithms tailored specifically for each problem, demonstrating its superior performance across these diverse optimization scenarios. Beyond the benchmarking problems showcased in this study, we also extend our modelings to encompass a broader spectrum of combinatorial optimization issues, to which FEM can be directly applied. For further details, please consult the Supplementary Materials.

In this study, our exploration was confined to the most fundamental mean-field theory within statistical physics. However, more sophisticated mean-field theories exist, such as the Thouless-Anderson-Palmer (TAP) equations associated with TAP free energy, and belief propagation, which connects to the Bethe free energy. These advanced theories have the potential to be integrated into the FEM framework, offering capabilities that could surpass those of the basic mean-field approaches. We put this into future work.

METHODS

The variational mean-field free energy formulations

As outlined in the opening of the Results section, FEM addresses COPs by minimizing the variational mean-field free energy through a process of annealing from high to low temperatures. To tackle a specific COP, we commence by constructing the variational mean-field free energy formulation for the problem at hand. Here, we establish the variational mean-field free energy formulations for the MaxCut, the bMinCut, and the Max k -SAT problems that are benchmarked in this study. The derivation details can be found in Supplementary Materials.

Starting with the MaxCut problem, the variational free energy reads

$$F_{\text{MF}}^{\text{MaxCut}}(\{P_i(\sigma_i)\}, \beta) = - \sum_{(i,j) \in \mathcal{E}} \sum_{\sigma_i} W_{ij} P_i(\sigma_i) [1 - P_j(\sigma_j)] + \frac{1}{\beta} \sum_i \sum_{\sigma_i} P_i(\sigma_i) \ln P_i(\sigma_i). \quad (9)$$

For the bMinCut problem, the variational free energy is written as

$$F_{\text{MF}}^{\text{bMinCut}}(\{P_i(\sigma_i)\}, \beta) = \sum_{(i,j) \in \mathcal{E}} \sum_{\sigma_i} W_{ij} P_i(\sigma_i) [1 - P_j(\sigma_j)] + \lambda \sum_{i,j} \sum_{\sigma_i} [P_i(\sigma_i) P_j(\sigma_j) - P_i^2(\sigma_i)] + \frac{1}{\beta} \sum_i \sum_{\sigma_i} P_i(\sigma_i) \ln P_i(\sigma_i), \quad (10)$$

and for the Max k -SAT problem, as

$$F_{\text{MF}}^{\text{MaxSAT}}(\{P_i(\sigma_i)\}, \beta) = \sum_{m=1}^M \prod_{i \in \partial m} [1 - P_i(W_{mi})] + \frac{1}{\beta} \sum_i \sum_{\sigma_i} P_i(\sigma_i) \ln P_i(\sigma_i). \quad (11)$$

Implementation of FEM with the automatic differentiation by Pytorch

The gradients to the marginal probabilities can be computed via automatic differentiation method, or the explicit formulations (see Supplementary Materials). Then we employ the gradient-based optimization methods, such as stochastic gradient descent (SGD), RMSprop [64], and Adam [29] to update the marginal distributions and the fields, then decrease the temperature. The Python code below demonstrates a clear and direct approach to solving both the MaxCut problem and the bMinCut problem, which are as the benchmarking problems in the work. Interestingly, despite the significant differences between these two problems in terms of the number of variable states, the presence of

constraints, and the nature of the objective function, the implementations for each problem differ by only a single line of code. This highlights the adaptability and efficiency of the approach in handling distinct optimization challenges. Please refer to Supplementary Materials for the codes with detailed explanations.

```

1  import torch
2
3  def cut(W,p):
4      return ((W @ p) * (1-p)).sum((1, 2))
5
6  def S(p):
7      return -(p*p.log()).sum(2).sum(1)
8
9  def argmax_cut(W,p):
10     s = torch.nn.functional.one_hot(
11         p.argmax(dim=2), num_classes=p.shape
12         [2])
13     return config, cut(W, s) / 2
14
15 def balance(p):
16     return (p.sum(1)**2).sum(1) - (p**2).
17         sum(2).sum(1)
18
19 def solve(problem,W,batch,q,paneltty,
20     beta_range):
21     n = W.shape[0]
22     h = torch.rand(batch, n, q)
23     optimizer = torch.optim.Adam([h], lr
24         =0.01)
25     for beta in beta_range:
26         p = torch.softmax(h, dim=2)
27         if problem == 'maxcut':
28             F = -cut(W,p) - S(p)/beta
29         if problem == 'bmincut':
30             F = cut(W,p)
31                 +paneltty*balance(p)-S(p)
32                 )/beta
33         optimizer.zero_grad()
34         F.backward(gradient=torch.
35             ones_like(F))
36         optimizer.step()
37     return argmax_cut(W, p)

```

As highlighted in the Results section, besides using automatic differentiation for gradient computation, we can specify explicit gradient formulas for each problem. Our numerical experiments show that employing explicit gradients can reduce computational time by half and it offers the potential to improve our algorithm's stability. For instance, in the maximum cut problem, the variance in node degrees within the graph can lead to significant fluctuations in the gradients for each marginal, potentially destabilizing the optimization process. To counteract this, we substitute the marginal distributions with one-hot vectors and normalize the gradient magnitude for each spin, ensuring the gradients remain stable and neither explode nor vanish. For technical specifics, we direct readers to the Supplementary Materials.

Different annealing schedules for the inverse temperature

Regarding the annealing process, this study employs two monotonic functions to structure the annealing schedule of β . The first function is named as the exponential scheduling, which is utilized for the exponential decrease of temperature T . This is defined as follows:

$$\beta(t_s) = \exp\left(\frac{\ln\beta_{\max} - \ln\beta_{\min}}{N_{\text{step}} - 1}t_s + \ln\beta_{\min}\right),$$

where $t_s \in \{0, 1, 2, \dots, N_{\text{step}} - 1\}$ represents the annealing step within a total of N_{step} steps, ensuring that $\beta(0) = \beta_{\min}$ and $\beta(N_{\text{step}} - 1) = \beta_{\max}$. The second function is named as the inverse-proportional scheduling, as

$$\beta(t_s) = \left(\frac{\beta_{\max}^{-1} - \beta_{\min}^{-1}}{N_{\text{step}} - 1}t_s + \beta_{\min}^{-1}\right)^{-1},$$

which is equivalent to linear cooling in terms of temperature, with

$$T(t_s) = 1/\beta(t_s) = \frac{T_{\min} - T_{\max}}{N_{\text{step}} - 1}t_s + T_{\max},$$

where $T_{\min} = 1/\beta_{\max}$ and $T_{\max} = 1/\beta_{\min}$.

Hyperparameter tuning of different optimizers used for the optimization

In this study, we employ SGD, RMSprop, and Adam as the principal optimization algorithms for our tasks. We utilize the Python implementations of these optimizers as provided by PyTorch [65]. Within PyTorch, the adjustable hyperparameters for these optimizers vary to some extent. We have carefully tuned these hyperparameters to enhance performance while maintaining the default settings for other parameters provided by PyTorch. Specifically, for SGD, we have optimized the learning rate, weight decay, momentum, and dampening. For RMSprop, we have adjusted the learning rate, weight decay, momentum, and the smoothing constant alpha. Lastly, for Adam, the learning rate, weight decay, and the exponential decay rates for the first and second moment estimates, β_1 and β_2 , have been the primary focus of our tuning efforts.

In our numerical experiments, we observed that the mean value of the energy function for COP largely depends on the hyperparameters of the optimizer used, showing a great insensitivity to the number of replicas. Consequently, hyperparameter tuning can initially be conducted with a small number of replicas, followed by incrementally increasing the number of replicas to identify the better energy values. For details on how increasing the number of replicas impacts the energy function values, please refer to the Supplementary Materials.

Relationship to the existing mean-field annealing approaches

It's noteworthy that the exploration of mean-field theory coupled with an annealing scheme for COPs began in the late 20th century, as indicated in [66]. This approach has also been instrumental in deciphering the efficacy of recently introduced algorithms inspired by quantum dynamics, as discussed in [9]. Traditional mean-field annealing algorithms, those addressing the Ising problem, revolve around the iterative application of mean-field equations (for reproducing the mean-field equations for the Ising problem from the FEM formalism, please refer to Supplementary Materials):

$$m_i = \tanh(\beta \sum_j W_{ij}m_j),$$

which determines the state of a spin based on the average value of neighboring spins. These algorithms incorporate a damping technique to enhance the convergence of the iterative equations, represented as:

$$m_i^{t+1} = \alpha m_i^t + (1 - \alpha) \tanh(\beta \sum_j W_{ij}m_j^t),$$

where $0 < \alpha < 1$. In contrast, our algorithm adopts a more statistical-physics-grounded approach, focusing on the direct minimization of replicas of mean-field free energies through contemporary machine-learning methodologies. Furthermore, whereas existing iterative mean-field annealing algorithms are tailored specifically to two-state Ising problems, our algorithm boasts broader applicability, seamlessly addressing a wide array of COPs characterized by multiple states and multi-body interactions.

DATA AVAILABILITY

The datasets utilized in this study for benchmarking the MaxCut problem, namely the K_{2000} and G-set, were obtained from Refs. [44] and [46], respectively. For the bMin-Cut problem benchmarks, graph instances were sourced from Ref. [47]. Additionally, the dataset employed for the MaxSAT benchmarks was acquired from the MaxSAT 2016 competition, as documented in Ref. [61].

CODE AVAILABILITY

The source code for this paper is publicly available at <https://github.com/Fanerst/FEM>.

ACKNOWLEDGEMENTS

This work is supported by Project 12047503, 12325501, and 12247104 of the National Natural Science Foundation of China and project ZDRW-XX-2022-3-02 of the Chinese

Academy of Sciences. P. Z. is partially supported by the Innovation Program for Quantum Science and Technology project 2021ZD0301900.

* These three authors contributed equally

† panzhang@itp.ac.cn

- [1] D. Du and P. M. Pardalos, *Handbook of combinatorial optimization*, Vol. 4 (Springer Science & Business Media, 1998).
- [2] S. Arora and B. Barak, *Computational complexity: a modern approach* (Cambridge University Press, 2009).
- [3] S. Kirkpatrick, C. D. Gelatt Jr, and M. P. Vecchi, Optimization by simulated annealing, *Science* **220**, 671 (1983).
- [4] B. Selman, H. A. Kautz, B. Cohen, *et al.*, Noise strategies for improving local search, *AAAI* **94**, 337 (1994).
- [5] F. Glover and M. Laguna, *Tabu search* (Springer, 1998).
- [6] S. Boettcher and A. G. Percus, Optimization with extremal dynamics, *Phys. Rev. Lett.* **86**, 5211 (2001).
- [7] F. Barahona, On the computational complexity of ising spin glass models, *Journal of Physics A: Mathematical and General* **15**, 3241 (1982).
- [8] E. S. Tiunov, A. E. Ulanov, and A. Lvovsky, Annealing by simulating the coherent Ising machine, *Optics Express* **27**, 10288 (2019).
- [9] A. D. King, W. Bernoudy, J. King, A. J. Berkley, and T. Lanting, Emulating the coherent Ising machine with a mean-field algorithm, *arXiv preprint arXiv:1806.08422* (2018).
- [10] H. Goto, K. Tatsumura, and A. R. Dixon, Combinatorial optimization by simulating adiabatic bifurcations in nonlinear Hamiltonian systems, *Science Advances* **5**, eaav2372 (2019).
- [11] H. Goto, K. Endo, M. Suzuki, Y. Sakai, T. Kanao, Y. Hamakawa, R. Hidaka, M. Yamasaki, and K. Tatsumura, High-performance combinatorial optimization based on classical mechanics, *Science Advances* **7**, eabe7953 (2021).
- [12] M. W. Johnson, M. H. Amin, S. Gildert, T. Lanting, F. Hamze, N. Dickson, R. Harris, A. J. Berkley, J. Johansson, P. Bunyk, *et al.*, Quantum annealing with manufactured spins, *Nature* **473**, 194 (2011).
- [13] T. Inagaki, K. Inaba, R. Hamerly, K. Inoue, Y. Yamamoto, and H. Takesue, Large-scale Ising spin network based on degenerate optical parametric oscillators, *Nature Photonics* **10**, 415 (2016).
- [14] T. Honjo, T. Sonobe, K. Inaba, T. Inagaki, T. Ikuta, Y. Yamada, T. Kazama, K. Enbutsu, T. Umeki, R. Kasahara, *et al.*, 100,000-spin coherent Ising machine, *Science Advances* **7**, eabh0952 (2021).
- [15] D. Pierangeli, G. Marcucci, and C. Conti, Large-scale photonic Ising machine by spatial light modulation, *Phys. Rev. Lett.* **122**, 213902 (2019).
- [16] A. Mallick, M. K. Bashar, D. S. Truesdell, B. H. Calhoun, S. Joshi, and N. Shukla, Using synchronized oscillators to compute the maximum independent set, *Nature Communications* **11**, 4689 (2020).
- [17] F. Cai, S. Kumar, T. Van Vaerenbergh, X. Sheng, R. Liu, C. Li, Z. Liu, M. Foltin, S. Yu, Q. Xia, *et al.*, Power-efficient combinatorial optimization using intrinsic noise in memristor Hopfield neural networks, *Nature Electronics* **3**, 409 (2020).
- [18] N. A. Aadit, A. Grimaldi, M. Carpentieri, L. Theogarajan, J. M. Martinis, G. Finocchio, and K. Y. Camsari, Massively parallel probabilistic computing with sparse Ising machines, *Nature Electronics* **5**, 460 (2022).
- [19] N. Mohseni, P. L. McMahon, and T. Byrnes, Ising machines as hardware solvers of combinatorial optimization problems, *Nature Reviews Physics* **4**, 363 (2022).
- [20] G. Kochenberger, J.-K. Hao, F. Glover, M. Lewis, Z. Lü, H. Wang, and Y. Wang, The unconstrained binary quadratic programming problem: a survey, *Journal of Combinatorial Optimization* **28**, 58 (2014).
- [21] A. Lucas, Ising formulations of many NP problems, *Frontiers in Physics* **2**, 5 (2014).
- [22] E. Gardner, Spin glasses with p -spin interactions, *Nuclear Physics B* **257**, 747 (1985).
- [23] R. M. Karp, *Reducibility among combinatorial problems* (Springer, 2010).
- [24] F.-Y. Wu, The Potts model, *Rev. Mod. Phys.* **54**, 235 (1982).
- [25] T. R. Jensen and B. Toft, *Graph coloring problems* (John Wiley & Sons, 2011).
- [26] M. E. Newman, Modularity and community structure in networks, *Proceedings of the National Academy of Sciences* **103**, 8577 (2006).
- [27] C. Papadimitriou and K. Steiglitz, *Combinatorial Optimization: Algorithms and Complexity*, Dover Books on Computer Science (Dover Publications, 1998).
- [28] M. Mézard, G. Parisi, and R. Zecchina, Analytic and algorithmic solution of random satisfiability problems, *Science* **297**, 812 (2002).
- [29] D. P. Kingma and J. Ba, Adam: A method for stochastic optimization, *arXiv preprint arXiv:1412.6980* (2014).
- [30] D. A. Chermoshentsev, A. O. Malyshev, M. Esencan, E. S. Tiunov, D. Mendoza, A. Aspuru-Guzik, A. K. Fedorov, and A. I. Lvovsky, Polynomial unconstrained binary optimisation inspired by optical simulation, *arXiv preprint arXiv:2106.13167* (2021).
- [31] C. Bybee, D. Kleyko, D. E. Nikonov, A. Khosrowshahi, B. A. Olshausen, and F. T. Sommer, Efficient optimization with higher-order Ising machines, *Nature Communications* **14**, 6033 (2023).
- [32] T. Kanao and H. Goto, Simulated bifurcation for higher-order cost functions, *Applied Physics Express* **16**, 014501 (2022).
- [33] S. Reifenstein, T. Leleu, T. McKenna, M. Jankowski, M.-G. Suh, E. Ng, F. Khoyratee, Z. Toroczka, and Y. Yamamoto, Coherent SAT solvers: a tutorial, *Advances in Optics and Photonics* **15**, 385 (2023).
- [34] M. Mézard, G. Parisi, N. Sourlas, G. Toulouse, and M. Virasoro, Replica symmetry breaking and the nature of the spin glass phase, *Journal de Physique* **45**, 843 (1984).
- [35] M. Mézard, G. Parisi, and M. A. Virasoro, *Spin glass theory and beyond: An Introduction to the Replica Method and Its Applications*, Vol. 9 (World Scientific Publishing Company, 1987).
- [36] D. Wu, L. Wang, and P. Zhang, Solving statistical mechanics using variational autoregressive networks, *Phys. Rev. Lett.* **122**, 080602 (2019).
- [37] M. Hibat-Allah, E. M. Inack, R. Wiersema, R. G. Melko, and J. Carrasquilla, Variational neural annealing, *Nature Machine Intelligence* **3**, 952 (2021).
- [38] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, *et al.*, Pytorch: An imperative style, high-performance deep learning library, *Advances in Neural Information Processing Systems* **32** (2019).
- [39] M. Abadi, A. Agarwal, P. Barham, E. Brevdo, Z. Chen, C. Citro, G. S. Corrado, A. Davis, J. Dean, M. Devin, *et al.*, Tensorflow: Large-scale machine learning on heterogeneous distributed systems, *arXiv preprint arXiv:1603.04467* (2016).

- [40] Y. Y. Boykov and M.-P. Jolly, Interactive graph cuts for optimal boundary & region segmentation of objects in N-D images, in *Proceedings eighth IEEE international conference on computer vision. ICCV 2001*, Vol. 1 (IEEE, 2001) pp. 105–112.
- [41] F. Barahona, M. Grötschel, M. Jünger, and G. Reinelt, An application of combinatorial optimization to statistical physics and circuit layout design, *Operations Research* **36**, 493 (1988).
- [42] G. Facchetti, G. Iacono, and C. Altafini, Computing global structural balance in large-scale signed social networks, *Proceedings of the National Academy of Sciences* **108**, 20953 (2011).
- [43] F. Böhm, T. V. Vaerenbergh, G. Verschaffelt, and G. Van der Sande, Order-of-magnitude differences in computational performance of analog Ising machines induced by the choice of nonlinearity, *Communications Physics* **4**, 149 (2021).
- [44] G. Rinaldy, rudy graph generator, <http://www-user.tu-chemnitz.de/~helmberg/rudy.tar.gz>.
- [45] T. Inagaki, Y. Haribara, K. Igarashi, T. Sonobe, S. Tamate, T. Honjo, A. Marandi, P. L. McMahon, T. Umeki, K. Enbutsu, *et al.*, A coherent Ising machine for 2000-node optimization problems, *Science* **354**, 603 (2016).
- [46] Y. Ye, G-set test problems, <https://web.stanford.edu/~yyye/Gset/>.
- [47] C. Walshaw, The graph partitioning archive, <https://chriswalshaw.co.uk/partition/>.
- [48] M. J. Schuetz, J. K. Brubaker, and H. G. Katzgraber, Combinatorial optimization with physics-inspired graph neural networks, *Nature Machine Intelligence* **4**, 367 (2022).
- [49] H. Ushijima-Mwesigwa, C. F. Negre, and S. M. Mniszewski, Graph partitioning using quantum annealing on the D-Wave system, in *Proceedings of the Second International Workshop on Post Moores Era Supercomputing* (2017) pp. 22–29.
- [50] P. W. Holland, K. B. Laskey, and S. Leinhardt, Stochastic blockmodels: First steps, *Social Networks* **5**, 109 (1983).
- [51] B. Karrer and M. E. Newman, Stochastic blockmodels and community structure in networks, *Phys. Rev. E* **83**, 016107 (2011).
- [52] G. Karypis and V. Kumar, A fast and high quality multilevel scheme for partitioning irregular graphs, *SIAM Journal on Scientific Computing* **20**, 359 (1998).
- [53] P. Sanders and C. Schulz, Think locally, act globally: Highly balanced graph partitioning, in *International Symposium on Experimental Algorithms* (Springer, 2013) pp. 164–175.
- [54] S. Acer, E. G. Boman, C. A. Glusa, and S. Rajamanickam, Sphynx: A parallel multi-gpu graph partitioner for distributed-memory systems, *Parallel Computing* **106**, 102769 (2021).
- [55] J. Chuzhoy, Y. Gao, J. Li, D. Nanongkai, R. Peng, and T. Saranurak, A deterministic algorithm for balanced cut with applications to dynamic connectivity, flows, and beyond, in *2020 IEEE 61st Annual Symposium on Foundations of Computer Science (FOCS)* (IEEE, 2020) pp. 1158–1167.
- [56] W. K. Lam, *Hardware design verification: simulation and formal method-based approaches (Prentice Hall Modern semiconductor design series)* (Prentice Hall PTR, 2005).
- [57] S. Patil and D. Kulkarni, K-way spectral graph partitioning for load balancing in parallel computing, *International Journal of Information Technology* **13**, 1893 (2021).
- [58] V. D. Blondel, J.-L. Guillaume, R. Lambiotte, and E. Lefebvre, Fast unfolding of communities in large networks, *Journal of statistical mechanics: theory and experiment* **2008**, P10008 (2008).
- [59] The graph partitioning framework KaHIP, <https://github.com/KaHIP/KaHIP>.
- [60] B. Molnár, F. Molnár, M. Varga, Z. Toroczkai, and M. Ercsey-Ravasz, A continuous-time MaxSAT solver with high analog performance, *Nature Communications* **9**, 4864 (2018).
- [61] Eleventh Max-SAT Evaluation, <http://www.maxsat.udl.cat/16/benchmarks/index.html>.
- [62] S. A. Cook, The complexity of theorem-proving procedures, in *Proceedings of the Third Annual ACM Symposium on Theory of Computing*, STOC '71 (Association for Computing Machinery, New York, NY, USA, 1971) p. 151–158.
- [63] S. Cook, The P versus NP problem, *Clay Mathematics Institute* **2**, 6 (2000).
- [64] T. Tieleman, G. Hinton, *et al.*, Lecture 6.5-rmsprop: Divide the gradient by a running average of its recent magnitude, COURSERA: Neural networks for machine learning **4**, 26 (2012).
- [65] Optim tools of pytorch, <https://pytorch.org/docs/stable/optim.html>.
- [66] G. Bilbro, R. Mann, T. Miller, W. Snyder, D. van den Bout, and M. White, Optimization by mean field annealing, *Advances in Neural Information Processing Systems* **1** (1988).
- [67] Metis software package: version 5.1.0, <http://glaros.dtc.umn.edu/gkhome/metis/metis/download>.
- [68] P. Sanders and C. Schulz, Kahip v3.00–karlsruhe high quality partitioning–user guide, *arXiv preprint arXiv:1311.1714* (2013).
- [69] J. Fujisaki, H. Oshima, S. Sato, and K. Fujii, Practical and scalable decoder for topological quantum error correction with an Ising machine, *Phys. Rev. Research* **4**, 043086 (2022).
- [70] J. Fujisaki, K. Maruyama, H. Oshima, S. Sato, T. Sakashita, Y. Takeuchi, and K. Fujii, Quantum error correction with an Ising machine under circuit-level noise, *Phys. Rev. Research* **5**, 043261 (2023).

SUPPLEMENTARY MATERIAL: FREE-ENERGY MACHINE FOR COMBINATORIAL OPTIMIZATION

Derivation of variational mean-field free energy formulations for the benchmarking problems

According to the definition of variational mean-field free energy, namely,

$$F_{\text{MF}}(\{P_i(\sigma_i)\}, \beta) = U_{\text{MF}} - \frac{1}{\beta} S_{\text{MF}}, \quad (\text{S1})$$

since the entropy term is irrelevant to $E(\sigma)$, we have the same entropy form for all benchmarking problems, as

$$S_{\text{MF}} = - \sum_i \sum_{\sigma_i} P_i(\sigma_i) \ln P_i(\sigma_i), \quad (\text{S2})$$

and it suffices to derive the formulae for different mean-field internal energies U_{MF} .

For the maximum cut (MaxCut) problem, the energy function can be defined as the negation of the cut value,

$$E(\sigma) = - \sum_{(i,j) \in \mathcal{E}} W_{ij} [1 - \delta(\sigma_i, \sigma_j)], \quad (\text{S3})$$

where $\mathcal{E}$ is the edge set of the graph, W_{ij} is the weight of edge (i, j) , and $\delta(\sigma_i, \sigma_j)$ stands for the delta function, which takes value 1 if $\sigma_i = \sigma_j$ and 0 otherwise. Then the corresponding mean-field internal energy reads

$$U_{\text{MF}}^{\text{MaxCut}} = \sum_{\sigma} \prod_i P_i(\sigma_i) \left\{ - \sum_{(i,j) \in \mathcal{E}} W_{ij} [1 - \delta(\sigma_i, \sigma_j)] \right\} \quad (\text{S4})$$

$$= - \sum_{(i,j) \in \mathcal{E}} \sum_{\sigma_i, \sigma_j} W_{ij} P_i(\sigma_i) P_j(\sigma_j) [1 - \delta(\sigma_i, \sigma_j)] \quad (\text{S5})$$

$$= - \sum_{(i,j) \in \mathcal{E}} \sum_{\sigma_i} W_{ij} P_i(\sigma_i) [1 - P_j(\sigma_j)], \quad (\text{S6})$$

where identity $\sum_{\sigma_j} P_j(\sigma_j) \delta(\sigma_i, \sigma_j) = P_j(\sigma_i)$ is used in Eq. (S5)-Eq. (S6). Thus, we have

$$\begin{aligned} F_{\text{MF}}^{\text{MaxCut}}(\{P_i(\sigma_i)\}, \beta) &= - \sum_{(i,j) \in \mathcal{E}} \sum_{\sigma_i} W_{ij} P_i(\sigma_i) [1 - P_j(\sigma_i)] \\ &\quad + \frac{1}{\beta} \sum_i \sum_{\sigma_i} P_i(\sigma_i) \ln P_i(\sigma_i). \end{aligned} \quad (\text{S7})$$

The energy function designed for the bMinCut problem given in the main text is

$$E(\sigma) = \sum_{(i,j) \in \mathcal{E}} W_{ij} [1 - \delta(\sigma_i, \sigma_j)] + \lambda \sum_i \sum_{j \neq i} \delta(\sigma_i, \sigma_j), \quad (\text{S8})$$

and the corresponding mean field internal energy reads

$$U_{\text{MF}}^{\text{bMinCut}} = \sum_{\sigma} \prod_i P_i(\sigma_i) \left\{ \sum_{(i,j) \in \mathcal{E}} W_{ij} [1 - \delta(\sigma_i, \sigma_j)] + \lambda \sum_i \sum_{j \neq i} \delta(\sigma_i, \sigma_j) \right\} \quad (\text{S9})$$

$$= \sum_{(i,j) \in \mathcal{E}} \sum_{\sigma_i, \sigma_j} W_{ij} P_i(\sigma_i) P_j(\sigma_j) [1 - \delta(\sigma_i, \sigma_j)] + \lambda \sum_i \sum_{j \neq i} P_i(\sigma_i) P_j(\sigma_j) \delta(\sigma_i, \sigma_j) \quad (\text{S10})$$

$$= \sum_{(i,j) \in \mathcal{E}} \sum_{\sigma_i} W_{ij} P_i(\sigma_i) [1 - P_j(\sigma_i)] + \lambda \sum_{i,j} \sum_{\sigma_i} [P_i(\sigma_i) P_j(\sigma_i) - P_i^2(\sigma_i)]. \quad (\text{S11})$$

Thus, we have

$$\begin{aligned} F_{\text{MF}}^{\text{bMinCut}}(\{P_i(\sigma_i)\}, \beta) &= \sum_{(i,j) \in \mathcal{E}} \sum_{\sigma_i} W_{ij} P_i(\sigma_i) [1 - P_j(\sigma_i)] \\ &\quad + \lambda \sum_{i,j} \sum_{\sigma_i} [P_i(\sigma_i) P_j(\sigma_i) - P_i^2(\sigma_i)] \\ &\quad + \frac{1}{\beta} \sum_i \sum_{\sigma_i} P_i(\sigma_i) \ln P_i(\sigma_i). \end{aligned} \quad (\text{S12})$$

The cost function designed for the Max-SAT problem given in the main text reads

$$E(\sigma) = \sum_{m=1}^M \prod_{i \in \partial m} [1 - \delta(W_{mi}, \sigma_i)] . \quad (\text{S13})$$

The corresponding mean field internal energy is

$$U_{\text{MF}}^{\text{MaxSAT}} = \sum_{\sigma} \prod_i P_i(\sigma_i) \left\{ \sum_{m=1}^M \prod_{i \in \partial m} [1 - \delta(W_{mi}, \sigma_i)] \right\} \quad (\text{S14})$$

$$= \sum_{m=1}^M \prod_{i \in \partial m} \sum_{\sigma_i} P_i(\sigma_i) [1 - \delta(W_{mi}, \sigma_i)] \quad (\text{S15})$$

$$= \sum_{m=1}^M \prod_{i \in \partial m} [1 - P_i(W_{mi})] . \quad (\text{S16})$$

Thus, we have

$$F_{\text{MF}}^{\text{MaxSAT}}(\{P_i(\sigma_i)\}, \beta) = \sum_{m=1}^M \prod_{i \in \partial m} [1 - P_i(W_{mi})] + \frac{1}{\beta} \sum_i \sum_{\sigma_i} P_i(\sigma_i) \ln P_i(\sigma_i). \quad (\text{S17})$$

The Pytorch implementation of FEM using the automatic differentiation

The following shows the Pytorch code with detailed annotations of implementing of FEM, on the MaxCut and bMinCut problems, using the automatic differentiation. Remarkably, even with the substantial disparities in the number of variable states, the existence of constraints, and the characteristics of the objective function between these two problems, the coding implementations for each only vary by a mere line.

```

1  import torch
2
3  def read_graph(file, index_start=0):
4      """function for reading graph files
5      the specific format should be n m in the first line, and m following lines
6      represent source end weight
7      Parameters:
8          file: string, the filename of the graph to be read
9          index_start: int, specify which is the start index of the graph"""
10     with open(file, "r") as f:
11         l = f.readline()
12         n, m = [int(x) for x in l.split("_") if x != "\n"]
13         W = torch.zeros([n, n])
14         for k in range(m):
15             l = f.readline()
16             l_split = l.split()
17             i, j = [int(x) for x in l_split[:2]]
18             if len(l_split) == 2:
19                 w = 1.0 # if no weight, 1.0 by default
20             elif len(l_split) == 3:
21                 w = float(l_split[2])
22             else:
23                 raise ValueError("Unkown_graph_file_format")
24             W[i-index_start, j-index_start], W[j-index_start, i-index_start] = w, w
25     return n, m, W
26
27 def cut(W, p):
28     """Calculating the cut size given the weight matrix and the marginal matrix"""
29     p = p.to(W.dtype)

```

```

30     return ((W @ p) * (1-p)).sum((1, 2))
31
32 def S(p):
33     """Calculating the entropy of the marginal matrix p"""
34     return -(p*p.log()).sum(2).sum(1)
35
36 def infer_cut(W, p):
37     """Infer the cut size given the weight matrix and the marginal matrix"""
38     s = torch.nn.functional.one_hot(
39         p.argmax(dim=2), num_classes=p.shape[2])
40     return s, cut(W, s) / 2
41
42 def balance(p):
43     """Calculate the imbalance penalty of the marginal matrix p to restrict the group
44         partition to be balanced"""
45     return (p.sum(1)**2).sum(1)-(p**2).sum(2).sum(1)
46
47 def solve(problem, W, batch, q, penalty, beta_range, seed=0):
48     """Function for solving the combinatorial optimization problem using FEM"""
49     torch.manual_seed(seed)
50     n = W.shape[0]
51     h = torch.rand(batch, n, q) # field matrix with shape [batch, num_vertices,
52         num_classes]
53     h.requires_grad = True
54     optimizer = torch.optim.Adam([h], lr=0.01)
55     for beta in beta_range:
56         p = torch.softmax(h, dim=2) # normalize the field matrix to get the marginal
57             matrix
58         if problem == 'maxcut':
59             F = -cut(W,p) - S(p)/beta
60         if problem == 'bmincut':
61             F = cut(W,p)+penalty*balance(p)-S(p)/beta
62         optimizer.zero_grad()
63         F.backward(gradient=torch.ones_like(F))
64         optimizer.step()
65     return infer_cut(W, p)
66
67 n, m, W = read_graph('G1', index_start=1) # the data file can be retrieved from https://
68     web.stanford.edu/~yye/yye/Gset/
69
70 beta_range = 1 / torch.linspace(10.0, 0.01, 1100)
71 configs, cuts = solve('maxcut', W, 1000, 2, 5.0, beta_range)
72 ind = torch.argmax(cuts)
73 print(configs[ind][:, 0], cuts[ind])
74
75 configs, cuts = solve('bmincut', W, 1000, 4, 5.0, beta_range)
76 ind = torch.argmax(cuts)
77 print(configs[ind], cuts[ind])

```

The explicit gradient formulations

The key step in FEM involves computing the gradients of F_{MF} with respect to the local fields $\{h_i(\sigma_i)\}$, denoted as $\{g_i^h(\sigma_i)\}$. This task can be accomplished by leveraging the automatic differentiation techniques. Additionally, we have the option to write down the explicit gradient formula for each problem at hand. Once the specific form of $E(\sigma)$ is known, the form of F_{MF} is also determined. Thanks to the mean-field ansatz, the explicit formula for the gradients of F_{MF} with respect to $\{P_i(\sigma_i)\}$ can be obtained, denoted as $\{g_i^P(\sigma_i)\}$. The benefits of obtaining $\{g_i^P(\sigma_i)\}$ are twofold. Firstly, explicit gradient computation can lead to substantial time savings by eliminating the need for forward propagation calculations. Our numerical experiments have revealed that the application of explicit gradient formulations can reduce computational time by half. Secondly, it enables problem-dependent gradient manipulations based on $\{g_i^P(\sigma_i)\}$, denoted as $\{\hat{g}_i^P(\sigma_i)\}$, enhancing numerical stability and facilitating smoother optimization within the gradient descent framework, extending beyond the conventional use of adaptive learning rates and momentum techniques commonly found in gradient-based optimization methods within the realm of machine learning.

Hence, in this work, we mainly adopt the explicit gradient approach for benchmarking FEM, and we use the manipulated gradients $\{\hat{g}_i^p(\sigma_i)\}$ to compute $\{g_i^h(\sigma_i)\}$. According to the chain rule for partial derivative calculation, $\{g_i^h(\sigma_i)\}$ can be computed from

$$g_i^h(\sigma_i) = \sum_{\sigma'_i} \frac{\partial F_{\text{MF}}}{\partial P_i(\sigma'_i)} \frac{\partial P_i(\sigma'_i)}{\partial h_i(\sigma_i)}, \quad (\text{S18})$$

and since $P_i(\sigma_i) = e^{h_i(\sigma_i)}/Z$, where $Z = \sum_{\sigma'_i} e^{h_i(\sigma'_i)}$ is the normalization factor, we also have

$$\frac{\partial P_i(\sigma'_i)}{\partial h_i(\sigma_i)} = \begin{cases} -\frac{e^{h_i(\sigma'_i)}}{Z} \frac{e^{h_i(\sigma_i)}}{Z} = -P_i(\sigma'_i)P_i(\sigma_i), & \sigma'_i \neq \sigma_i \\ \frac{e^{h_i(\sigma_i)}}{Z} (1 - \frac{e^{h_i(\sigma_i)}}{Z}) = P_i(\sigma_i)[1 - P_i(\sigma_i)], & \sigma'_i = \sigma_i \end{cases} \quad (\text{S19})$$

Therefore, by substituting Eq. (S19) into Eq. (S18) and employing the modified gradient variable $\hat{g}_i^p(\sigma'_i)$ in place of $\frac{\partial F_{\text{MF}}}{\partial P_i(\sigma'_i)}$ (instead of using $g_i^p(\sigma_i)$ directly), we have the following unified form for $\{g_i^h(\sigma_i)\}$,

$$g_i^h(\sigma_i) = \gamma_{\text{grad}} \left[\hat{g}_i^p(\sigma_i) - \sum_{\sigma'_i=1}^q P_i(\sigma'_i) \hat{g}_i^p(\sigma'_i) \right] P_i(\sigma_i), \quad (\text{S20})$$

where γ_{grad} is a hyperparameter that controls the magnitudes of $\{g_i^h(\sigma_i)\}$ to accommodate different optimizers. Once we have the values of $\{P_i(\sigma_i)\}$ and $\{\hat{g}_i^p(\sigma_i)\}$ (also computed using $\{P_i(\sigma_i)\}$), we can obtain $\{g_i^h(\sigma_i)\}$ immediately according to Eq. (S20).

The explicit gradients and manipulated gradients for the benchmarking problems

The MaxCut problem

The explicit gradients of $\{g_i^p(\sigma_i)\}$ for the MaxCut problem can be derived analytically from $F_{\text{MF}}^{\text{MaxCut}}$, as

$$g_i^p(\sigma_i) = \nabla_{P_i(\sigma_i)} F_{\text{MF}}^{\text{MaxCut}} = \sum_j W_{ij} [2P_j(\sigma_i) - 1] + \frac{1}{\beta} [\ln P_i(\sigma_i) + 1]. \quad (\text{S21})$$

The other derivation method may be based on by rewriting Eq. (S6) as

$$U_{\text{MF}}^{\text{MaxCut}} = - \sum_{(i,j) \in \mathcal{E}} \sum_{\sigma_i} W_{ij} P_i(\sigma_i) [1 - P_j(\sigma_i)] \quad (\text{S22})$$

$$= - \sum_{(i,j) \in \mathcal{E}} W_{ij} + \sum_{(i,j) \in \mathcal{E}} W_{ij} P_i(\sigma_i) P_j(\sigma_i), \quad (\text{S23})$$

where the first term $-\sum_{(i,j) \in \mathcal{E}} W_{ij}$ is a constant. Hence, we also have the following different formula

$$g_i^p(\sigma_i) = 2 \sum_j W_{ij} P_j(\sigma_i) + \frac{1}{\beta} [\ln P_i(\sigma_i) + 1]. \quad (\text{S24})$$

However, using Eq. (S21) or Eq. (S24) will result in the same result for computing $g_i^h(\sigma_i)$, for the reason that the constant $-\sum_j W_{ij}$ in Eq. (S21) for each index i will be canceled when computing the gradients for the local fields using Eq. (S20). Then the manipulated gradients can be designed as follows

$$\hat{g}_i^p(\sigma_i) = c_i \sum_j W_{ij} e_j(\sigma_i) + \frac{1}{\beta} [\ln P_i(\sigma_i) + 1], \quad (\text{S25})$$

where two modifications on gradient have been made to improve the optimization performance. Firstly, $[e_j(1), e_j(2), \dots, e_j(q)]$ is the one-hot vector (length of 2 in MaxCut problem) corresponding to $[P_j(1), P_j(2), \dots, P_j(q)]$. The introduction of $\{e_j(\sigma_j)\}$ to replace $\{P_j(\sigma_j)\}$ is called the discretization that enables reducing the analog errors introduced by $\sum_j W_{ij} P_j(\sigma_j)$ in the explicit gradients. Note that the similar numerical tricks have been also employed in the previous work [11]. Secondly, when the graph has inhomogeneity in the node degrees or the edge weights, the magnitude of $\sum_j W_{ij} e_j(\sigma_j)$ for each spin variable can

exhibit significant differences. Hence, the gradient normalization factor c_i enables robust optimization and better numerical performance.

For the MaxCut problem, the gradient normalization factor c_i is set to $c_i = \frac{1}{\sum_j |W_{ij}|}$ in this work. In this context, the L_1 norm, represented by $\sum_j |W_{ij}|$, normalizes the gradient magnitude for each spin, ensuring that the values of $\sum_j W_{ij}e_j(\sigma_j)$ across all spins remain below one. This normalization is critical. Since the range of the gradients of the entropy term, given by $\ln P_i(\sigma_i) + 1$, is consistent across all spins. In our experiments, we found the constrains made on the gradient magnitude of the internal energy can prevent the system from becoming ensnared in local minima. Although other tricks for the normalization factors can be employed, we have observed that our current settings yield satisfactory performance in our numerical experiments.

The bMinCut problem

In the bMinCut problem, the gradients with respect to $P_i(\sigma_i)$ are

$$g_i^p(\sigma_i) = \nabla_{P_i(\sigma_i)} F_{\text{MF}}^{\text{bMinCut}} = -2 \sum_j W_{ij} P_j(\sigma_j) + 2\lambda \left[\sum_j P_j(\sigma_j) - P_i(\sigma_i) \right] + \frac{1}{\beta} [\ln P_i(\sigma_i) + 1] . \quad (\text{S26})$$

The modifications can be made for better optimization on graphs with different topologies, as done in the MaxCut problem. We have the following manipulated gradients

$$\hat{g}_i^p(\sigma_i) = -c_i^f \sum_j W_{ij} e_j(\sigma_j) + \lambda c_i^a \left[\sum_j e_j(\sigma_j) - e_i(\sigma_i) \right] + \frac{1}{\beta} [\ln P_i(\sigma_i) + 1] , \quad (\text{S27})$$

where c_i^f and c_i^a are the gradient normalization factors for the ferromagnetic and antiferromagnetic terms, respectively. The $\{e_i(\sigma_i)\}$ are again the one-hot vectors for $\{P_i(\sigma_i)\}$, which serves to mitigate the analog error introduced in the explicit gradients, as we done in the MaxCut problem.

For the bMinCut problem, analogous to the approach taken with the MaxCut problem, the ferromagnetic normalization factor c_i^f is defined as $c_i^f = \frac{q}{\sum_j W_{ij}}$ (in the bMinCut problem, $W_{ij} > 0$), while the antiferromagnetic normalization factor c_i^a is determined by $c_i^a = \frac{q}{\sqrt{\sum_j W_{ij}}}$. The rationale behind the setting for c_i^a stems from the intuition that spins with larger values of $\sum_j W_{ij}$ should remain in their current states, implying that they should not be significantly influenced by the antiferromagnetic force to transition into other states. Although these settings for the normalization factors may not be optimal, and alternative schemes could be implemented, we have observed that these particular settings yield satisfactory performance in our numerical experiments.

The MaxSAT problem

The explicit gradient can be readily computed as follows

$$g_i^p(\sigma_i) = - \sum_{m=1}^M \delta(\sigma_i, W_{mi}) \prod_{k \in \partial m, k \neq i} [1 - P_k(W_{kj})] + \frac{1}{\beta} [\ln P_i(\sigma_i) + 1] . \quad (\text{S28})$$

Note that, for each m in the summation, the spin variable σ_i must appear as a literal in the clause C_m , otherwise the gradients of the internal energy with respect to σ_i are zeros in this clause. For the random Max k -SAT problem benchmarked in this work, we make no modifications. Therefore, we set $\hat{g}_i^p(\sigma_i)$ equal to $g_i^p(\sigma_i)$.

Numerical experiments on the MaxCut problem

Simplification of FEM for solving the MaxCut problem

To facilitate an efficient implementation, we can simplify FEM's approach for solving the Ising problem since the gradients for $\{h_i(+1)\}$ and $\{h_i(-1)\}$ are dependent. In the MaxCut problem with $q = 2$ ($\sigma_i = +1$ or $\sigma_i = -1$), it is straightforward

to prove $g_i^h(+1) = -g_i^h(-1) = \gamma_{\text{grad}}[\hat{g}_i^p(+1) - \hat{g}_i^p(-1)]P_i(+1)P_i(-1)$ from Eq. (S20). Given that $P_i(+1) = 1 - P_i(-1) = e^{h_i(+1)} / (e^{h_i(+1)} + e^{h_i(-1)}) = \text{sigmoid}(h_i(+1) - h_i(-1))$, we actually only need to update $\{P_i(+1)\}$ (or simply $\{P_i\}$) to save considerable computational resources. Thus, we introduce the new local field variables $\{h_i\}$ to replace $\{h_i(+1) - h_i(-1)\}$, such that $P_i = \text{sigmoid}(h_i)$.

According to Eq. (S25), $\hat{g}_i^p(+1) = c_i \sum_j W_{ij} e_j(+1) + \frac{1}{\beta} [\ln P_i(+1) + 1]$ and $\hat{g}_i^p(-1) = c_i \sum_j W_{ij} e_j(-1) + \frac{1}{\beta} [\ln P_i(-1) + 1]$. Based on $g_i^h(+1) = \gamma_{\text{grad}}[\hat{g}_i^p(+1) - \hat{g}_i^p(-1)]P_i(+1)P_i(-1)$, the gradients regarding to local fields $\{h_i\}$ can be written as

$$\begin{aligned} g_i^h &= \gamma_{\text{grad}}[\hat{g}_i^p(+1) - \hat{g}_i^p(-1)]P_i(+1)P_i(-1) \\ &= \gamma_{\text{grad}} \left[c_i \sum_j W_{ij} (2e_j(+1) - 1) + \frac{1}{\beta} \ln \frac{P_i}{1 - P_i} \right] P_i(1 - P_i). \end{aligned} \quad (\text{S29})$$

We can further simplify Eq. (S29) by introducing the magnetization $m_i = 2P_i - 1 = \tanh(h_i/2)$, such that

$$g_i^h = \frac{\gamma_{\text{grad}}}{4} \left[c_i \sum_j W_{ij} \text{sgn}(m_j) + \frac{1}{\beta} h_i \right] (1 - m_i^2), \quad (\text{S30})$$

where $\text{sgn}(\cdot)$ is the sign function, and the identity $\text{arctanh}(x) = \frac{1}{2} \ln \frac{1+x}{1-x}$ has been used for the simplifications. Thus, we have used $\{m_i\}$ and $\{h_i\}$ to simplify the original gradients, requiring optimizations for only half of the variational variables as compared to the non-simplified case.

The numerical experiment of the MaxCut problem on the complete graph K_{2000}

For the numerical experiment of the MaxCut problem on the complete graph K_{2000} , as shown in Fig. 3(a) in the main text (where we evaluate the performance by only varying the total annealing steps N_{step} while keeping other hyperparameters unchanged), we implemented the dSBM algorithm, which is about to simulate the following Hamiltonian equations of motion [11]

$$y_i(t_{k+1}) = y_i(t_k) + \left\{ -[a_0 - a(t_k)]x_i(t_k) + c_0 \sum_{j=1}^N W_{ij} \text{sgn}[x_j(t_k)] \right\} \Delta_t, \quad (\text{S31})$$

$$x_i(t_{k+1}) = x_i(t_k) + a_0 y_i(t_{k+1}) \Delta_t, \quad (\text{S32})$$

where x_i and y_i represent the position and momentum of a particle corresponding to the i -th spin in a N -particle dynamical system, respectively, Δ_t is the time step, t_k is discrete time with $t_{k+1} = t_k + \Delta_t$, J is the edge weight matrix, $a(t_k)$ is the bifurcation parameter linearly increased from 0 to $a_0 = 1$, and $c_0 = 0.5 \sqrt{\frac{N-1}{\sum_{i,j} W_{ij}^2}}$ is according to the settings in Ref. [11]. In addition, at every t_k , if $|x_i| > 1$, we set $x_i = \text{sgn}(x_i)$ and $y_i = 0$. For the dSBM benchmarks on K_{2000} , we set $\Delta_t = 1.25$ following the recommended settings in Ref. [11], and the initial values of x_i and y_i are randomly initialized from the range $[-0.1, 0.1]$. Regarding FEM, we employ the explicit gradient formulations, setting the values of $\gamma_{\text{grad}} = 1$, $T_{\text{max}} = 1.16$, $T_{\text{min}} = 6e-5$, and utilize the inverse-proportional scheduling for annealing. We employ RMSprop as the optimizer, with the optimizer hyperparameters alpha, momentum, weight decay, and learning rate set to 0.56, 0.63, 0.013, and 0.03, respectively. Both dSBM and FEM were executed on a GPU.

The numerical experiment of the MaxCut problem on the G-set

For the benchmarks on the G-set problems, we have presented the detailed TTS results obtained by FEM in Tab. S1, along with a comparison of the reported data provided by dSBM. Given the capability of FEM for optimizing many replicas parallelly, here we assess the TTS using the batch processing method introduced in Ref. [11]. All the parameter settings for FEM are listed in Tab. S2. We also utilized the same GPU used in Ref. [11] for implementing FEM in this benchmark. In this study, we adopt the batch processing method as introduced in Ref. [11] for calculating TTS. Therefore, for an accurate comparison, the values of N_{rep} for each instance shown in Tab. S2 are consistent with those used for dSBM in Ref. [11]. Throughout the benchmarking, we initialize the local fields with random values according to $h_i(\sigma_i)^{\text{ini}} = 0.001 * \text{randn}$, where randn represents a random number sampled from the standard Gaussian distribution. All variables are represented using 32-bit single-precision floating-point numbers.

TABLE S1: **Benchmarking results on the G-Set instances.** The TTS is defined as the time taken to achieve the best cut found by the solver, and the parentheses for the TTS results indicate that the best cut for computing TTS is not the best known cut. Shorter TTS results are highlighted in bold.

Graph type	Instance	N	Best known	FEM			dSBM		
				Best cut	TTS(ms)	P_s	Best cut	TTS(ms)	P_s
Random	G1	800	11624	11624	24.9	100.0%	11624	33.3	98.7%
	G2	800	11620	11620	96.5	99.6%	11620	239	82%
	G3	800	11622	11622	23.3	100.0%	11622	46.2	99.6%
	G4	800	11646	11646	19.8	99.5%	11646	34.4	98.3%
	G5	800	11631	11631	21.6	98.9%	11631	58.6	97.2%
	G6	800	2178	2178	5.6	95.5%	2178	6.3	97.9%
	G7	800	2006	2006	11.5	98.6%	2006	6.85	97.4%
	G8	800	2005	2005	21.3	98.5%	2005	11.9	95.4%
	G9	800	2054	2054	35.6	98.8%	2054	36	86.7%
	G10	800	2000	2000	193	53.2%	2000	47.7	40.7%
Toroidal	G11	800	564	564	24.2	99.0%	564	3.49	98%
	G12	800	556	556	31.2	97.8%	556	5.16	97.3%
	G13	800	582	582	203	63.9%	582	11.9	99.6%
Planar	G14	800	3064	3064	2689	36.5%	3064	71633	0.5%
	G15	800	3050	3050	164	96.5%	3050	340	80.4%
	G16	800	3052	3052	165	99.8%	3052	347	99.2%
	G17	800	3047	3047	800	70.2%	3047	1631	28.3%
	G18	800	992	992	264	37.9%	992	375	7.4%
	G19	800	906	906	17.5	98.8%	906	17.8	99.5%
	G20	800	941	941	8.5	99.2%	941	9.02	98%
	G21	800	931	931	67.3	34%	931	260	13.6%
Random	G22	2000	13359	13359	917	56.3%	13359	429	92.8%
	G23	2000	13344	13342	(98)	36.9%	13342	(89)	-
	G24	2000	13337	13337	1262	92.4%	13337	459	64.8%
	G25	2000	13340	13340	5123	31.9%	13340	2279	39.9%
	G26	2000	13328	13328	991	83.2%	13328	476	64.3%
	G27	2000	3341	3341	127	90.1%	3341	49.9	97.1%
	G28	2000	3298	3298	306	82.7%	3298	87.2	95.2%
	G29	2000	3405	3405	200	98.6%	3405	221	73.7%
	G30	2000	3413	3413	948	64.7%	3413	439	73.8%
	G31	2000	3310	3310	4523	19.6%	3310	1201	19.9%
Toroidal	G32	2000	1410	1410	23749	1.3%	1410	3622	9.3%
	G33	2000	1382	1382	659607	0.6%	1382	57766	0.5%
	G34	2000	1384	1384	12643	28.1%	1384	2057	23.1%
Planar	G35	2000	7687	7686	(5139390)	0.01%	7686	(8319000)	-
	G36	2000	7680	7680	5157009	0.01%	7680	62646570	0.01%
	G37	2000	7691	7690	(3509541)	0.01%	7691	27343457	0.02%
	G38	2000	7688	7688	41116	7.3%	7688	98519	6.8%
	G39	2000	2408	2408	12461	17.5%	2408	56013	10.7%
	G40	2000	2400	2400	3313	54.1%	2400	24131	15.4%
	G41	2000	2405	2405	1921	80.9%	2405	10585	28.2%
	G42	2000	2481	2481	91405	0.23%	2480	(550000)	-
Random	G43	1000	6660	6660	19.8	66.1%	6660	5.86	99.2%
	G44	1000	6650	6650	13.2	80.1%	6650	6.5	98.5%
	G45	1000	6654	6654	35	98.7%	6654	43.4	98.5%
	G46	1000	6649	6649	141	69.8%	6649	16	99.2%
	G47	1000	6657	6657	33.9	98.7%	6657	44.8	98.2%
	G48	3000	6000	6000	0.35	95.3%	6000	0.824	100.0%

to be continued...

Graph type	Instance	N	Best known	FEM			dSBM		
				Best cut	TTS(ms)	P_s	Best cut	TTS(ms)	P_s
Toroidal	G49	3000	6000	6000	0.66	97.8%	6000	0.784	99.5%
	G50	3000	5880	5880	2.01	85.1%	5880	2.63	100.0%
Planar	G51	1000	3848	3848	5268	16.1%	3848	12209	6.7%
	G52	1000	3851	3851	4580	30.4%	3851	6937	21.3%
	G53	1000	3850	3850	6155	24.1%	3850	93899	4.3%
	G54	1000	3852	3852	55055	0.24%	3852	2307235	0.06%

TABLE S2: **The hyperparameter settings for FEM in the TTS benchmarking on G-set instances.** Here, we utilize different optimizers, SGD and RMDprop, for different sets of instances in this benchmarking based on their performance. For the explanations of the hyperparameters of the different optimizers. The inverse-proportional scheduling is used for the annealing. The meanings of N_{step} , N_{batch} and N_{rep} are the same with the TTS experiments documented in Ref. [11]. For N_{batch} in FEM, we refer to the number of replicas.

Ins.	T_{max}	T_{min}	γ_{grad}	Optimizer	lr	alpha	dampening	weight decay	momentum	N_{step}	N_{batch}	N_{rep}
G1	0.5	8e-5	1	RMSprop	0.2	0.623	-	0.02	0.693	1000	130	1000
G2	0.2592	6.34e-4	1	RMSprop	0.0717	0.5485	-	0.0264	0.9082	5000	100	1000
G3	0.264	1.1e-3	1	RMSprop	0.3174	0.7765	-	0.00672	0.7804	1000	120	1000
G4	0.29	8.9e-4	1	RMSprop	0.2691	0.4718	-	0.00616	0.7414	800	130	1000
G5	0.2	9e-4	1	RMSprop	0.24	0.9999	-	0.0056	0.8215	1000	110	1000
G6	0.44	1.7e-3	1	RMSprop	0.534	0.6045	-	0.00657	0.4733	1000	20	1000
G7	0.54	1.8e-3	1	RMSprop	0.452	0.8966	-	0.0087	0.632	700	80	1000
G8	0.19	7.92e-4	1	RMSprop	0.296	0.9999	-	0.00731	0.737	1000	100	1000
G9	0.208	9e-4	1	RMSprop	0.305	0.9999	-	0.00205	0.718	2500	70	1000
G10	1.28	5.21e-6	0.75	SGD	1.2	-	0.082	0.03	0.88	2000	100	1000
G11	1.28	4.96e-6	0.98	SGD	1.2	-	0.13	0.061	0.88	1800	120	1000
G12	1.28	7.8e-6	0.65	SGD	1.98	-	0.13	0.06	0.88	1600	140	1000
G13	1.28	3.12e-6	1.7	SGD	3	-	0.082	0.033	0.76	3000	130	1000
G14	0.387	8.64e-4	1	RMSprop	0.44	0.9999	-	0.0089	0.793	7000	250	1000
G15	0.5	1e-3	1	RMSprop	0.45	0.9999	-	0.0056	0.7327	4000	200	1000
G16	0.54	8.1e-4	1	RMSprop	0.288	0.9999	-	0.00756	0.7877	7000	160	1000
G17	0.253	1.06e-3	1	RMSprop	0.631	0.9999	-	0.01341	0.7642	7000	200	1000
G18	0.4	1e-3	1	RMSprop	0.345	0.99	-	0.01	0.9	1200	150	1000
G19	0.962	3.98e-6	1.75	SGD	4.368	-	0.05175	0.01336	0.729	1700	85	1000
G20	0.37	9.4e-4	1.55	RMSprop	1.38	0.9089	-	0.00445	0.8186	500	100	1000
G21	0.6	9.6e-4	1	RMSprop	0.33	0.9999	-	0.0092	0.692	1000	40	1000
G22	0.352	2.4e-4	1	RMSprop	0.481	0.9999	-	0.00382	0.7166	4700	90	1000
G23	0.406	1.15e-6	2.72	SGD	8.042	-	0.1443	0.00184	0.714	3200	10	1000
G24	0.528	1.6e-4	1	RMSprop	0.39	0.9999	-	0.00413	0.74	7000	250	1000
G25	0.4	4.83e-6	5.33	SGD	3.66	-	0.0905	0.00987	0.672	7000	200	1000
G26	0.361	4.43e-6	2.18	SGD	8.46	-	0.0612	0.0078	0.714	6000	200	1000
G27	0.28	5e-4	1	RMSprop	0.7	0.9995	-	0.00575	0.78	2000	80	1000
G28	0.32	5e-4	1	RMSprop	0.69	0.999	-	0.006	0.78	3000	100	1000
G29	0.38	2.7e-4	1	RMSprop	0.44	0.9999	-	0.013	0.7	4000	120	1000
G30	0.96	4.92e-6	1.9	SGD	2.59	-	0.05	0.053	0.715	7000	100	1000
G31	1.834	2.76e-6	1.32	SGD	1.38	-	0.0104	0.083	0.7566	7000	100	1000
G32	0.89	1.42e-5	3.17	SGD	1.67	-	0.1285	0.018	0.9	12000	20	1000
G33	0.605	7.8e-6	2	SGD	4.05	-	0.098	0.0366	0.91	12000	260	1000
G34	0.605	6.24e-6	2.33	SGD	2.638	-	0.1182	0.0384	0.8967	12000	260	1000

to be continued...

Ins.	$T_{\max}$	$T_{\min}$	γ_{grad}	Optimizer	lr	alpha	dampening	weight decay	dampening	N_{step}	N_{batch}	N_{rep}
G35	0.9	1e-4	1	RMSprop	0.023	0.9999	-	0.016	0.92	15000	20	10000
G36	1	1e-3	1	RMSprop	0.1	0.999	-	0.025	0.89	12000	25	10000
G37	0.9	1e-4	1	RMSprop	0.03	0.999	-	0.02	0.92	10000	20	10000
G38	0.4	8e-4	1	RMSprop	0.3	0.9999	-	0.0113	0.8595	7000	260	1000
G39	0.76	1.5e-4	1	RMSprop	0.064	0.9999	-	0.0264	0.9081	7000	200	1000
G40	0.95	1.1e-4	1	RMSprop	0.0525	0.9999	-	0.029	0.9082	10000	150	1000
G41	0.655	1.32e-5	4.61	SGD	1.345	-	0.0725	0.0092	0.897	12000	200	1000
G42	1	1e-4	1	RMSprop	0.096	0.9999	-	0.024	0.73275	8000	10	10000
G43	0.65	6e-4	1	SGD	6.29	-	0.077	0.0285	0.7515	1000	30	1000
G44	0.65	7e-4	1.2	SGD	5.8	-	0.097	0.026	0.7554	1000	30	1000
G45	0.63	8.4e-4	1.36	SGD	6.1	-	0.129	0.01	0.755	3000	70	1000
G46	0.504	8.11e-6	2.07	SGD	1.54	-	0.156	0.0295	0.8965	2000	120	1000
G47	0.58	5.4e-4	1	SGD	7.5	-	0.13	0.026	0.76	3000	70	1000
G48	1.34	1e-3	1	SGD	5.5	-	0.08	0.032	0.737	180	3	1000
G49	1.77	5.9e-4	1	SGD	6.415	-	0.42	0.073	0.572	200	6	1000
G50	22.94	3.54e-5	0.833	SGD	0.436	-	0.0617	0.0503	0.3335	200	10	1000
G51	1.48	6.5e-6	1	SGD	1.345	-	0.283	0.029	0.863	7000	200	1000
G52	0.604	4.2e-6	2.4	SGD	2.9	-	0.19	0.027	0.81	10000	250	1000
G53	0.27	3.5e-6	10.8	SGD	6	-	0.35	0.015	0.79	10000	250	1000
G54	0.63	1e-5	6	SGD	1.27	-	0.11	0.018	0.71	10000	20	10000

Effects of the number of replicas to the cut-value distribution

We explored how varying the number of replicas impacts the cut value distribution among replicas in the MaxCut problem on G55 in the G-set dataset. It is a random graph with 5000 nodes and $\{+1, -1\}$ edge weights. After optimizing FEM's hyperparameters, we incrementally increased the number of replicas R to examine changes in the cut value distribution. The histograms of the cut values with different R values are shown in Fig. S1(a). In Fig. S1(b), we found that the average cut value remains stable with different R values crossing several magnitudes. We also see that the standard deviation is also quite stable as shown in Fig. S1(c). As a consequence, the maximum cut value achieved by FEM is an increasing function of R . It is clearly shown in Fig. S1(a) that the maximum cut value of FEM approaches the best-known results for G55 when R increases. These findings also suggest that the hyperparameters of FEM can be fine-tuned using the mean cut value at a small R , while the final results can be obtained using a large R with fine-tuned parameters.

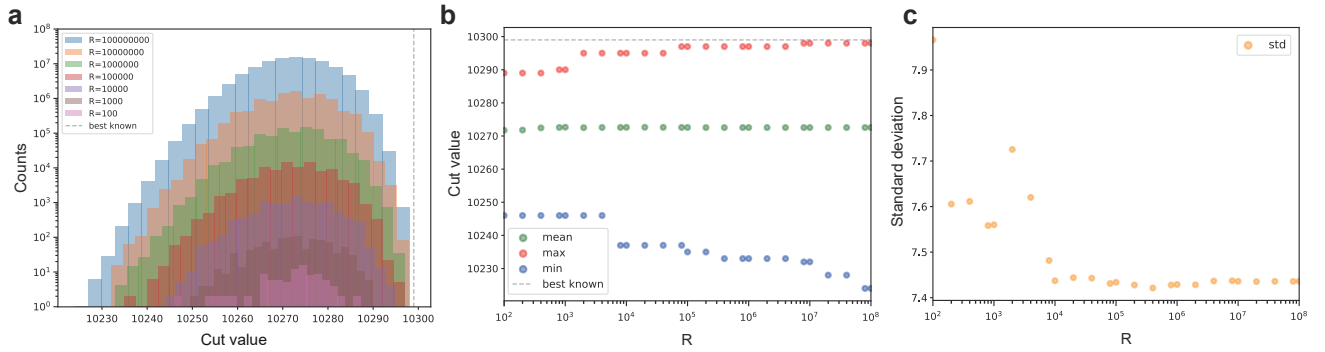


FIG. S1. The cut values as a function of the number of replicas R used in FEM for the MaxCut Problem on the G55 dataset. (a). The histograms of the cut values with the different R . (b). The maximum, minimum, and mean cut values as a function of R . (c). The standard deviation of the cut-value distribution in replicas with R changes.

Additional numerical experiments on the balanced minimum cut problem

Demonstration of FEM solving the bMinCut problem

To elucidate the principles of FEM in tackling q -way bMinCut problem in the main text, firstly, we provide an example to demonstrate the numerical details in our experiments. We employ the real-world graph named *3elt* as a demonstrative case study. This graph, which is collected in Chris Walshaw’s graph partitioning archive [47], comprises 4,720 nodes and 13,722 edges, and we specifically address its 4-way bMinCut problem as an illustrative example. Fig. S2(a) shows the evolution of the marginal probabilities for 4 states for a typical variable σ_i . From the figure, we can identify 3 optimization stages. At early annealing steps, the 4 marginal probabilities $\{P_i(\sigma_i = 1), P_i(\sigma_i = 2), P_i(\sigma_i = 3), P_i(\sigma_i = 4)\}$ are all very close to the initial value 0.25; with annealing step increases, the marginal probabilities for 4 states begin to fluctuate; then finally converge. Only one probability converges to unity and the other three probabilities converge to 0. This indicates that during annealing and the minimization of mean-field free energy, the marginal probability will evolve towards localizing on a single state. In Fig. S2(b), we plot the evolutions of the cut values and the largest group sizes (the largest value of the sizes among all groups) averaged over $R = 1000$ replicas of mean-field approximations. In our approach, the hyperparameter λ linearly increases from 0 to $\lambda_{\max}$ with the annealing step, for preferentially searching the states that minimize the cut value at the initial optimization stage. From the figure, we can see that our algorithm first searches for a low-cut but un-balanced solution with a large group and three small groups, then gradually decreases the maximum group, and finally finds a balanced solution with a global minimum cut.

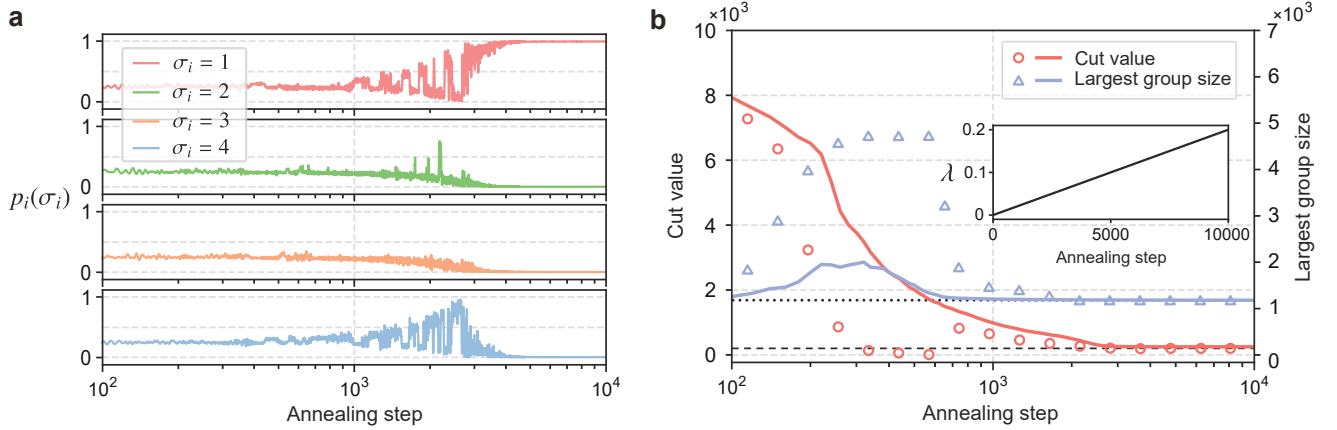


FIG. S2. **Illustrative example of the 4-way perfectly bMinCut problem on the real-world graph *3elt*.** (a). The evolutions of the state probabilities of four typical spins in four allowed state classes. (b). The evolutions of the cut values and the largest class sizes in 1000 replicas. The circles denote the minimum cut values, and triangles indicate the maximum values of the largest class size, with solid lines illustrating the average values. The dotted line indicates the perfectly balanced class size 1180. The dashed line indicate the best known minimal cut 201. Inset: The hyperparameter λ is linearly increased with the annealing step from 0 to $\lambda_{\max} = 0.2$.

Benchmarking experiments for the bMinCut problem on the real-world graphs

In our benchmarking experiments, we utilized the latest release of the METIS software, with version 5.1.0 [67], specifically employing the stand-alone program named *gpmets* for the bMinCut tasks. Throughout the experiment, we consistently configured *gpmets* with specific options as outlined in the documentation [67]: *-ptype=rb*, which denotes multilevel recursive bisectioning, and *-ufactor=1* (1 is the lowest permissible integer value allowed by METIS, indicating that while a perfectly balanced partitioning cannot be guaranteed, we aim to approach it as closely as possible). All other parameters or options were left at their default settings.

For KaHIP, we used the advanced variant of KaHIP named KaFFPaE [59] for the bMinCut tasks, with the following settings (see also the documentation [68] for more explanations): *-n=24* (number of processes to use), *-time_limit=300* (limiting the execution time to 300 seconds), *-imbalance=0*, *-preconfiguration=strong*, and we also enabled *-mh_enable_tabu_search* and *-mh_enable_kabapE* to optimize performance. Both the programs of *gpmets* and KaFFPaE, implemented in C, were executed on a computing node equipped with dual 24-core processors with 2.90GHz.

For FEM, we have detailed hyperparameter settings in Tab. S3. Throughout the experiments, we initialize the local fields with random values according to $h_i(\sigma_i)^{ini} = 0.001 * \text{randn}$, where randn represents a random number sampled from the standard Gaussian distribution. The execution of FEM was performed on a GPU. All variables are represented using 32-bit single-precision floating-point numbers.

TABLE S3: **The hyperparameter settings for FEM in the bMinCut benchmarking on real-world graphs.** The optimization algorithm employed in this experiment is Adam. Note, the imbalance penalty coefficient, λ , is progressively incremented from 0 to its maximum value, λ_{max} , over N_{step} steps. The term RC time refers to the abbreviation of replica computation time when running FEM. The exponential scheduling is used for the annealing.

Graph	q	β_{min}	β_{max}	λ_{max}	N_{step}	$N_{replica}$	γ_{grad}	lr	weight decay	β_1	β_2	RC time(sec.)
<i>add20</i>	2	2.12e-2	756	1	10000	2000	17.8	0.2914	3.4e-4	0.9408	0.7829	24.2
	4	5.054e-2	840.84	0.2332	10000	2000	13.33	0.3664	3.411e-3	0.9158	0.7691	46.2
	8	3.048e-2	2178.64	0.6229	10000	2000	10.05	0.4564	4.198e-4	0.9018	0.7225	90.2
	16	3.634e-2	1607.45	1.0553	10000	2000	81.04	0.6564	8.264e-3	0.9032	0.6009	181.2
	32	3.77e-2	2827.44	1.9607	10000	2000	7.384	1.4246	0.01719	0.911	0.8199	360.1
<i>data</i>	2	5.054e-2	840.84	0.2292	10000	2000	13.328	0.2629	1.706e-3	0.9347	0.7692	29.1
	4	5.054e-2	840.84	0.2292	10000	2000	13.328	0.2629	1.706e-3	0.9347	0.7692	57.1
	8	5.054e-2	840.84	0.3438	10000	2000	13.328	0.3023	1.365e-3	0.9369	0.8076	109.2
	16	7.316e-2	1766.14	1.3376	10000	2000	15.736	0.8358	1.874e-4	0.7801	0.4039	217.5
	32	1.968e-2	613.88	0.697	12000	2000	57.56	0.6633	1.65e-3	0.5263	0.4681	519
<i>3elt</i>	2	3.648e-2	2458.64	2.07	10000	2000	7.146	1.2032	2.293e-2	0.9374	0.9215	45.8
	4	3.648e-2	2458.64	2.07	10000	2000	7.146	1.2032	2.293e-2	0.9374	0.9215	90.2
	8	2.918e-2	1966.92	0.8	10000	2000	7.384	0.2117	1.031e-3	0.724	0.5397	177.1
	16	3.648e-2	2458.64	1.267	10000	2000	7.146	0.2238	2.577e-3	0.77	0.7698	355.3
	32	4.626e-2	3581.42	1.369	12000	2000	6.768	0.9946	2.508e-2	0.777	0.8982	850
<i>bcsstk33</i>	2	3.648e-2	2458.64	2.07	10000	2000	7.146	1.2032	2.2926e-2	0.9374	0.9215	103
	4	3.648e-2	2458.64	2.07	10000	2000	7.146	1.2032	2.2926e-2	0.9374	0.9215	204
	8	3.72e-2	754.22	0.5016	12000	2000	15.6	0.1948	1.663e-3	0.7196	0.9	480
	16	3.336e-2	950	1.167	12000	2000	7.052	0.5254	3.089e-3	0.8894	0.6223	968
	32	2.918e-2	2827.44	1.656	12000	2000	7.384	1.5541	1.146e-2	0.9582	0.8686	1986

More details on deploying large-scale operators in the chip verification task

In this chip verification task described in the main text, the large-scale realistic dataset consists of 1495802 operators and 3380910 logical operator connections. We map this dataset into an undirected weighted graph with 1495802 nodes and 3380910 edges. The information of this original graph is listed in Fig. S3(a). Note that, in the original graph, there are many edges with weights larger than 1, which differs from the other graphs used in bMinCut benchmarking.

Besides, since the realistic datasets in this task often have locally connected structures among operators, meaning many nodes form local clusters or community structures, we consider applying the coarsening trick to reduce the graph size for the sake of performance and speed. A good option is to coarsen the graph based on its natural community structure. We group the nodes within the same community into a new community node, hierarchically contracting the original graph. We refer to the coarsened graph as the community graph. Note that in the community graph, the community nodes now have a node weight larger than 1, equal to the number of nodes grouped in the community. The weight of the community edges connecting different community nodes is thus the sum of the original edges that have endpoints in different communities.

To effectively identify the community structure in a large-scale graph, we apply the Louvain algorithm [58], which was developed for community detection. The information of the community graphs output from the Louvain algorithm, corresponding to level 2 and level 3, is also listed in Fig. S3(a). The modularity metric of the original graph is 0.961652. Fig. S3(b)

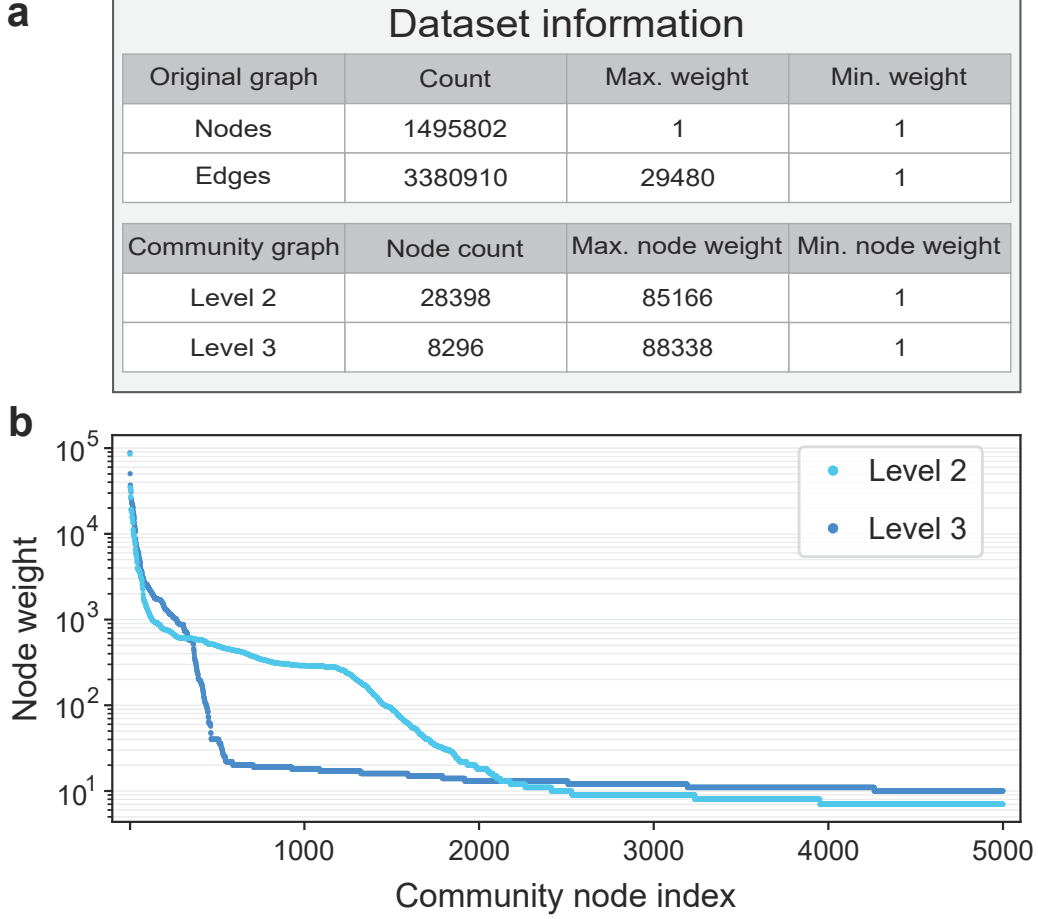


FIG. S3. **The detailed information of the large-scale realistic dataset used in this chip verification task.** (a). The information of the original graph and the two community graphs given by the Louvain algorithm. (b). The top 5000 community nodes with the largest node weight in the two community graphs.

shows the top 5000 community nodes with the largest node weight in the two community graphs. Note that the coarsening technique used to generate coarsened graphs here is different from the methods based on maximal matching techniques employed in METIS.

Since FEM now has to address the issue of ensuring the balanced constraint on nodes with different node weights, a slight modification on the constraint term in the formula of mean-field free energy should be made, which is

$$\lambda \sum_{i,j} \sum_{\sigma_i} [P_i(\sigma_i) P_j(\sigma_i) - P_i^2(\sigma_i)] \rightarrow \lambda \sum_{i,j} \sum_{\sigma_i} [V_i V_j P_i(\sigma_i) P_j(\sigma_i) - V_i^2 P_i^2(\sigma_i)], \quad (\text{S33})$$

where V_i is the node weight of the i -th node. The corresponding modifications on gradients are

$$g_i^p(\sigma_i) : 2\lambda [\sum_j P_j(\sigma_i) - P_i(\sigma_i)] \rightarrow 2\lambda V_i [\sum_j V_j P_j(\sigma_i) - V_i P_i(\sigma_i)], \quad (\text{S34})$$

$$\hat{g}_i^p(\sigma_i) : \lambda c_i^a [\sum_j e_j(\sigma_i) - e_i(\sigma_i)] \rightarrow \lambda c_i^a [\sum_j V_j e_j(\sigma_i) - V_i e_i(\sigma_i)], \quad (\text{S35})$$

while keeping other things unchanged.

For better performance, we choose the community graph for partitioning according to the different values of q . For $q = 4, 8$, we use the graph “level 3”. For $q = 16$, we use the graph “level 2”. The main reason is that the larger the value of q , the smaller the largest group size due to the balanced constraint. Therefore, we have to pick the community graph with a smaller maximum node weight to give enough room for the search. If the community graph has a maximum node weight larger than the allowed largest group size, then the balanced constraint condition with respect to the original graph will always be violated.

Numerical experiments on the Max k -SAT problem

For the benchmarking experiments targeting the Max k -SAT problem, we collated the reported data encompassing the best-known results and corresponding shortest computation times from the most efficient incomplete solvers. These solvers were evaluated against 454 random instances from the random MaxSAT 2016 competition problems [61], with the summarized data presented in Tab. S4. The incomplete solvers included are SsMonteCarlo, Ramp, borealis, SC2016, CCLS, CnC-LS, Swcca-ms, HS-Greedy, and CCEHC. For the FEM approach, we have also compiled the summarized results in Tab. S4 and the hyperparameter settings in Tab. S5. Still, throughout the experiments, we initialize the local fields with random values according to $h_i(\sigma_i)^{ini} = 0.001 * \text{randn}$, where randn represents a random number sampled from the standard Gaussian distribution. The implementation of FEM was executed on a GPU to ensure efficient computation. All variables are represented using 32-bit single-precision floating-point numbers.

TABLE S4: **The results for FEM in the benchmarking on the random MaxSAT 2016 competition problems.** RC time refers to the abbreviation of replica computation time, and $\Delta_E = E_{\min} - E_{\text{bkr}}$. $E_{\min}$ is the minimal energy found by FEM, and E_{bkr} is the best known value in the literature.

Index	Instance	E_{bkr}	Fastest solver	Solver's time(sec.)	$E_{\min}$	Δ_E	RC time(sec.)
1	s2v120c2600-2	458	SsMonteCarlo	1.75	458	0	0.055
2	s2v120c2600-3	440	SsMonteCarlo	1.34	440	0	0.055
3	s2v120c2600-1	439	SsMonteCarlo	1.41	439	0	0.053
4	s2v120c2500-2	435	Ramp	1.36	435	0	0.055
5	s2v120c2500-3	425	SsMonteCarlo	1.62	425	0	0.056
6	s2v140c2600-2	425	Ramp	2.0	425	0	0.049
7	s2v140c2600-1	422	Ramp	2.12	422	0	0.056
8	s2v120c2500-1	418	SsMonteCarlo	1.53	418	0	0.056
9	s2v160c2600-3	413	borealis	2.12	413	0	0.055
10	s2v140c2500-2	407	SC2016	2.08	407	0	0.049
11	s2v140c2600-3	406	Ramp	1.29	406	0	0.058
12	s2v120c2400-2	402	SsMonteCarlo	1.28	402	0	0.055
13	s2v140c2500-3	398	Ramp	1.95	398	0	0.055
14	s2v160c2600-2	397	SC2016	2.61	397	0	0.055
15	s2v140c2500-1	393	Ramp	1.68	393	0	0.056
16	s2v160c2600-1	392	CCLS	2.59	392	0	0.055
17	s2v120c2400-1	389	borealis	1.3	389	0	0.055
18	s2v140c2400-1	388	SsMonteCarlo	1.45	388	0	0.057
19	s2v140c2400-2	388	CnC-LS	1.45	388	0	0.056
20	s2v120c2300-2	383	SsMonteCarlo	1.27	383	0	0.075
21	s2v160c2500-1	383	borealis	2.35	383	0	0.051
22	s2v160c2500-3	382	borealis	2.31	382	0	0.054
23	s2v140c2400-3	381	CnC-LS	1.16	381	0	0.088
24	s2v120c2300-1	380	Ramp	1.44	380	0	0.055
25	s2v120c2400-3	380	SsMonteCarlo	1.31	380	0	0.056
26	s2v160c2400-3	380	SC2016	2.64	380	0	0.048
27	s2v160c2500-2	380	CCLS	2.67	380	0	0.055
28	s2v120c2200-2	371	SsMonteCarlo	1.29	371	0	0.049
29	s2v160c2400-1	370	SC2016	1.84	370	0	0.055

to be continued...

Index	Instance	E_{bkr}	Fastest solver	Solver's time(sec.)	E_{min}	Δ_E	RC time(sec.)
30	s2v120c2300-3	365	SsMonteCarlo	1.42	365	0	0.055
31	s2v140c2300-2	365	SsMonteCarlo	1.46	365	0	0.056
32	s2v160c2400-2	365	CCLS	1.83	365	0	0.055
33	s2v120c2200-3	359	SsMonteCarlo	1.31	359	0	0.054
34	s2v140c2300-1	359	SsMonteCarlo	1.63	359	0	0.088
35	s2v120c2200-1	358	CCLS	1.18	358	0	0.049
36	s2v160c2300-1	353	SC2016	2.25	353	0	0.056
37	s2v140c2300-3	352	SsMonteCarlo	1.53	352	0	0.055
38	s2v160c2300-2	349	Ramp	2.32	349	0	0.049
39	s2v160c2300-3	343	SC2016	2.16	343	0	0.087
40	s2v120c2100-1	336	SsMonteCarlo	1.23	336	0	0.045
41	s2v120c2100-2	336	CCLS	1.02	336	0	0.049
42	s2v140c2200-1	336	Ramp	1.57	336	0	0.086
43	s2v160c2200-2	333	Swcca-ms	2.25	333	0	0.056
44	s2v120c2100-3	332	SsMonteCarlo	1.25	332	0	0.056
45	s2v140c2200-2	332	SsMonteCarlo	1.35	332	0	0.055
46	s2v180c2200-1	330	Swcca-ms	2.76	330	0	0.047
47	s2v140c2200-3	329	SsMonteCarlo	1.43	329	0	0.054
48	s2v160c2200-3	326	SC2016	2.3	326	0	0.064
49	s2v120c2000-2	321	CCLS	1.29	321	0	0.06
50	s2v140c2100-3	319	SsMonteCarlo	1.59	319	0	0.047
51	s2v180c2200-2	318	CnC-LS	2.51	318	0	0.053
52	s2v180c2200-4	317	Swcca-ms	2.17	317	0	0.053
53	s2v140c2100-2	310	SsMonteCarlo	1.73	310	0	0.058
54	s2v160c2200-1	310	CCLS	2.29	310	0	0.054
55	s2v140c2000-2	308	SsMonteCarlo	1.38	308	0	0.047
56	s2v120c2000-1	307	CCLS	1.22	307	0	0.053
57	s2v120c2000-3	307	CCLS	1.19	307	0	0.053
58	s2v180c2200-3	306	Ramp	2.12	306	0	0.054
59	s2v160c2100-3	305	HS-Greedy	2.56	305	0	0.054
60	s2v180c2100-4	303	Ramp	3.17	303	0	0.054
61	s2v180c2100-2	301	Swcca-ms	2.83	301	0	0.054
62	s2v160c2100-2	299	HS-Greedy	1.69	299	0	0.054
63	s2v140c2000-1	298	SsMonteCarlo	1.5	298	0	0.053
64	s2v120c1900-2	296	CCLS	0.86	296	0	0.05
65	s2v140c2000-3	296	SsMonteCarlo	1.39	296	0	0.045
66	s2v120c1900-3	294	CCLS	1.04	294	0	0.047
67	s2v160c2100-1	294	CCLS	1.93	294	0	0.053
68	s2v180c2100-1	294	Swcca-ms	1.97	294	0	0.053
69	s2v120c1900-1	293	CCLS	1.25	293	0	0.056

to be continued...

Index	Instance	E_{bkr}	Fastest solver	Solver's time(sec.)	E_{min}	Δ_E	RC time(sec.)
70	s2v140c2100-1	293	SsMonteCarlo	1.66	293	0	0.045
71	s2v160c2000-1	293	CCLS	2.44	293	0	0.054
72	s2v180c2100-3	293	borealis	2.65	293	0	0.054
73	s2v120c1800-1	291	Swcca-ms	1.25	291	0	0.046
74	s2v180c2000-2	285	Swcca-ms	1.76	285	0	0.046
75	s2v140c1900-1	283	SsMonteCarlo	1.33	283	0	0.048
76	s2v160c2000-3	282	Swcca-ms	1.87	282	0	0.047
77	s2v180c2000-1	282	CCLS	2.36	282	0	0.054
78	s2v180c2000-4	280	SC2016	2.69	280	0	0.053
79	s2v120c1800-3	279	CCLS	0.96	279	0	0.048
80	s2v140c1900-3	278	SsMonteCarlo	1.46	278	0	0.053
81	s2v140c1900-2	273	SsMonteCarlo	1.42	273	0	0.056
82	s2v160c2000-2	272	SC2016	2.59	272	0	0.053
83	s2v180c2000-3	271	Swcca-ms	2.3	271	0	0.048
84	s2v160c1900-3	264	Ramp	2.2	264	0	0.047
85	s2v120c1800-2	262	CCLS	1.21	262	0	0.053
86	s2v160c1900-2	262	borealis	2.57	262	0	0.047
87	s2v160c1900-1	261	SC2016	2.7	261	0	0.054
88	s2v120c1700-1	257	CCLS	1.19	257	0	0.053
89	s2v140c1800-2	257	SsMonteCarlo	1.42	257	0	0.053
90	s2v180c1900-4	257	SsMonteCarlo	3.34	257	0	0.049
91	s2v160c1800-2	256	Swcca-ms	1.81	256	0	0.053
92	s2v140c1800-3	255	SsMonteCarlo	1.31	255	0	0.049
93	s2v160c1800-3	255	Ramp	2.1	255	0	0.053
94	s2v140c1700-1	254	SsMonteCarlo	1.43	254	0	0.046
95	s2v140c1800-1	254	SsMonteCarlo	1.43	254	0	0.056
96	s2v160c1800-1	252	SC2016	2.15	252	0	0.054
97	s2v180c1900-3	252	CnC-LS	3.03	252	0	0.047
98	s2v160c1700-3	249	borealis	2.13	249	0	0.054
99	s2v120c1700-2	248	CnC-LS	1.35	248	0	0.053
100	s2v180c1900-2	246	Swcca-ms	1.76	246	0	0.046
101	s2v180c1900-1	245	Swcca-ms	2.58	245	0	0.047
102	s2v140c1700-2	242	SsMonteCarlo	1.36	242	0	0.048
103	s2v180c1800-2	242	Swcca-ms	1.55	242	0	0.054
104	s2v180c1800-1	241	Swcca-ms	2.69	241	0	0.046
105	s2v180c1800-4	241	Ramp	2.24	241	0	0.054
106	s2v120c1600-2	239	CCLS	1.25	239	0	0.053
107	s2v120c1700-3	239	CCLS	1.17	239	0	0.046
108	s2v140c1700-3	236	SsMonteCarlo	1.54	236	0	0.056
109	s2v120c1600-1	233	SsMonteCarlo	1.12	233	0	0.055

to be continued...

Index	Instance	E_{bkr}	Fastest solver	Solver's time(sec.)	E_{min}	Δ_E	RC time(sec.)
110	s2v120c1600-3	233	SsMonteCarlo	1.06	233	0	0.045
111	s2v180c1800-3	232	Swcca-ms	2.31	232	0	0.055
112	s2v160c1700-2	230	CCEHC	2.22	230	0	0.053
113	s2v180c1700-2	229	SsMonteCarlo	2.38	229	0	0.046
114	s2v200c1800-3	229	CCLS	2.75	229	0	0.053
115	s2v200c1800-6	228	CnC-LS	2.61	228	0	0.053
116	s2v140c1600-3	226	SsMonteCarlo	1.15	226	0	0.048
117	s2v160c1700-1	226	borealis	2.25	226	0	0.048
118	s2v200c1800-7	226	borealis	2.64	226	0	0.053
119	s2v180c1700-1	225	Swcca-ms	2.31	225	0	0.053
120	s2v200c1800-2	223	SC2016	2.44	223	0	0.054
121	s2v160c1600-1	222	borealis	2.21	222	0	0.057
122	s2v160c1600-2	222	Swcca-ms	2.24	222	0	0.049
123	s2v140c1600-1	221	SsMonteCarlo	1.21	221	0	0.053
124	s2v140c1600-2	221	Ramp	1.32	221	0	0.053
125	s2v200c1800-4	221	borealis	3.15	221	0	0.056
126	s2v200c1800-5	220	CnC-LS	2.57	220	0	0.046
127	s2v200c1700-5	217	Ramp	2.17	217	0	0.05
128	s2v180c1600-3	216	SC2016	1.91	216	0	0.052
129	s2v120c1500-2	213	CCLS	1.43	213	0	0.047
130	s2v160c1600-3	213	SC2016	2.47	213	0	0.047
131	s2v200c1700-3	213	CnC-LS	3.05	213	0	0.053
132	s2v140c1500-3	212	Ramp	1.77	212	0	0.086
133	s2v200c1700-4	212	CnC-LS	2.39	212	0	0.047
134	s2v120c1500-1	211	CCLS	1.02	211	0	0.047
135	s2v180c1700-3	209	SC2016	2.58	209	0	0.047
136	s2v200c1700-6	209	Swcca-ms	2.5	209	0	0.053
137	s2v120c1500-3	207	Swcca-ms	1.13	207	0	0.072
138	s2v200c1700-2	207	SC2016	3.49	207	0	0.053
139	s2v180c1700-4	206	borealis	2.83	206	0	0.047
140	s2v200c1800-1	206	Ramp	2.84	206	0	0.054
141	s2v140c1500-1	205	Ramp	1.24	205	0	0.046
142	s2v180c1600-2	204	Swcca-ms	1.31	204	0	0.053
143	s2v200c1700-7	201	Ramp	1.48	201	0	0.053
144	s2v160c1500-2	200	Swcca-ms	1.49	200	0	0.053
145	s2v180c1600-1	200	CnC-LS	2.46	200	0	0.084
146	s2v180c1600-4	200	borealis	2.59	200	0	0.085
147	s2v140c1500-2	199	SsMonteCarlo	1.29	199	0	0.051
148	s2v120c1400-1	197	CCLS	1.04	197	0	0.053
149	s2v200c1600-1	195	Swcca-ms	2.32	195	0	0.053

to be continued...

Index	Instance	E_{bkr}	Fastest solver	Solver's time(sec.)	E_{min}	Δ_E	RC time(sec.)
150	s2v200c1600-4	195	CnC-LS	1.58	195	0	0.087
151	s2v200c1600-5	194	HS-Greedy	1.82	194	0	0.053
152	s2v140c1400-3	193	SsMonteCarlo	1.39	193	0	0.046
153	s2v200c1700-1	193	CnC-LS	1.76	193	0	0.045
154	s2v200c1600-7	192	Swcca-ms	2.66	192	0	0.085
155	s2v120c1400-2	191	CCLS	1.22	191	0	0.053
156	s2v120c1400-3	189	CCLS	1.1	189	0	0.045
157	s2v200c1600-3	186	CCEHC	2.85	186	0	0.052
158	s2v160c1500-3	184	CnC-LS	1.35	184	0	0.057
159	s2v180c1500-1	184	CCEHC	2.53	184	0	0.053
160	s2v160c1500-1	183	Ramp	2.03	183	0	0.053
161	s2v140c1400-1	182	SsMonteCarlo	1.19	182	0	0.056
162	s2v180c1500-4	181	borealis	2.03	181	0	0.054
163	s2v120c1300-1	180	CCLS	1.03	180	0	0.061
164	s2v180c1500-2	180	CnC-LS	1.7	180	0	0.053
165	s2v200c1500-6	180	SC2016	2.66	180	0	0.085
166	s2v180c1500-3	179	CCLS	2.45	179	0	0.052
167	s2v200c1500-7	179	Ramp	1.59	179	0	0.053
168	s2v200c1600-6	179	CnC-LS	2.29	179	0	0.046
169	s2v140c1400-2	178	SsMonteCarlo	1.24	178	0	0.049
170	s2v200c1500-4	178	SC2016	2.78	178	0	0.047
171	s2v200c1600-2	178	CnC-LS	2.99	178	0	0.052
172	s2v200c1500-5	174	CnC-LS	2.93	174	0	0.048
173	s2v120c1300-3	173	SsMonteCarlo	1.17	173	0	0.052
174	s2v120c1300-2	172	Swcca-ms	1.12	172	0	0.051
175	s2v140c1300-2	171	SsMonteCarlo	1.37	171	0	0.051
176	s2v140c1300-3	168	SsMonteCarlo	1.39	168	0	0.1
177	s2v160c1300-1	167	SC2016	1.97	167	0	0.052
178	s2v160c1400-3	167	CnC-LS	1.62	167	0	0.048
179	s2v160c1400-1	166	CCEHC	1.97	166	0	0.048
180	s2v180c1400-1	165	CnC-LS	2.2	165	0	0.052
181	s2v180c1400-2	165	Ramp	2.74	165	0	0.048
182	s2v200c1500-1	165	CCEHC	2.24	165	0	0.053
183	s2v200c1500-2	165	CnC-LS	1.78	165	0	0.052
184	s2v200c1500-3	165	CnC-LS	2.71	165	0	0.053
185	s2v160c1400-2	164	SC2016	2.13	164	0	0.047
186	s2v200c1400-4	164	SC2016	2.48	164	0	0.052
187	s2v140c1300-1	162	Swcca-ms	0.71	162	0	0.047
188	s2v120c1200-1	161	CCLS	1.06	161	0	0.047
189	s2v120c1200-3	160	SsMonteCarlo	1.1	160	0	0.045

to be continued...

Index	Instance	E_{bkr}	Fastest solver	Solver's time(sec.)	E_{min}	Δ_E	RC time(sec.)
190	s2v180c1400-4	160	SC2016	2.84	160	0	0.053
191	s2v120c1200-2	159	Swcca-ms	1.0	159	0	0.061
192	s2v160c1300-3	157	CCLS	2.2	157	0	0.045
193	s2v180c1400-3	156	CnC-LS	1.98	156	0	0.053
194	s2v200c1400-1	156	SC2016	2.61	156	0	0.052
195	s2v140c1200-2	155	SsMonteCarlo	1.17	155	0	0.09
196	s2v140c1200-3	155	SsMonteCarlo	1.26	155	0	0.045
197	s2v180c1300-1	153	SsMonteCarlo	1.69	153	0	0.05
198	s2v200c1400-5	153	SsMonteCarlo	2.11	153	0	0.085
199	s2v180c1300-3	150	Ramp	2.51	150	0	0.05
200	s2v200c1400-3	150	Swcca-ms	2.67	150	0	0.051
201	s2v180c1300-4	149	borealis	2.6	149	0	0.051
202	s2v200c1400-2	147	SC2016	2.7	148	1	0.018
203	s2v200c1400-6	145	SC2016	2.76	145	0	0.053
204	s2v140c1200-1	144	SsMonteCarlo	1.12	144	0	0.045
205	s2v200c1300-1	144	Ramp	2.63	144	0	0.083
206	s2v160c1200-3	143	Ramp	1.68	143	0	0.051
207	s2v200c1300-2	143	Swcca-ms	2.5	143	0	0.082
208	s2v160c1200-1	142	Ramp	2.16	142	0	0.045
209	s2v200c1300-5	142	Swcca-ms	2.54	142	0	0.046
210	s2v200c1400-7	142	CnC-LS	2.4	142	0	0.053
211	s2v160c1300-2	139	Ramp	1.94	139	0	0.046
212	s2v200c1300-4	139	CnC-LS	2.28	139	0	0.057
213	s2v180c1300-2	138	CnC-LS	2.46	138	0	0.052
214	s2v180c1200-1	134	CCLS	1.99	134	0	0.052
215	s2v200c1300-6	134	Swcca-ms	2.4	134	0	0.052
216	s2v160c1200-2	133	Ramp	1.76	133	0	0.052
217	s2v180c1200-3	131	borealis	2.19	131	0	0.048
218	s2v180c1200-2	130	Ramp	1.5	130	0	0.052
219	s2v180c1200-4	128	CCLS	2.01	128	0	0.044
220	s2v200c1300-7	128	borealis	2.73	128	0	0.051
221	s2v200c1200-2	127	borealis	2.39	127	0	0.046
222	s2v200c1200-6	127	CnC-LS	1.34	127	0	0.046
223	s2v200c1200-3	125	SC2016	2.27	125	0	0.057
224	s2v200c1300-3	121	CnC-LS	1.89	121	0	0.094
225	s2v200c1200-1	118	CnC-LS	2.03	118	0	0.052
226	s2v200c1200-5	117	Ramp	2.12	117	0	0.045
227	s2v200c1200-4	115	CnC-LS	1.92	115	0	0.052
228	s2v200c1200-7	112	Swcca-ms	2.35	112	0	0.05
229	s3v70c1500-5	94	Swcca-ms	0.43	94	0	0.055

to be continued...

Index	Instance	E_{bkr}	Fastest solver	Solver's time(sec.)	E_{min}	Δ_E	RC time(sec.)
230	s3v70c1500-4	91	Swcca-ms	1.24	91	0	0.053
231	s3v70c1500-1	90	Ramp	0.96	90	0	0.055
232	s3v70c1500-2	89	CnC-LS	0.96	89	0	0.055
233	s3v70c1500-3	86	borealis	0.91	86	0	0.055
234	s3v70c1400-2	83	borealis	0.86	83	0	0.053
235	s3v70c1400-5	82	Swcca-ms	0.96	82	0	0.053
236	s3v70c1400-3	79	Ramp	1.03	79	0	0.054
237	s3v70c1400-1	78	borealis	1.11	78	0	0.053
238	s3v70c1300-1	75	borealis	1.17	75	0	0.054
239	s3v70c1400-4	75	Swcca-ms	0.5	75	0	0.046
240	s3v70c1300-2	74	CnC-LS	0.69	74	0	0.047
241	s3v70c1300-3	70	CnC-LS	0.95	70	0	0.054
242	s3v70c1300-5	70	CnC-LS	0.98	70	0	0.046
243	s3v70c1300-4	68	Swcca-ms	0.63	68	0	0.053
244	s3v70c1200-4	67	borealis	0.92	67	0	0.053
245	s3v70c1200-1	66	CnC-LS	0.64	66	0	0.053
246	s3v70c1200-3	65	Swcca-ms	0.55	65	0	0.054
247	s3v70c1200-5	65	Ramp	1.12	65	0	0.053
248	s3v90c1300-2	64	CnC-LS	1.17	64	0	0.047
249	s3v90c1300-3	64	SC2016	1.41	64	0	0.053
250	s3v70c1200-2	63	CnC-LS	0.69	63	0	0.054
251	s3v90c1300-4	63	SC2016	1.62	63	0	0.054
252	s3v90c1300-7	63	Swcca-ms	1.04	63	0	0.047
253	s3v90c1300-1	60	SC2016	0.98	60	0	0.054
254	s3v90c1200-5	59	CnC-LS	1.19	59	0	0.051
255	s3v90c1300-6	59	borealis	1.07	59	0	0.054
256	s3v90c1300-5	58	Swcca-ms	0.75	58	0	0.05
257	s3v70c1100-1	56	CCLS	0.85	56	0	0.051
258	s3v70c1100-2	55	HS-Greedy	1.11	55	0	0.053
259	s3v90c1200-4	55	Ramp	1.43	55	0	0.055
260	s3v90c1200-6	55	CCLS	0.98	55	0	0.055
261	s3v90c1200-7	55	HS-Greedy	1.12	55	0	0.048
262	s3v90c1200-2	54	borealis	1.28	54	0	0.06
263	s3v70c1100-3	53	Swcca-ms	1.15	53	0	0.053
264	s3v70c1100-5	53	Swcca-ms	0.88	53	0	0.054
265	s3v70c1100-4	52	Ramp	1.05	52	0	0.055
266	s3v90c1200-3	50	Swcca-ms	0.6	50	0	0.054
267	s3v70c1000-1	47	CCEHC	0.84	47	0	0.096
268	s3v70c1000-4	47	Ramp	0.59	47	0	0.054
269	s3v90c1100-3	47	SsMonteCarlo	1.29	47	0	0.052

to be continued...

Index	Instance	E_{bkr}	Fastest solver	Solver's time(sec.)	E_{min}	Δ_E	RC time(sec.)
270	s3v90c1100-6	47	CnC-LS	0.97	47	0	0.056
271	s3v90c1100-7	47	borealis	0.99	47	0	0.056
272	s3v90c1200-1	47	Swcca-ms	0.83	47	0	0.054
273	s3v70c1000-3	45	CnC-LS	1.03	45	0	0.084
274	s3v90c1100-1	44	CnC-LS	1.08	44	0	0.053
275	s3v90c1100-2	44	CnC-LS	1.32	44	0	0.046
276	s3v90c1100-5	44	borealis	1.27	44	0	0.06
277	s3v70c1000-2	43	Swcca-ms	0.99	43	0	0.053
278	s3v90c1100-4	43	borealis	1.23	43	0	0.048
279	s3v70c1000-5	42	CCLS	1.14	42	0	0.053
280	s3v90c1000-5	41	CnC-LS	1.04	41	0	0.053
281	s3v90c1000-6	41	Swcca-ms	1.15	41	0	0.053
282	s3v70c900-5	40	borealis	0.96	40	0	0.05
283	s3v110c1100-4	39	Swcca-ms	1.08	39	0	0.058
284	s3v110c1100-5	39	Swcca-ms	0.9	39	0	0.053
285	s3v70c900-1	39	CnC-LS	1.04	39	0	0.048
286	s3v70c900-3	39	Swcca-ms	0.85	39	0	0.051
287	s3v70c900-4	39	borealis	1.08	39	0	0.045
288	s3v90c1000-3	39	Ramp	0.98	39	0	0.053
289	s3v110c1100-3	38	CnC-LS	1.36	38	0	0.078
290	s3v110c1100-6	38	SC2016	1.48	38	0	0.059
291	s3v110c1100-7	38	borealis	1.5	38	0	0.082
292	s3v70c900-2	38	Ramp	0.86	38	0	0.054
293	s3v90c1000-2	38	Ramp	1.02	38	0	0.053
294	s3v90c1000-7	38	CnC-LS	0.93	38	0	0.054
295	s3v110c1100-2	37	CnC-LS	1.41	37	0	0.098
296	s3v110c1100-10	36	borealis	1.32	36	0	0.053
297	s3v110c1100-9	36	SsMonteCarlo	1.17	36	0	0.053
298	s3v110c1100-8	35	Ramp	1.5	35	0	0.046
299	s3v110c1100-1	34	CnC-LS	0.92	34	0	0.051
300	s3v70c800-2	34	Ramp	0.58	34	0	0.054
301	s3v90c1000-1	34	borealis	1.24	34	0	0.05
302	s3v90c1000-4	34	CnC-LS	0.91	34	0	0.047
303	s3v110c1000-6	33	borealis	1.5	33	0	0.054
304	s3v90c900-3	32	borealis	0.97	32	0	0.084
305	s3v110c1000-10	31	borealis	1.19	31	0	0.084
306	s3v110c1000-7	31	CnC-LS	1.38	31	0	0.057
307	s3v70c800-1	31	Swcca-ms	0.8	31	0	0.081
308	s3v70c800-5	31	CnC-LS	0.58	31	0	0.082
309	s3v110c1000-1	30	CCLS	1.04	30	0	0.085

to be continued...

Index	Instance	E_{bkr}	Fastest solver	Solver's time(sec.)	E_{min}	Δ_E	RC time(sec.)
310	s3v110c1000-5	30	CnC-LS	1.28	30	0	0.055
311	s3v70c800-3	30	Ramp	1.07	30	0	0.064
312	s3v90c900-6	30	Swcca-ms	1.03	30	0	0.054
313	s3v110c1000-4	29	CnC-LS	1.15	29	0	0.052
314	s3v110c1000-9	29	borealis	1.36	29	0	0.085
315	s3v110c1000-3	28	CnC-LS	1.06	28	0	0.049
316	s3v70c800-4	28	CnC-LS	0.7	28	0	0.06
317	s3v90c900-1	28	borealis	1.31	28	0	0.073
318	s3v90c900-2	28	Swcca-ms	1.15	28	0	0.054
319	s3v90c900-4	28	Swcca-ms	0.86	28	0	0.053
320	s3v110c1000-2	27	CnC-LS	1.17	27	0	0.054
321	s3v90c900-7	27	Swcca-ms	1.19	27	0	0.047
322	s3v110c1000-8	26	CnC-LS	0.96	26	0	0.055
323	s3v110c900-9	26	Ramp	1.33	26	0	0.055
324	s3v90c800-6	26	borealis	1.25	26	0	0.05
325	s3v70c700-3	25	CnC-LS	0.64	25	0	0.065
326	s3v90c900-5	25	Swcca-ms	1.06	25	0	0.054
327	s3v70c700-4	24	CnC-LS	0.9	24	0	0.056
328	s3v90c800-5	24	borealis	0.99	24	0	0.051
329	s3v90c800-7	24	borealis	1.1	24	0	0.053
330	s3v110c900-6	23	Ramp	1.52	23	0	0.054
331	s3v90c800-2	23	CnC-LS	0.9	23	0	0.053
332	s3v90c800-3	23	CnC-LS	1.1	23	0	0.053
333	s3v110c900-1	22	borealis	1.25	22	0	0.085
334	s3v110c900-2	22	borealis	1.13	22	0	0.085
335	s3v110c900-5	22	borealis	1.05	22	0	0.061
336	s3v110c900-8	22	Ramp	1.25	22	0	0.057
337	s3v70c700-5	22	HS-Greedy	0.64	22	0	0.048
338	s3v90c800-1	22	borealis	0.91	22	0	0.053
339	s3v110c900-10	21	CnC-LS	1.22	21	0	0.06
340	s3v110c900-4	21	CnC-LS	1.09	21	0	0.061
341	s3v110c900-7	21	CnC-LS	1.09	21	0	0.052
342	s3v70c700-1	21	HS-Greedy	0.67	21	0	0.052
343	s3v70c700-2	21	Ramp	0.63	21	0	0.052
344	s3v90c800-4	20	CCLS	0.94	20	0	0.051
345	s3v110c800-4	18	CnC-LS	0.54	18	0	0.051
346	s3v110c800-6	18	CnC-LS	1.3	18	0	0.083
347	s3v90c700-4	18	CCLS	0.96	18	0	0.06
348	s3v90c700-6	18	CnC-LS	0.98	18	0	0.058
349	s3v90c700-7	18	CnC-LS	1.04	18	0	0.052

to be continued...

Index	Instance	E_{bkr}	Fastest solver	Solver's time(sec.)	E_{min}	Δ_E	RC time(sec.)
350	s3v110c800-1	17	CnC-LS	1.14	17	0	0.074
351	s3v110c800-2	17	CnC-LS	0.91	17	0	0.051
352	s3v110c800-5	17	CnC-LS	1.34	17	0	0.052
353	s3v110c800-8	17	CnC-LS	1.08	17	0	0.083
354	s3v110c900-3	17	CnC-LS	1.19	17	0	0.065
355	s3v90c700-2	17	Ramp	1.11	17	0	0.052
356	s3v110c800-7	16	CnC-LS	1.22	16	0	0.052
357	s3v110c800-9	16	CnC-LS	0.96	16	0	0.052
358	s3v90c700-1	16	borealis	1.08	16	0	0.052
359	s3v90c700-5	16	CCLS	1.01	16	0	0.085
360	s3v110c800-3	15	CnC-LS	0.82	15	0	0.052
361	s3v90c700-3	15	CnC-LS	0.59	15	0	0.06
362	s3v110c800-10	14	CnC-LS	1.55	14	0	0.052
363	s3v110c700-2	12	Swcca-ms	0.59	12	0	0.073
364	s3v110c700-1	11	CnC-LS	0.97	11	0	0.055
365	s3v110c700-4	11	CnC-LS	0.98	11	0	0.052
366	s3v110c700-6	11	CnC-LS	1.23	11	0	0.053
367	s3v110c700-8	11	Ramp	1.0	11	0	0.08
368	s3v110c700-3	10	Ramp	0.91	10	0	0.067
369	s3v110c700-5	10	CnC-LS	1.04	10	0	0.054
370	s3v110c700-9	10	CnC-LS	0.92	10	0	0.057
371	s3v110c700-10	9	CnC-LS	0.49	9	0	0.062
372	s3v110c700-7	9	CnC-LS	0.61	9	0	0.052
373	HG-3SAT-V250-C1000-20	7	CnC-LS	1.01	7	0	0.29
374	HG-3SAT-V300-C1200-10	7	CnC-LS	1.7	7	0	0.24
375	HG-3SAT-V300-C1200-12	7	CnC-LS	1.1	7	0	0.12
376	HG-3SAT-V300-C1200-15	7	CnC-LS	1.32	7	0	0.09
377	HG-3SAT-V300-C1200-17	7	CnC-LS	2.01	7	0	0.11
378	HG-3SAT-V300-C1200-18	7	CnC-LS	1.2	7	0	0.11
379	HG-3SAT-V300-C1200-23	7	CnC-LS	1.32	7	0	0.12
380	HG-3SAT-V300-C1200-24	7	CnC-LS	1.68	7	0	0.12
381	HG-3SAT-V300-C1200-4	7	CnC-LS	1.71	7	0	0.11
382	HG-3SAT-V300-C1200-8	7	CnC-LS	1.01	7	0	0.12
383	HG-3SAT-V300-C1200-9	7	CnC-LS	1.04	7	0	0.11
384	HG-3SAT-V250-C1000-10	6	CnC-LS	1.44	6	0	0.085
385	HG-3SAT-V250-C1000-100	6	CnC-LS	1.59	6	0	0.084
386	HG-3SAT-V250-C1000-11	6	CnC-LS	1.42	6	0	0.085
387	HG-3SAT-V250-C1000-12	6	CnC-LS	1.0	6	0	0.085
388	HG-3SAT-V250-C1000-17	6	CnC-LS	1.07	6	0	0.12
389	HG-3SAT-V250-C1000-18	6	CnC-LS	1.78	6	0	0.19

to be continued...

Index	Instance	E_{bkr}	Fastest solver	Solver's time(sec.)	E_{min}	Δ_E	RC time(sec.)
390	HG-3SAT-V250-C1000-23	6	CnC-LS	1.09	6	0	0.32
391	HG-3SAT-V250-C1000-4	6	CnC-LS	1.46	6	0	0.33
392	HG-3SAT-V250-C1000-5	6	CnC-LS	1.18	6	0	0.33
393	HG-3SAT-V250-C1000-6	6	CnC-LS	0.81	6	0	0.35
394	HG-3SAT-V250-C1000-7	6	CnC-LS	1.3	6	0	0.34
395	HG-3SAT-V250-C1000-9	6	CnC-LS	1.54	6	0	0.33
396	HG-3SAT-V300-C1200-1	6	CnC-LS	1.71	7	1	0.11
397	HG-3SAT-V300-C1200-100	6	CnC-LS	1.56	7	1	0.13
398	HG-3SAT-V300-C1200-11	6	CnC-LS	2.09	6	0	0.22
399	HG-3SAT-V300-C1200-13	6	CnC-LS	1.27	6	0	0.24
400	HG-3SAT-V300-C1200-14	6	CnC-LS	1.81	7	1	0.1
401	HG-3SAT-V300-C1200-16	6	CnC-LS	1.93	6	0	0.13
402	HG-3SAT-V300-C1200-19	6	CnC-LS	2.14	6	0	0.11
403	HG-3SAT-V300-C1200-20	6	CnC-LS	1.29	6	0	0.24
404	HG-3SAT-V300-C1200-22	6	CnC-LS	1.6	6	0	0.22
405	HG-3SAT-V300-C1200-3	6	CnC-LS	1.73	6	0	0.11
406	HG-3SAT-V300-C1200-5	6	CnC-LS	1.66	6	0	0.11
407	HG-3SAT-V300-C1200-6	6	CnC-LS	1.42	6	0	0.12
408	HG-3SAT-V250-C1000-1	5	CnC-LS	1.32	5	0	0.076
409	HG-3SAT-V250-C1000-13	5	CnC-LS	1.65	5	0	0.092
410	HG-3SAT-V250-C1000-14	5	CnC-LS	1.39	5	0	1.17
411	HG-3SAT-V250-C1000-15	5	CnC-LS	1.41	5	0	0.081
412	HG-3SAT-V250-C1000-16	5	CnC-LS	1.51	5	0	0.13
413	HG-3SAT-V250-C1000-19	5	CnC-LS	1.18	5	0	0.29
414	HG-3SAT-V250-C1000-2	5	CnC-LS	1.18	5	0	0.28
415	HG-3SAT-V250-C1000-22	5	CnC-LS	1.12	5	0	0.34
416	HG-3SAT-V250-C1000-24	5	CnC-LS	1.09	5	0	0.35
417	HG-3SAT-V250-C1000-3	5	CnC-LS	0.74	5	0	0.33
418	HG-3SAT-V250-C1000-8	5	CnC-LS	0.9	5	0	0.34
419	HG-3SAT-V300-C1200-2	5	CnC-LS	1.55	5	0	0.55
420	HG-3SAT-V300-C1200-21	5	CnC-LS	1.24	5	0	0.23
421	HG-3SAT-V300-C1200-7	5	CnC-LS	1.36	5	0	0.11
422	HG-3SAT-V250-C1000-21	4	CnC-LS	1.08	4	0	0.29
423	HG-4SAT-V100-C900-14	2	CnC-LS	0.83	2	0	0.051
424	HG-4SAT-V100-C900-19	2	CnC-LS	0.71	2	0	0.052
425	HG-4SAT-V100-C900-2	2	CnC-LS	0.72	2	0	0.037
426	HG-4SAT-V100-C900-20	2	CnC-LS	0.77	2	0	0.041
427	HG-4SAT-V100-C900-23	2	CnC-LS	0.86	2	0	0.045
428	HG-4SAT-V100-C900-4	2	borealis	0.92	2	0	0.036
429	HG-4SAT-V100-C900-7	2	borealis	0.82	2	0	0.055

to be continued...

Index	Instance	E_{bkr}	Fastest solver	Solver's time(sec.)	E_{min}	ΔE	RC time(sec.)
430	HG-4SAT-V150-C1350-12	2	Swcca-ms	1.04	2	0	0.12
431	HG-4SAT-V150-C1350-17	2	CnC-LS	1.4	2	0	0.09
432	HG-4SAT-V150-C1350-24	2	CnC-LS	1.21	2	0	0.11
433	HG-4SAT-V150-C1350-5	2	CnC-LS	1.25	2	0	0.091
434	HG-4SAT-V150-C1350-8	2	CnC-LS	1.2	2	0	0.082
435	HG-4SAT-V150-C1350-9	2	CnC-LS	1.07	2	0	0.091
436	HG-4SAT-V150-C1350-1	1	borealis	1.49	1	0	0.055
437	HG-4SAT-V150-C1350-10	1	SC2016	1.2	1	0	0.057
438	HG-4SAT-V150-C1350-100	1	CnC-LS	1.1	1	0	0.081
439	HG-4SAT-V150-C1350-11	1	CnC-LS	1.25	1	0	0.1
440	HG-4SAT-V150-C1350-13	1	CnC-LS	1.27	1	0	0.083
441	HG-4SAT-V150-C1350-14	1	CnC-LS	1.65	1	0	0.093
442	HG-4SAT-V150-C1350-15	1	Swcca-ms	1.52	1	0	0.082
443	HG-4SAT-V150-C1350-16	1	CnC-LS	1.22	1	0	0.1
444	HG-4SAT-V150-C1350-18	1	CnC-LS	0.93	1	0	0.12
445	HG-4SAT-V150-C1350-19	1	CnC-LS	1.74	1	0	0.12
446	HG-4SAT-V150-C1350-2	1	CnC-LS	1.07	1	0	0.11
447	HG-4SAT-V150-C1350-20	1	SC2016	1.67	2	1	0.052
448	HG-4SAT-V150-C1350-21	1	CnC-LS	0.91	2	1	0.051
449	HG-4SAT-V150-C1350-22	1	CnC-LS	1.1	1	0	0.064
450	HG-4SAT-V150-C1350-3	1	CnC-LS	1.33	1	0	0.083
451	HG-4SAT-V150-C1350-4	1	CnC-LS	1.13	1	0	0.085
452	HG-4SAT-V150-C1350-6	1	CnC-LS	0.44	1	0	0.092
453	HG-4SAT-V150-C1350-7	1	CnC-LS	1.45	1	0	0.083
454	HG-4SAT-V150-C1350-23	0	Ramp	0.25	0	0	0.1

TABLE S5: **The hyperparameter settings for FEM in the benchmarking on the random MaxSAT 2016 competition problems.** Throughout the experiments, we employed the RMSprop as the optimizer. The inverse-proportional scheduling is used for the annealing.

Index	Instance	N_{step}	N_{replica}	T_{min}	T_{max}	γ_{grad}	lr	alpha	weight decay	momentum
1	s2v120c2600-2	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
2	s2v120c2600-3	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
3	s2v120c2600-1	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
4	s2v120c2500-2	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
5	s2v120c2500-3	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
6	s2v140c2600-2	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
7	s2v140c2600-1	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
8	s2v120c2500-1	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
9	s2v160c2600-3	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9

to be continued...

Index	Instance	N_{step}	N_{replica}	T_{min}	T_{max}	γ_{grad}	lr	alpha	weight decay	momen- tum
10	s2v140c2500-2	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
11	s2v140c2600-3	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
12	s2v120c2400-2	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
13	s2v140c2500-3	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
14	s2v160c2600-2	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
15	s2v140c2500-1	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
16	s2v160c2600-1	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
17	s2v120c2400-1	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
18	s2v140c2400-1	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
19	s2v140c2400-2	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
20	s2v120c2300-2	60	50	0.0001	0.47	1.46	1.05	0.33	0.028	0.87
21	s2v160c2500-1	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
22	s2v160c2500-3	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
23	s2v140c2400-3	80	50	0.0001	0.6	1.2	0.83	0.3	0.02	0.9
24	s2v120c2300-1	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
25	s2v120c2400-3	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
26	s2v160c2400-3	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
27	s2v160c2500-2	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
28	s2v120c2200-2	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
29	s2v160c2400-1	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
30	s2v120c2300-3	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
31	s2v140c2300-2	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
32	s2v160c2400-2	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
33	s2v120c2200-3	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
34	s2v140c2300-1	80	50	0.0001	0.6	1.2	0.83	0.3	0.02	0.9
35	s2v120c2200-1	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
36	s2v160c2300-1	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
37	s2v140c2300-3	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
38	s2v160c2300-2	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
39	s2v160c2300-3	80	50	0.0001	0.6	1.2	0.83	0.3	0.02	0.9
40	s2v120c2100-1	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
41	s2v120c2100-2	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
42	s2v140c2200-1	80	50	0.0001	0.71	0.93	1.23	0.31	0.014	0.83
43	s2v160c2200-2	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
44	s2v120c2100-3	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
45	s2v140c2200-2	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
46	s2v180c2200-1	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
47	s2v140c2200-3	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
48	s2v160c2200-3	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
49	s2v120c2000-2	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9

to be continued...

Index	Instance	N_{step}	N_{replica}	T_{min}	T_{max}	γ_{grad}	lr	alpha	weight decay	momen- tum
50	s2v140c2100-3	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
51	s2v180c2200-2	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
52	s2v180c2200-4	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
53	s2v140c2100-2	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
54	s2v160c2200-1	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
55	s2v140c2000-2	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
56	s2v120c2000-1	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
57	s2v120c2000-3	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
58	s2v180c2200-3	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
59	s2v160c2100-3	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
60	s2v180c2100-4	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
61	s2v180c2100-2	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
62	s2v160c2100-2	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
63	s2v140c2000-1	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
64	s2v120c1900-2	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
65	s2v140c2000-3	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
66	s2v120c1900-3	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
67	s2v160c2100-1	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
68	s2v180c2100-1	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
69	s2v120c1900-1	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
70	s2v140c2100-1	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
71	s2v160c2000-1	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
72	s2v180c2100-3	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
73	s2v120c1800-1	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
74	s2v180c2000-2	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
75	s2v140c1900-1	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
76	s2v160c2000-3	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
77	s2v180c2000-1	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
78	s2v180c2000-4	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
79	s2v120c1800-3	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
80	s2v140c1900-3	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
81	s2v140c1900-2	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
82	s2v160c2000-2	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
83	s2v180c2000-3	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
84	s2v160c1900-3	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
85	s2v120c1800-2	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
86	s2v160c1900-2	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
87	s2v160c1900-1	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
88	s2v120c1700-1	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
89	s2v140c1800-2	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9

to be continued...

Index	Instance	N_{step}	N_{replica}	T_{min}	T_{max}	γ_{grad}	lr	alpha	weight decay	momen- tum
90	s2v180c1900-4	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
91	s2v160c1800-2	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
92	s2v140c1800-3	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
93	s2v160c1800-3	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
94	s2v140c1700-1	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
95	s2v140c1800-1	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
96	s2v160c1800-1	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
97	s2v180c1900-3	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
98	s2v160c1700-3	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
99	s2v120c1700-2	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
100	s2v180c1900-2	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
101	s2v180c1900-1	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
102	s2v140c1700-2	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
103	s2v180c1800-2	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
104	s2v180c1800-1	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
105	s2v180c1800-4	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
106	s2v120c1600-2	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
107	s2v120c1700-3	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
108	s2v140c1700-3	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
109	s2v120c1600-1	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
110	s2v120c1600-3	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
111	s2v180c1800-3	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
112	s2v160c1700-2	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
113	s2v180c1700-2	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
114	s2v200c1800-3	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
115	s2v200c1800-6	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
116	s2v140c1600-3	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
117	s2v160c1700-1	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
118	s2v200c1800-7	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
119	s2v180c1700-1	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
120	s2v200c1800-2	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
121	s2v160c1600-1	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
122	s2v160c1600-2	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
123	s2v140c1600-1	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
124	s2v140c1600-2	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
125	s2v200c1800-4	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
126	s2v200c1800-5	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
127	s2v200c1700-5	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
128	s2v180c1600-3	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
129	s2v120c1500-2	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9

to be continued...

Index	Instance	N_{step}	N_{replica}	T_{min}	T_{max}	γ_{grad}	lr	alpha	weight decay	momen- tum
130	s2v160c1600-3	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
131	s2v200c1700-3	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
132	s2v140c1500-3	80	50	0.0001	0.71	0.93	1.23	0.31	0.014	0.83
133	s2v200c1700-4	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
134	s2v120c1500-1	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
135	s2v180c1700-3	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
136	s2v200c1700-6	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
137	s2v120c1500-3	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
138	s2v200c1700-2	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
139	s2v180c1700-4	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
140	s2v200c1800-1	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
141	s2v140c1500-1	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
142	s2v180c1600-2	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
143	s2v200c1700-7	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
144	s2v160c1500-2	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
145	s2v180c1600-1	80	50	0.0001	0.6	1.2	0.83	0.3	0.02	0.9
146	s2v180c1600-4	80	50	0.0001	0.6	1.2	0.83	0.3	0.02	0.9
147	s2v140c1500-2	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
148	s2v120c1400-1	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
149	s2v200c1600-1	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
150	s2v200c1600-4	80	50	0.0001	0.83	1.0	0.73	0.3	0.02	0.9
151	s2v200c1600-5	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
152	s2v140c1400-3	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
153	s2v200c1700-1	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
154	s2v200c1600-7	80	50	0.0001	0.83	1.0	0.73	0.3	0.02	0.9
155	s2v120c1400-2	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
156	s2v120c1400-3	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
157	s2v200c1600-3	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
158	s2v160c1500-3	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
159	s2v180c1500-1	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
160	s2v160c1500-1	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
161	s2v140c1400-1	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
162	s2v180c1500-4	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
163	s2v120c1300-1	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
164	s2v180c1500-2	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
165	s2v200c1500-6	80	50	0.0001	0.83	1.0	0.73	0.3	0.02	0.9
166	s2v180c1500-3	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
167	s2v200c1500-7	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
168	s2v200c1600-6	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
169	s2v140c1400-2	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9

to be continued...

Index	Instance	N_{step}	N_{replica}	T_{min}	T_{max}	γ_{grad}	lr	alpha	weight decay	momen- tum
170	s2v200c1500-4	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
171	s2v200c1600-2	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
172	s2v200c1500-5	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
173	s2v120c1300-3	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
174	s2v120c1300-2	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
175	s2v140c1300-2	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
176	s2v140c1300-3	100	50	0.0001	0.62	0.92	0.92	0.3	0.02	0.9
177	s2v160c1300-1	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
178	s2v160c1400-3	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
179	s2v160c1400-1	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
180	s2v180c1400-1	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
181	s2v180c1400-2	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
182	s2v200c1500-1	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
183	s2v200c1500-2	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
184	s2v200c1500-3	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
185	s2v160c1400-2	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
186	s2v200c1400-4	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
187	s2v140c1300-1	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
188	s2v120c1200-1	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
189	s2v120c1200-3	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
190	s2v180c1400-4	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
191	s2v120c1200-2	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
192	s2v160c1300-3	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
193	s2v180c1400-3	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
194	s2v200c1400-1	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
195	s2v140c1200-2	100	50	0.0001	0.62	0.92	0.92	0.3	0.02	0.9
196	s2v140c1200-3	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
197	s2v180c1300-1	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
198	s2v200c1400-5	80	50	0.0001	0.83	1.0	0.73	0.3	0.02	0.9
199	s2v180c1300-3	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
200	s2v200c1400-3	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
201	s2v180c1300-4	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
202	s2v200c1400-2	20	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
203	s2v200c1400-6	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
204	s2v140c1200-1	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
205	s2v200c1300-1	80	50	0.0001	0.6	1.2	0.83	0.3	0.02	0.9
206	s2v160c1200-3	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
207	s2v200c1300-2	80	50	0.0001	0.6	1.2	0.83	0.3	0.02	0.9
208	s2v160c1200-1	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
209	s2v200c1300-5	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9

to be continued...

Index	Instance	N_{step}	N_{replica}	T_{min}	T_{max}	γ_{grad}	lr	alpha	weight decay	momen- tum
210	s2v200c1400-7	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
211	s2v160c1300-2	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
212	s2v200c1300-4	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
213	s2v180c1300-2	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
214	s2v180c1200-1	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
215	s2v200c1300-6	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
216	s2v160c1200-2	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
217	s2v180c1200-3	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
218	s2v180c1200-2	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
219	s2v180c1200-4	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
220	s2v200c1300-7	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
221	s2v200c1200-2	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
222	s2v200c1200-6	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
223	s2v200c1200-3	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
224	s2v200c1300-3	80	50	0.0001	0.6	1.2	0.83	0.3	0.02	0.9
225	s2v200c1200-1	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
226	s2v200c1200-5	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
227	s2v200c1200-4	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
228	s2v200c1200-7	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
229	s3v70c1500-5	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
230	s3v70c1500-4	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
231	s3v70c1500-1	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
232	s3v70c1500-2	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
233	s3v70c1500-3	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
234	s3v70c1400-2	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
235	s3v70c1400-5	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
236	s3v70c1400-3	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
237	s3v70c1400-1	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
238	s3v70c1300-1	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
239	s3v70c1400-4	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
240	s3v70c1300-2	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
241	s3v70c1300-3	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
242	s3v70c1300-5	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
243	s3v70c1300-4	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
244	s3v70c1200-4	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
245	s3v70c1200-1	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
246	s3v70c1200-3	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
247	s3v70c1200-5	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
248	s3v90c1300-2	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
249	s3v90c1300-3	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9

to be continued...

Index	Instance	N_{step}	N_{replica}	T_{min}	T_{max}	γ_{grad}	lr	alpha	weight decay	momen- tum
250	s3v70c1200-2	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
251	s3v90c1300-4	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
252	s3v90c1300-7	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
253	s3v90c1300-1	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
254	s3v90c1200-5	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
255	s3v90c1300-6	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
256	s3v90c1300-5	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
257	s3v70c1100-1	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
258	s3v70c1100-2	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
259	s3v90c1200-4	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
260	s3v90c1200-6	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
261	s3v90c1200-7	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
262	s3v90c1200-2	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
263	s3v70c1100-3	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
264	s3v70c1100-5	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
265	s3v70c1100-4	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
266	s3v90c1200-3	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
267	s3v70c1000-1	80	50	0.0001	0.63	3.13	2.2	0.16	0.013	0.56
268	s3v70c1000-4	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
269	s3v90c1100-3	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
270	s3v90c1100-6	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
271	s3v90c1100-7	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
272	s3v90c1200-1	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
273	s3v70c1000-3	80	50	0.0001	0.63	3.13	2.2	0.16	0.013	0.56
274	s3v90c1100-1	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
275	s3v90c1100-2	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
276	s3v90c1100-5	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
277	s3v70c1000-2	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
278	s3v90c1100-4	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
279	s3v70c1000-5	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
280	s3v90c1000-5	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
281	s3v90c1000-6	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
282	s3v70c900-5	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
283	s3v110c1100-4	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
284	s3v110c1100-5	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
285	s3v70c900-1	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
286	s3v70c900-3	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
287	s3v70c900-4	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
288	s3v90c1000-3	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
289	s3v110c1100-3	80	50	0.0001	0.83	1.0	0.73	0.3	0.02	0.9

to be continued...

Index	Instance	N_{step}	N_{replica}	T_{min}	T_{max}	γ_{grad}	lr	alpha	weight decay	momen- tum
290	s3v110c1100-6	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
291	s3v110c1100-7	80	50	0.0001	0.83	1.0	0.73	0.3	0.02	0.9
292	s3v70c900-2	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
293	s3v90c1000-2	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
294	s3v90c1000-7	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
295	s3v110c1100-2	80	50	0.0001	0.83	1.0	0.73	0.3	0.02	0.9
296	s3v110c1100-10	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
297	s3v110c1100-9	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
298	s3v110c1100-8	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
299	s3v110c1100-1	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
300	s3v70c800-2	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
301	s3v90c1000-1	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
302	s3v90c1000-4	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
303	s3v110c1000-6	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
304	s3v90c900-3	80	50	0.0001	0.63	3.13	2.2	0.16	0.013	0.56
305	s3v110c1000-10	80	50	0.0001	0.83	1.0	0.73	0.3	0.02	0.9
306	s3v110c1000-7	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
307	s3v70c800-1	80	50	0.0001	0.63	3.13	2.2	0.16	0.013	0.56
308	s3v70c800-5	80	50	0.0001	0.63	3.13	2.2	0.16	0.013	0.56
309	s3v110c1000-1	80	50	0.0001	0.83	1.0	0.73	0.3	0.02	0.9
310	s3v110c1000-5	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
311	s3v70c800-3	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
312	s3v90c900-6	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
313	s3v110c1000-4	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
314	s3v110c1000-9	80	50	0.0001	0.83	1.0	0.73	0.3	0.02	0.9
315	s3v110c1000-3	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
316	s3v70c800-4	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
317	s3v90c900-1	80	50	0.0001	0.63	3.13	2.2	0.16	0.013	0.56
318	s3v90c900-2	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
319	s3v90c900-4	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
320	s3v110c1000-2	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
321	s3v90c900-7	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
322	s3v110c1000-8	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
323	s3v110c900-9	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
324	s3v90c800-6	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
325	s3v70c700-3	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
326	s3v90c900-5	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
327	s3v70c700-4	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
328	s3v90c800-5	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
329	s3v90c800-7	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9

to be continued...

Index	Instance	N_{step}	N_{replica}	T_{min}	T_{max}	γ_{grad}	lr	alpha	weight decay	momen- tum
330	s3v110c900-6	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
331	s3v90c800-2	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
332	s3v90c800-3	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
333	s3v110c900-1	80	50	0.0001	0.83	1.0	0.73	0.3	0.02	0.9
334	s3v110c900-2	80	50	0.0001	0.63	3.13	2.2	0.16	0.013	0.56
335	s3v110c900-5	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
336	s3v110c900-8	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
337	s3v70c700-5	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
338	s3v90c800-1	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
339	s3v110c900-10	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
340	s3v110c900-4	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
341	s3v110c900-7	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
342	s3v70c700-1	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
343	s3v70c700-2	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
344	s3v90c800-4	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
345	s3v110c800-4	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
346	s3v110c800-6	80	50	0.0001	0.83	1.0	0.73	0.3	0.02	0.9
347	s3v90c700-4	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
348	s3v90c700-6	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
349	s3v90c700-7	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
350	s3v110c800-1	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
351	s3v110c800-2	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
352	s3v110c800-5	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
353	s3v110c800-8	80	50	0.0001	0.83	1.0	0.73	0.3	0.02	0.9
354	s3v110c900-3	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
355	s3v90c700-2	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
356	s3v110c800-7	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
357	s3v110c800-9	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
358	s3v90c700-1	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
359	s3v90c700-5	80	50	0.0001	0.63	3.13	2.2	0.16	0.013	0.56
360	s3v110c800-3	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
361	s3v90c700-3	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
362	s3v110c800-10	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
363	s3v110c700-2	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
364	s3v110c700-1	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
365	s3v110c700-4	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
366	s3v110c700-6	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
367	s3v110c700-8	80	50	0.0001	0.83	1.0	0.73	0.3	0.02	0.9
368	s3v110c700-3	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
369	s3v110c700-5	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9

to be continued...

Index	Instance	N_{step}	N_{replica}	T_{min}	T_{max}	γ_{grad}	lr	alpha	weight decay	momen- tum
370	s3v110c700-9	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
371	s3v110c700-10	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
372	s3v110c700-7	50	50	0.0001	0.6	0.8	0.8	0.3	0.02	0.9
373	HG-3SAT-V250-C1000-20	300	100	0.0001	0.83	0.67	0.56	0.1	0.02	0.9
374	HG-3SAT-V300-C1200-10	200	200	0.0001	0.67	0.55	0.39	0.08	0.02	0.91
375	HG-3SAT-V300-C1200-12	100	200	0.0001	0.82	0.56	0.77	0.09	0.02	0.7
376	HG-3SAT-V300-C1200-15	100	100	0.0001	0.7	0.96	0.54	0.12	0.02	0.88
377	HG-3SAT-V300-C1200-17	100	100	0.0001	0.64	0.57	0.56	0.1	0.02	0.9
378	HG-3SAT-V300-C1200-18	100	100	0.0001	0.8	0.71	0.56	0.1	0.02	0.9
379	HG-3SAT-V300-C1200-23	100	200	0.0001	0.56	0.42	0.84	0.07	0.02	0.75
380	HG-3SAT-V300-C1200-24	100	200	0.0001	0.8	0.62	0.56	0.1	0.02	0.9
381	HG-3SAT-V300-C1200-4	100	200	0.0001	0.8	0.85	0.56	0.1	0.02	0.9
382	HG-3SAT-V300-C1200-8	100	200	0.0001	0.76	0.53	0.84	0.1	0.02	0.63
383	HG-3SAT-V300-C1200-9	100	200	0.0001	0.64	0.79	0.49	0.12	0.02	0.92
384	HG-3SAT-V250-C1000-10	80	50	0.0001	0.6	1.01	0.73	0.3	0.02	0.9
385	HG-3SAT-V250-C1000-100	80	50	0.0001	0.6	1.01	0.73	0.3	0.02	0.9
386	HG-3SAT-V250-C1000-11	80	50	0.0001	0.6	1.01	0.73	0.3	0.02	0.9
387	HG-3SAT-V250-C1000-12	80	50	0.0001	0.6	1.01	0.73	0.3	0.02	0.9
388	HG-3SAT-V250-C1000-17	130	80	0.0001	0.8	0.6	0.56	0.1	0.02	0.9
389	HG-3SAT-V250-C1000-18	200	100	0.0001	0.7	1.28	0.75	0.07	0.02	0.83
390	HG-3SAT-V250-C1000-23	300	200	0.0001	0.8	0.85	0.56	0.1	0.02	0.9
391	HG-3SAT-V250-C1000-4	300	200	0.0001	0.64	0.53	0.62	0.12	0.016	0.92
392	HG-3SAT-V250-C1000-5	300	200	0.0001	0.64	0.81	0.56	0.09	0.02	0.9
393	HG-3SAT-V250-C1000-6	300	200	0.0001	0.64	0.81	0.56	0.09	0.02	0.9
394	HG-3SAT-V250-C1000-7	300	200	0.0001	0.64	0.81	0.56	0.09	0.02	0.9
395	HG-3SAT-V250-C1000-9	300	200	0.0001	0.64	0.81	0.56	0.09	0.02	0.9
396	HG-3SAT-V300-C1200-1	100	200	0.0001	0.91	0.76	0.64	0.08	0.02	0.63
397	HG-3SAT-V300-C1200-100	100	200	0.0001	0.67	0.55	0.39	0.08	0.02	0.91
398	HG-3SAT-V300-C1200-11	200	200	0.0001	0.7	0.8	0.73	0.11	0.02	0.63
399	HG-3SAT-V300-C1200-13	200	200	0.0001	0.54	1.29	1.17	0.13	0.02	0.72
400	HG-3SAT-V300-C1200-14	100	100	0.0001	0.7	0.85	0.56	0.1	0.03	0.9
401	HG-3SAT-V300-C1200-16	100	100	0.0001	0.8	0.76	0.56	0.1	0.02	0.9
402	HG-3SAT-V300-C1200-19	100	100	0.0001	0.53	0.55	0.55	0.09	0.02	0.88
403	HG-3SAT-V300-C1200-20	200	200	0.0001	0.8	0.6	0.56	0.1	0.02	0.9
404	HG-3SAT-V300-C1200-22	200	200	0.0001	0.7	0.48	0.56	0.1	0.02	0.9
405	HG-3SAT-V300-C1200-3	100	200	0.0001	0.92	0.59	0.85	0.13	0.02	0.54
406	HG-3SAT-V300-C1200-5	100	200	0.0001	0.8	0.71	0.56	0.1	0.02	0.9
407	HG-3SAT-V300-C1200-6	100	200	0.0001	0.8	0.6	0.56	0.1	0.02	0.9
408	HG-3SAT-V250-C1000-1	80	50	0.0001	0.66	0.7	1.06	0.26	0.035	0.56
409	HG-3SAT-V250-C1000-13	100	50	0.0001	0.83	0.92	0.64	0.3	0.02	0.9

to be continued...

Index	Instance	N_{step}	N_{replica}	T_{min}	T_{max}	γ_{grad}	lr	alpha	weight decay	momen- tum
410	HG-3SAT-V250-C1000-14	500	800	0.0001	0.9	0.6	0.7	0.1	0.02	0.9
411	HG-3SAT-V250-C1000-15	90	50	0.0001	0.64	0.72	0.64	0.12	0.02	0.876
412	HG-3SAT-V250-C1000-16	130	60	0.0001	0.8	0.6	0.56	0.1	0.02	0.9
413	HG-3SAT-V250-C1000-19	300	100	0.0001	0.76	0.85	0.56	0.1	0.02	0.9
414	HG-3SAT-V250-C1000-2	300	100	0.0001	0.8	0.53	0.56	0.1	0.02	0.9
415	HG-3SAT-V250-C1000-22	300	200	0.0001	0.8	0.85	0.56	0.1	0.02	0.9
416	HG-3SAT-V250-C1000-24	300	200	0.0001	0.8	0.6	0.72	0.1	0.02	0.9
417	HG-3SAT-V250-C1000-3	300	200	0.0001	0.8	0.6	0.72	0.1	0.02	0.9
418	HG-3SAT-V250-C1000-8	300	200	0.0001	0.64	0.81	0.56	0.09	0.02	0.9
419	HG-3SAT-V300-C1200-2	500	200	0.0001	0.45	0.94	0.68	0.08	0.03	0.92
420	HG-3SAT-V300-C1200-21	200	200	0.0001	0.73	0.51	0.54	0.13	0.02	0.88
421	HG-3SAT-V300-C1200-7	100	200	0.0001	0.64	0.48	0.54	0.09	0.02	0.88
422	HG-3SAT-V250-C1000-21	300	100	0.0001	0.83	0.81	0.75	0.1	0.02	0.83
423	HG-4SAT-V100-C900-14	50	50	0.0001	0.7	0.85	0.63	0.08	0.02	0.67
424	HG-4SAT-V100-C900-19	50	50	0.0001	0.8	1.03	0.62	0.09	0.02	0.77
425	HG-4SAT-V100-C900-2	50	50	0.0001	0.85	0.66	0.65	0.06	0.03	0.44
426	HG-4SAT-V100-C900-20	50	50	0.0001	0.66	1.03	0.49	0.05	0.03	0.5
427	HG-4SAT-V100-C900-23	50	50	0.0001	1.31	1.26	0.71	0.09	0.03	0.41
428	HG-4SAT-V100-C900-4	50	50	0.0001	0.64	0.48	0.56	0.1	0.02	0.9
429	HG-4SAT-V100-C900-7	50	50	0.0001	0.8	0.6	0.56	0.1	0.02	0.9
430	HG-4SAT-V150-C1350-12	100	100	0.0001	0.64	0.62	0.67	0.1	0.02	0.9
431	HG-4SAT-V150-C1350-17	80	100	0.0001	0.7	0.57	0.71	0.1	0.02	0.71
432	HG-4SAT-V150-C1350-24	100	100	0.0001	0.76	0.85	0.56	0.1	0.02	0.9
433	HG-4SAT-V150-C1350-5	80	100	0.0001	0.8	0.76	0.56	0.1	0.02	0.9
434	HG-4SAT-V150-C1350-8	80	100	0.0001	0.67	0.71	0.69	0.13	0.02	0.62
435	HG-4SAT-V150-C1350-9	80	100	0.0001	0.7	0.53	0.9	0.11	0.03	0.75
436	HG-4SAT-V150-C1350-1	50	50	0.0001	0.83	0.62	0.56	0.1	0.02	0.9
437	HG-4SAT-V150-C1350-10	50	50	0.0001	0.76	0.75	0.77	0.07	0.03	0.75
438	HG-4SAT-V150-C1350-100	70	50	0.0001	0.76	0.85	0.61	0.11	0.02	0.75
439	HG-4SAT-V150-C1350-11	100	100	0.0001	0.76	0.9	0.54	0.12	0.03	0.88
440	HG-4SAT-V150-C1350-13	80	100	0.0001	0.7	0.76	0.56	0.1	0.02	0.9
441	HG-4SAT-V150-C1350-14	80	100	0.0001	0.8	0.6	0.72	0.1	0.02	0.9
442	HG-4SAT-V150-C1350-15	80	100	0.0001	0.83	0.96	0.49	0.12	0.02	0.88
443	HG-4SAT-V150-C1350-16	100	100	0.0001	0.8	0.76	0.56	0.1	0.02	0.9
444	HG-4SAT-V150-C1350-18	100	100	0.0001	0.64	0.81	0.56	0.09	0.03	0.79
445	HG-4SAT-V150-C1350-19	100	100	0.0001	0.8	0.6	0.61	0.1	0.02	0.9
446	HG-4SAT-V150-C1350-2	100	100	0.0001	0.64	0.57	0.61	0.09	0.02	0.9
447	HG-4SAT-V150-C1350-20	50	50	0.0001	0.83	0.64	0.49	0.09	0.02	0.67
448	HG-4SAT-V150-C1350-21	50	50	0.0001	0.9	0.52	0.65	0.09	0.02	0.62
449	HG-4SAT-V150-C1350-22	50	100	0.0001	0.76	0.67	0.62	0.09	0.02	0.75

to be continued...

Index	Instance	N_{step}	N_{replica}	T_{min}	T_{max}	γ_{grad}	lr	alpha	weight decay	momen- tum
450	HG-4SAT-V150-C1350-3	80	100	0.0001	0.8	0.6	0.61	0.1	0.02	0.9
451	HG-4SAT-V150-C1350-4	80	100	0.0001	0.8	0.57	0.56	0.1	0.02	0.9
452	HG-4SAT-V150-C1350-6	80	100	0.0001	0.8	0.6	0.56	0.1	0.02	0.9
453	HG-4SAT-V150-C1350-7	80	100	0.0001	0.67	0.68	0.6	0.1	0.02	0.7
454	HG-4SAT-V150-C1350-23	100	100	0.0001	0.8	0.71	0.56	0.1	0.02	0.9

The mean field equations for the Ising problem derived from the FEM formalism

We introduce reproducing the mean-field equations for the Ising problem from the FEM formalism. For the case of Ising problem with the Ising Hamiltonian $E_{\text{Ising}}(\sigma) = -\frac{1}{2} \sum_{i,j} W_{ij} \sigma_i \sigma_j$, where $\sigma_i \in \{-1, +1\}$, the corresponding mean field free energy $F_{\text{MF}} = U_{\text{MF}} - \frac{1}{\beta} S_{\text{MF}}$ is computed as follows. The internal energy U_{MF} is defined as the expectation of $E_{\text{Ising}}(\sigma)$ regarding to mean field joint probability $P_{\text{MF}}(\sigma) = \prod_i P_i(\sigma_i)$, which reads

$$\begin{aligned}
U_{\text{MF}} &= -\frac{1}{2} \sum_{\sigma} \prod_i P_i(\sigma_i) \sum_{i,j} W_{ij} \sigma_i \sigma_j \\
&= -\frac{1}{2} \sum_{i,j} W_{ij} \left[\sum_{\sigma_i} P_i(\sigma_i) \sigma_i \right] \left[\sum_{\sigma_j} P_j(\sigma_j) \sigma_j \right] \\
&= -\frac{1}{2} \sum_{i,j} W_{ij} m_i m_j,
\end{aligned} \tag{S36}$$

where $m_i = \sum_{\sigma_i} P_i(\sigma_i) \sigma_i = P_i(+1) - P_i(-1)$ is the magnetization (or mean spin). And the entropy S_{MF} reads

$$\begin{aligned}
S_{\text{MF}} &= - \sum_i \sum_{\sigma_i} P_i(\sigma_i) \ln P_i(\sigma_i) \\
&= - \sum_i \sum_{\sigma_i} \frac{1 + m_i \sigma_i}{2} \ln \frac{1 + m_i \sigma_i}{2} \\
&= - \sum_i \left(\frac{1 + m_i}{2} \ln \frac{1 + m_i}{2} + \frac{1 - m_i}{2} \ln \frac{1 - m_i}{2} \right),
\end{aligned} \tag{S37}$$

where the identity relation $P_i(\sigma_i) = \frac{1 + m_i \sigma_i}{2}$ is used. Accordingly, the mean field free energy for the Ising Hamiltonian turns out to be

$$F_{\text{MF}} = -\frac{1}{2} \sum_{i,j} W_{ij} m_i m_j + \frac{1}{\beta} \sum_i \left(\frac{1 + m_i}{2} \ln \frac{1 + m_i}{2} + \frac{1 - m_i}{2} \ln \frac{1 - m_i}{2} \right). \tag{S38}$$

At any given β , the optimal $\{P_i^*(\sigma_i)\}$ that minimizes F_{MF} can be obtained by the zero-gradient equations $\frac{\partial F_{\text{MF}}}{\partial P_i(+1)} = -\frac{\partial F_{\text{MF}}}{\partial P_i(-1)} = 0$ (since $P_i(+1) + P_i(-1) = 1$). And since $P_i(\sigma_i) = \frac{1 + m_i \sigma_i}{2}$, it indicates that the information of all marginal probabilities is completely described by the magnetizations. Thus, it suffices to solve the zero-gradient equations with respect to $\{m_i\}$, which is

$$\begin{aligned}
\frac{\partial F_{\text{MF}}}{\partial m_i} &= 0 = - \sum_j W_{ij} m_j + \frac{1}{\beta} \left(\frac{1}{2} \ln \frac{1 + m_i}{1 - m_i} \right) \\
&= - \sum_j W_{ij} m_j + \frac{1}{\beta} \text{arctanh}(m_i).
\end{aligned} \tag{S39}$$

As a result, we find that

$$m_i = \tanh \left(\beta \sum_j W_{ij} m_j \right), \quad i, j = 1, 2, \dots, N, \tag{S40}$$

which reproduces the mean field equations in the existing mean-field annealing approaches for combinatorial optimization. Then, solving Eq. (S40) by using the fixed-point iteration method with a slow annealing for $\beta \rightarrow \infty$ leads to the solution $\{m_i^*\}$ to Eq. (S40). We can identify the ground state by the operation of $\sigma_i^{GS} = \arg \max_{\sigma_i} P_i^*(\sigma_i) = \text{sign}(m_i^*)$.

Modelings for other combinatorial optimization problems

Building upon the three example problems explored in the main text, we now outline a concise methodology for modeling or formulating additional well-known COPs within the FEM framework. As demonstrated previously, the essential step for solving a given COP using FEM is to precisely define the expectation of the designed cost function with respect to the marginal probabilities. Given the fixed form of the entropy in the mean-field ansatz, calculating the internal energy provides the complete free energy expression that is a function of the marginals. Subsequently, the COP can be tackled using either the automatic differentiation approach or the explicit gradient formulations.

COPs that can be naturally translated into QUBO formulations

Probably the most simple and straightforward ones will be problems that can be naturally translated into QUBO problems (or Ising formulations). The target function of QUBO problems can be written as

$$E(\sigma) = \sigma^T Q \sigma = \sum_{i,j} Q_{ij} \sigma_i \sigma_j . \quad (\text{S41})$$

Here σ is a vector of binary spin variables (0 or 1) and Q is a symmetric matrix encoding the QUBO problems.

The expectation value of the cost function, or U_{MF} , will be

$$\langle E \rangle = \sum_i \sum_j p_i Q_{ij} p_j , \quad (\text{S42})$$

with the p_i represents the marginal probability of spin variable σ_i to take on value 1.

Many kinds of COPs are either naturally QUBO or can be translated into QUBO problems with simple steps. Some prominent such examples are listed below.

1. Vertex cover
2. Maximum Independent Set Problems
3. Binary Integer Linear Programming
4. Set Packing
5. Maximum cliques

Problems of multi-valued variables

The COPs with multi-state variables, commonly referred as Potts model in the statistical physics community, have different target function formulation with the aforementioned QUBO-like problems. In the main text, we have shown a example of balanced minimum cut problem. Another such problem will be the graph coloring problems, which detects whether a coloring configuration of a graph can satisfy that no end points of an edge have the same color. This problem can be transferred into a binary variable case with ancillary extra variables [21], the cost function then will be

$$E(\sigma) = \sum_i \left(1 - \sum_c \sigma_{i,c} \right)^2 + \sum_{(i,j)} \sum_c \sigma_{i,c} \sigma_{j,c} . \quad (\text{S43})$$

Here there are overall $N \times C$ binary variables $\sigma_{i,c}$ with N being the number of vertices, C representing the overall number of colors and $\sigma_{i,c}$ being a variable to determine whether vertex i has the color c . The first term of this cost function is a regularization term to let all vertices to have only one color and the second term is the actual target function which describes the number of violation edges whose two end vertices have the same color.

If we change the framework into the FEM solver, modeling will become much easier with only N multi-state variables and no regularization term. The target function can be written as

$$E(\boldsymbol{\sigma}) = \sum_{(i,j)} \delta(\sigma_i, \sigma_j) , \quad (\text{S44})$$

with σ_i being a multi-valued spin variable which represents the color of vertex i . The expectation value of such target function can be deducted easily as

$$\langle E(\boldsymbol{\sigma}) \rangle = \sum_{(i,j)} \sum_{\sigma_i} p_i(\sigma_i) p_j(\sigma_i) . \quad (\text{S45})$$

With the graph coloring problem being the natural one, there are many other problems can be translated into such formulations. We have listed some of the examples below.

- Hamiltonian Cycles and Paths
- Traveling Salesman
- Community Detection

Problems with multi-variable interactions

For problems like SAT and quantum error corrections (QEC), the target function will encounter terms with multi-variable interactions. In the main text, we have already introduce how to solve Max k -SAT. Here we demonstrate the formulation of QEC that compatible to FEM solver.

The QEC problems can be translated into Ising spin-glass problems with the Hamiltonian of the variable $\sigma_i \in \{-1, +1\}$ [69, 70]

$$H = -J \sum_v^{N_v} b_v \prod_{i \in \delta v}^4 \sigma_i - h \sum_i^{N_d} \sigma_i , \quad (\text{S46})$$

The expectation value of such Hamiltonian can then be written as

$$\begin{aligned} \langle E \rangle &= -J \sum_v^{N_v} b_v \prod_{i \in \delta v}^4 (p_i(+1) - p_i(-1)) - h \sum_i^{N_d} (p_i(+1) - p_i(-1)) \\ &= -J \sum_v^{N_v} b_v \prod_{i \in \delta v}^4 m_i - h \sum_i^{N_d} m_i . \end{aligned} \quad (\text{S47})$$

Here $m_i = p_i(+1) - p_i(-1)$ can be viewed as the magnetization of variable i .